

3

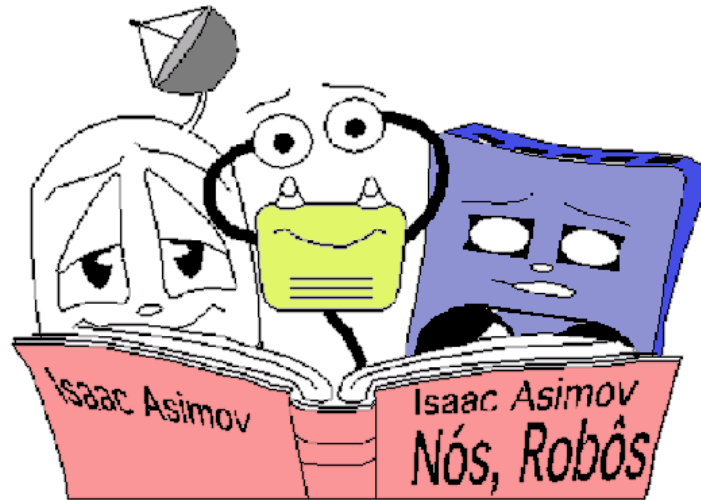
•• série livros didáticos informática ufrgs ••

linguagens formais e autômatos

•• 6ª edição

•• paulo blauth menezes





Introdução

Teoria das Linguagens Formais foi originariamente desenvolvida na década de 1950 com o objetivo de desenvolver teorias relacionadas com as linguagens naturais. Entretanto, logo foi verificado que esta teoria era importante para o estudo de linguagens artificiais e, em especial, para as linguagens originárias na Ciência da Computação. Desde então, o estudo das Linguagens Formais desenvolveu-se significativamente e com diversos enfoques, com destaque para aplicações em Análise Léxica e Sintática de linguagens de programação, modelos de sistemas biológicos, desenhos de circuitos e relacionamentos com linguagens naturais. Recentemente, inclui-se a ênfase no tratamento de Linguagens Não-Lineares, como Planares, Espaciais e n-Dimensionais.

Conceitos Básicos

Sintaxe e Semântica

Linguagens Formais preocupa-se com os problemas sintáticos das linguagens. Assim, inicialmente, é importante introduzir os conceitos de sintaxe e semântica de linguagens.

Historicamente, no estudo do entendimento das linguagens de programação, o problema sintático foi reconhecido antes do problema semântico e foi o primeiro a receber um tratamento adequado. Adicionalmente, os problemas sintáticos são de tratamento mais simples que os semânticos. Como consequência, foi dada uma grande ênfase à sintaxe, ao ponto de levar à idéia de que as questões das linguagens de programação resumiam-se às questões da sintaxe. Atualmente, a teoria da sintaxe possui construções matemáticas bem definidas e universalmente reconhecidas como, por exemplo, as *Gramáticas de Chomsky*.

Uma linguagem de programação (bem como qualquer modelo matemático) pode ser vista de duas formas:

- como uma entidade livre, ou seja, sem qualquer significado associado;
- como uma entidade juntamente com uma interpretação do seu significado.

A *sintaxe* trata das propriedades livres da linguagem como, por exemplo, a verificação gramatical de programas. A *semântica* objetiva dar uma interpretação para a linguagem como, por exemplo, um significado ou valor para um determinado programa. Conseqüentemente, a sintaxe basicamente manipula símbolos sem considerar os seus correspondentes significados. Note-se que, para resolver qualquer problema real, é necessário dar uma interpretação semântica aos símbolos como, por exemplo, "estes símbolos representam os inteiros".

Sintaticamente falando, não existe uma noção de programa "errado": neste caso, simplesmente não é um programa. Por outro lado, um programa sintaticamente válido ("correto"), pode não ser o programa que o programador esperava escrever. Assim, a questão de considerar um programa "correto" ou "errado" deve considerar se o mesmo modela adequadamente o comportamento desejado.

Nem sempre os limites entre a sintaxe e a semântica são claros. Um exemplo é a ocorrência de um nome em um programa o qual pode ser tratado de forma igualmente fácil como um problema sintático ou semântico. Entretanto, a distinção entre sintaxe e semântica em linguagens artificiais é, em geral, óbvia para a maioria dos problemas relevantes.

Abordagem

A abordagem desta publicação é centrada no tratamento sintático de linguagens lineares abstratas com fácil associação às linguagens típicas da Ciência da Computação. Os formalismos usados podem ser classificados nos seguintes tipos:

- a) *Operacional*. Define-se um autômato ou uma máquina abstrata, baseada em estados, em instruções primitivas e na especificação de como cada instrução

modifica cada estado. Uma máquina abstrata deve ser suficientemente simples para não permitir dúvidas sobre a execução de seu código. Também é dito um formalismo *Reconhecedor*, no sentido em que permite a análise de uma dada entrada para verificar se é "reconhecida" pela máquina. As principais máquinas definidas nesta publicação são Autômato Finito, Autômato com Pilha e Máquina de Turing;

b) *Axiomático*. Associam-se regras às componentes da linguagem. As regras permitem afirmar o que será verdadeiro após a ocorrência de cada cláusula considerando o que era verdadeiro antes da ocorrência. A abordagem axiomática que segue é sobre Gramáticas (Regulares, Livres do Contexto, Sensíveis ao Contexto e Irrestritas). Uma gramática também é dita um formalismo *Gerador* no sentido em que permite verificar se um determinado elemento da linguagem é "gerado";

c) *Denotacional*. Também é denominado formalismo *Funcional*. Define-se uma função que caracteriza o conjunto de palavras admissíveis na linguagem. Em geral, trata-se de uma função construída a partir de funções elementares de forma composicional (horizontalmente) no sentido em que a linguagem denotada pela função pode ser determinada em termos de suas funções componentes. Nesta publicação, a abordagem denotacional é restrita às Expressões Regulares. Como, a partir de uma expressão regular, é simples inferir ("gerar") as palavras da linguagem denotada, freqüentemente também é denominado, de forma não muito precisa, como um formalismo *Gerador*.

Organização dos Capítulos e Carga Horária Recomendada

A estruturação e uma breve introdução dos capítulos desta publicação é, resumidamente, a seguinte:

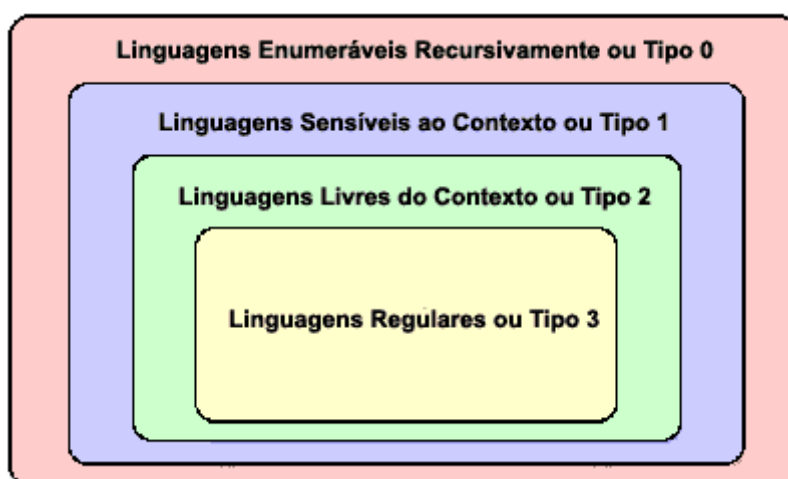
a) *Capítulo 1*. Os demais tópicos deste capítulo introduzem conceitos básicos necessários para o que segue. Note-se que a parte referente a Conjuntos, Relações, Funções, Lógica e Técnicas de Demonstração objetiva realizar uma revisão e normalização de notações e, portanto, considera algum conhecimento prévio e não esgota o assunto. Caso o leitor domine adequadamente estes assuntos, recomenda-se passar diretamente para a seção *Alfabetos, Palavras, Linguagens e Gramáticas*;

b) *Capítulo 2*. As Linguagens Regulares e as noções de Autômato Finito e Expressão Regular são originárias de estudos biológicos de redes de neurônios e circuitos de chaveamentos. Mais recentemente, são usadas para o desenvolvimento de Analisadores Léxicos (parte de um compilador que identifica e codifica as unidades básicas de uma linguagem como variáveis, números, etc.), editores de textos, sistemas de pesquisa e atualização em arquivos (em geral, do tipo busca e substituição de informações não complexas), linguagens simples de comunicação homem-máquina (como interface do sistema operacional) e máquina-máquina (como protocolos de comunicação). Note-se que a análise léxica pode ser considerada como um caso particular e simples de análise sintática;

c) **Capítulo 3.** As Linguagens Livres do Contexto e as correspondentes noções de Gramática Livre do Contexto e Autômato com Pilha são usadas principalmente para o desenvolvimento de Analisadores Sintáticos, uma importante parte de um compilador. Historicamente, o desenvolvimento de analisadores sintáticos era um problema complexo, de difícil depuração e com eficiência relativamente baixa. Hoje, considerando o conhecimento já adquirido relativo às Linguagens Livre do Contexto, o desenvolvimento de um Analisador Sintático é simples (assim como a sua depuração) e somente uma pequena percentagem do tempo de processamento de um compilador é gasto em tal atividade;

d) **Capítulo 4.** As Linguagens Enumeráveis Recursivamente e Sensíveis ao Contexto e as correspondentes noções de Máquina de Turing (e eventuais variações/restrições deste modelo) e as Gramáticas Irrestritas e Sensíveis ao Contexto permitem explorar os limites da capacidade de desenvolvimento de reconhecedores ou geradores de linguagens. Ou seja, estudam a solucionabilidade do problema da existência de algum reconhecedor ou gerador para determinada linguagem;

e) **Capítulo 5.** Conclui os capítulos anteriores, classificando as diversas classes de linguagens em uma ordem hierárquica, denominada Hierarquia de Chomsky (ilustrada na [figura abaixo](#)) e apresenta conclusões gerais e perspectivas futuras.



O trabalho que segue é baseado em experiências letivas no Curso de Bacharelado em Ciência da Computação da Universidade Federal do Rio Grande do Sul. É destinado, principalmente, como um primeiro curso de Linguagens Formais, sendo auto-contido e podendo ser adotado como bibliografia básica. Possui um texto simples, exemplos detalhados e exercícios em níveis crescentes de raciocínio. Embora todos os conceitos necessários sejam introduzidos, recomenda-se como pré-requisitos conhecimentos básicos de lógica, teoria dos conjuntos e algoritmos. No caso de desenvolvimento de um curso, a carga horária recomendada varia de 45 a 90 horas.

Conjuntos, Relações e Funções

No texto que segue é suposto que o leitor está familiarizado com os conceitos básicos relativos à **Teoria dos Conjuntos**.

Conjuntos e Operações sobre Conjuntos

Definição:

Conjunto

Um *Conjunto* é uma coleção de zero ou mais objetos distintos, denominados *Elementos* do conjunto.

Observação:

Um *elemento* é uma entidade básica a qual não é definida formalmente. Relativamente ao relacionamento entre elementos e conjuntos, tem-se que:

- a) Se um elemento a *pertence* a um conjunto A denota-se por $a \in A$; caso contrário, $a \notin A$
- b) Se todos os elementos de um conjunto A também são elementos de um conjunto B , então afirma-se que A está *contido* em B ou que A é *subconjunto* de B e denota-se por $A \subseteq B$ (ou ainda B *contém* A e $B \supseteq A$). Adicionalmente, se existe $b \in B$ tal que $b \notin A$, então afirma-se que A está *contido propriamente* em B ou que A é *subconjunto próprio* de B e denota-se por $A \subset B$ (ou ainda B *contém propriamente* A e $B \supset A$)
- c) Os conjuntos A e B são *iguais* se, e somente se, possuem os mesmos elementos, ou seja, $A = B$ se, e somente se, $A \subseteq B$ e $B \subseteq A$

Observação:

Um conjunto pode possuir um número finito ou infinito de elementos. Os conjuntos finitos podem ser denotados por *extensão*, listando todos os seus elementos entre chaves e em qualquer ordem como, por exemplo:

$$\{ a, b, c \}$$

O conjunto sem elementos (ou seja, com zero elementos) é denominado *conjunto vazio* e é denotado por $\{ \}$ ou $\emptyset$. Conjuntos (finitos ou infinitos) também podem ser denotados por *compreensão* na forma:

$$\{ a \mid a \in A \text{ e } p(a) \} \quad \text{ou} \quad \{ a \in A \mid p(a) \}$$

a qual é interpretada como:

"o conjunto de todos os elementos a pertencentes ao conjunto A tal que $p(a)$ é verdadeiro".

Quando é claro que $a \in A$, pode-se denotar simplesmente na forma:

$$\{ a \mid p(a) \}$$

EXEMPLO:

Conjuntos, Elementos

- a) $a \in \{ b, a \}$ e $c \notin \{ b, a \}$
- b) $\{ a, b \} = \{ b, a \}$, $\{ a, b \} \subseteq \{ b, a \}$ e $\{ a, b \} \subset \{ a, b, c \}$
- c) Os seguintes conjuntos são infinitos:

$\mathbb{N}$ Conjunto dos *Números Naturais*;

$\mathbb{Z}$ Conjunto dos *Números Inteiros*;

$\mathbb{Q}$ Conjunto dos *Números Racionais*;

$\mathbb{I}$ Conjunto dos *Números Irracionais*;

$\mathbb{R}$ Conjunto dos *Números Reais*.

- d) $\{ 1, 2, 3 \} = \{ x \in \mathbb{N} \mid x > 0 \text{ e } x < 4 \}$ e $\mathbb{N} = \{ x \in \mathbb{Z} \mid x \geq 0 \}$

- e) O conjunto dos números pares pode ser denotado por compreensão como segue:

$$\{ y \mid y = 2x \text{ e } x \in \mathbb{N} \}$$

Definição:

União, Intersecção, Diferença, Complemento, Conjunto das Partes, Produto Cartesiano

Sejam A e B conjuntos. Então:

- a) *União*.

$$A \cup B = \{ x \mid x \in A \text{ ou } x \in B \}$$

- b) *Intersecção*.

$$A \cap B = \{ x \mid x \in A \text{ e } x \in B \}$$

- c) *Diferença*.

$$A - B = \{ x \mid x \in A \text{ e } x \notin B \}$$

- d) *Complemento*. A operação de complemento é definida em relação a um conjunto fixo U denominado *conjunto universo*.

$$A' = \{x \mid x \in U \text{ e } x \notin A\}$$

e) *Conjunto das Partes.*

$$2^A = \{S \mid S \subseteq A\}$$

f) *Produto Cartesiano.*

$$A \times B = \{(a, b) \mid a \in A \text{ e } b \in B\}$$

Observação:

É usual denotar um produto cartesiano de um conjunto com ele mesmo como um expoente. Por exemplo:

$$A \times A = A^2$$

Um elemento de um produto cartesiano denotado na forma (a, b) é denominado *par ordenado* e não deve ser confundido com o conjunto $\{a, b\}$: em um par ordenado, a ordem é importante, pois são distinguidas as duas componentes. O conceito de par ordenado pode ser generalizado para n-upla ordenada, ou seja, com $n \geq 0$ componentes.

EXEMPLO:

Operações sobre Conjuntos

Suponha o universo $\mathbb{N}$ e sejam $A = \{0, 1, 2\}$ e $B = \{2, 3\}$. Então:

a) $A \cup B = \{0, 1, 2, 3\}$

b) $A \cap B = \{2\}$

c) $A - B = \{0, 1\}$

d) $A' = \{x \in \mathbb{N} \mid x > 2\}$

e) $2B = \{\emptyset, \{2\}, \{3\}, \{2, 3\}\}$

f) $A \times B = \{(0, 2), (0, 3), (1, 2), (1, 3), (2, 2), (2, 3)\}$

Observação:

As seguintes propriedades das operações sobre conjuntos podem ser facilmente verificadas (suponha o universo U e os conjuntos A , B e C):

a) *Idempotência.*

$$A \cup A = A$$

$$A \cap A = A$$

b) *Comutatividade.*

$$A \cup B = B \cup A$$

$$A \cap B = B \cap A$$

c) *Associatividade.*

$$A \cup (B \cap C) = (A \cup B) \cap C$$

$$A \cap (B \cup C) = (A \cap B) \cup C$$

d) *Distributividade.*

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$

$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$$

e) *Duplo Complemento.*

$$(A')' = A$$

f) *Morgan.*

$$(A \cup B)' = A' \cap B'$$

$$(A \cap B)' = A' \cup B'$$

g) *Universo e Vazio.*

$$A \cup A' = U$$

$$A \cap A' = \emptyset$$

Definição:

Relação

Uma *Relação* (Binária) é um subconjunto de um produto cartesiano.

Suponha os conjuntos A e B e a relação $R \subseteq A \times B$. Então A e B são denominados *domínio* e *contra-domínio* (ou *codomínio*) de R , respectivamente. Um elemento $(a, b) \in R$ é usualmente denotado por $a R b$. Uma relação $R \subseteq A \times A$ (onde o domínio e o contra-domínio coincidem) é dita uma relação em A e, neste caso, é freqüentemente denotada por (A, R) .

Definição:

Relação Reflexiva, Simétrica, Antissimétrica, Transitiva

Sejam A um conjunto e R uma relação em A . Então R é uma:

- a) **Relação Reflexiva**, se, para todo $a \in A$, $a R a$
- b) **Relação Simétrica**, se $a R b$, então $b R a$
- c) **Relação Antissimétrica**, se $a R b$ e $b R a$, então $a = b$
- d) **Relação Transitiva**, se $a R b$ e $b R c$, então $a R c$

Note-se que uma relação pode não ser simétrica nem antissimétrica, ou seja, não são noções complementares. Adicionalmente, uma relação pode ser simultaneamente simétrica e antissimétrica.

EXEMPLO:

Propriedades de Relações

Considere um conjunto não vazio A . Então:

- a) As relações $(\mathbb{N}, \leq)$ e $(2^A, \subseteq)$ são reflexivas, antissimétricas e transitivas;
- b) As relações $(\mathbb{Z}, <)$ e $(2^A, \subset)$ são transitivas;
- c) A relação $(\mathbb{Q}, =)$ é reflexiva, simétrica, antissimétrica e transitiva;
- d) A relação $\{ (1, 2), (2, 1), (2, 3) \}$ **não** é reflexiva, simétrica, antissimétrica e nem transitiva.

Definição:

Relação de Ordem, Ordem Parcial, Ordem Total

Sejam A um conjunto e R uma relação em A . Então R é uma:

- a) **Relação de Ordem**, se é transitiva;
- b) **Relação de Ordem Parcial**, se é reflexiva, antissimétrica e transitiva;
- c) **Relação de Ordem Total**, se é uma relação de ordem parcial e, para todo $a, b \in A$, ou $a R b$ ou $b R a$.

EXEMPLO:

Relação de Ordem, Ordem Parcial, Ordem Total

Considere um conjunto não vazio A . Então:

- a) As relações $(\mathbb{N}, \leq)$, $(2^A, \subseteq)$, $(\mathbb{Z}, <)$, $(2^A, \subset)$ e $(\mathbb{Q}, =)$ são de ordem;
- b) As relações $(\mathbb{N}, \leq)$, $(2^A, \subseteq)$ e $(\mathbb{Q}, =)$ são de ordem parcial;
- c) A relação $(\mathbb{N}, \leq)$ é de ordem total.

Definição:

Relação de Equivalência

Sejam A um conjunto e R uma relação em A . Então R é uma *Relação de Equivalência* se for reflexiva, simétrica e transitiva.

Um importante resultado é que cada relação de equivalência induz um particionamento do conjunto (em que a relação é definida) em subconjuntos disjuntos e não vazios denominados *classes de equivalência*.

EXEMPLO:

Relação de Equivalência

Considere a relação $R = \{(a, b) \in \mathbb{N} \times \mathbb{N} \mid a \text{ MOD } 2 = b \text{ MOD } 2\}$ onde MOD é a operação que resulta no resto da divisão inteira. É fácil verificar que R é uma relação de equivalência. Portanto, R induz um particionamento de $\mathbb{N}$ em dois subconjuntos: pares (resto zero) e ímpares (resto um).

Freqüentemente é desejável estender uma relação de forma a satisfazer determinado conjunto de propriedades.

Definição:

Fecho de uma Relação

Sejam R uma relação e P um conjunto de propriedades. Então, o *Fecho de R em Relação ao P* , denotado por $\text{FECHO-}P(R)$, é a menor relação que contém R e que satisfaz às propriedades em P .

Definição:

Fecho Transitivo, Fecho Transitivo e Reflexivo

Sejam R uma relação em A . Então:

a) O fecho de R em relação ao conjunto de propriedades $\{\text{transitiva}\}$, denominado *Fecho Transitivo de R* e denotado por R^+ , é definido como segue:

a.1) Se $(a, b) \in R$, então $(a, b) \in R^+$;

a.2) Se $(a, b) \in R^+$ e $(b, c) \in R^+$, então $(a, c) \in R^+$;

a.3) Os únicos elementos de R^+ são os construídos como acima;

b) O fecho de R em relação ao conjunto de propriedades $\{ \text{transitiva, reflexiva} \}$, denominado *Fecho Transitivo e Reflexivo* de R e denotado por R^* , é tal que:

$$R^* = R^+ \cup \{ (a, a) \mid a \in A \}$$

EXEMPLO:

Fecho de um Grafo visto como uma Relação

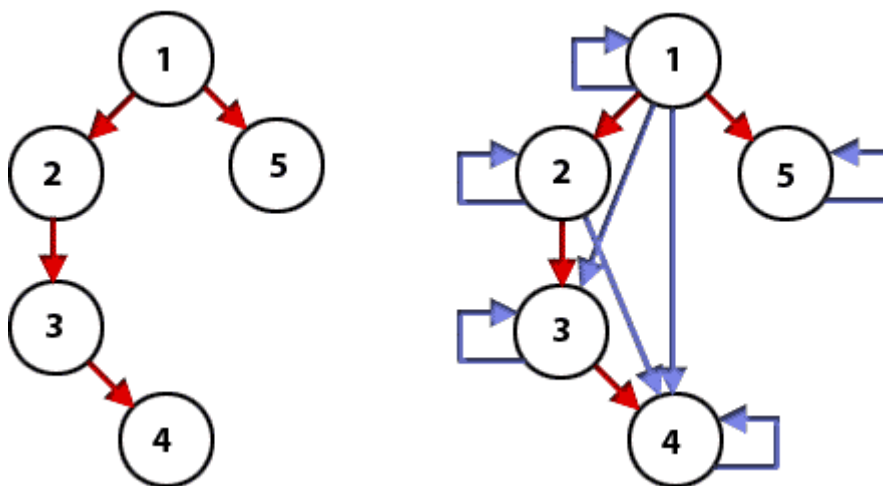
Um grafo (direto) pode ser definido como uma relação A (de arestas) em um conjunto V (de vértices). Assim:

$$A = \{ (1, 2), (2, 3), (3, 4), (1, 5) \}$$

é um grafo em $V = \{ 1, 2, 3, 4, 5 \}$ como ilustrado na *figura à esquerda* onde o par ordenado que define uma aresta é representado por uma seta na qual as circunferências de origem e de destino representam a primeira e a segunda componente do par, respectivamente. O fecho transitivo e reflexivo:

$$A^* = \{ (1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 2), (2, 3), (2, 4), (3, 3), (3, 4), (4, 4), (5, 5) \}$$

é ilustrado na *figura à direita* onde as arestas adicionadas pelo fecho são representadas com um traço diferente.



Definição:

Função Parcial

Uma *Função Parcial* é uma relação $f \subseteq A \times B$ tal que se $(a, b) \in f$ e $(a, c) \in f$, então $b = c$.

Portanto, uma função parcial é uma relação onde cada elemento do domínio está relacionado com, no máximo, um elemento do contra-domínio.

Uma função parcial $f \subseteq A \times B$ normalmente é denotada por $f: A \rightarrow B$. Adicionalmente, $(a, b) \in f$ é usualmente denotado por $f(a) = b$. Se $f(a) = b$ então afirma-se que f está definida para a e que b é *imagem* de a . O conjunto:

$$\{ b \in B \text{ existe } a \in A \text{ tal que } f(a) = b \}$$

é denominado *conjunto imagem* de f e é denotado por $f(A)$ ou $\text{Img}(f)$.

Definição:

Função, Aplicação

Uma *Função (Total)* ou *Aplicação* é uma função parcial $f: A \rightarrow B$ onde para todo $a \in A$ existe $b \in B$ tal que $f(a) = b$.

Portanto, uma função (total) é uma função parcial definida para todos os elementos do domínio.

EXEMPLO:

Função, Função Parcial.

- a) *Função Identidade*. Para um dado conjunto A , a função (total) $\text{id}_A: A \rightarrow A$ é tal que, para todo $a \in A$, $\text{id}_A(a) = a$;
- b) *Adição nos naturais*. A operação $\text{ad}: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ tal que $\text{ad}(a, b) = a + b$ é uma função (total);
- c) *Divisão nos reais*. A operação $\text{div}: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ tal que $\text{div}(x, y) = x/y$ é uma função parcial pois não é definida para $(x, 0)$, qualquer que seja $x \in \mathbb{R}$.

Definição:

Composição de Funções

Sejam $f: A \rightarrow B$ e $g: B \rightarrow C$ funções. A *Composição* de f e g é a função $g \circ f: A \rightarrow C$ tal que $(g \circ f)(a) = g(f(a))$.

Portanto, a composição de duas funções $f: A \rightarrow B$ e $g: B \rightarrow C$ é uma função $g \circ f: A \rightarrow C$ onde $(g \circ f)(a) = g(f(a))$ é a aplicação da função f ao elemento a e, na sequência, da função g à imagem $f(a)$.

EXEMPLO:

Composição de Funções

A composição das funções $\text{ad}: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ e $\text{quadrado}: \mathbb{N} \rightarrow \mathbb{N}$ é a função $\text{quadrado} \circ \text{ad}: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$, para $(3, 1) \in \mathbb{N} \times \mathbb{N}$, tem-se que:

$$(\text{quadrado} \circ \text{ad})(3, 1) = \text{quadrado}(\text{ad}(3, 1)) = \text{quadrado}(4) = 16$$

Ou seja, a composição das funções resulta no quadrado da adição.

Definição:

Função Injetora, Sobrejetora, Bijetora ou Isomorfismo

Uma função $f: A \rightarrow B$ é dita:

- a) *Injetora* se, para todo $b \in B$, existe no máximo um $a \in A$ tal que $f(a) = b$;
- b) *Sobrejetora* se, para todo $b \in B$, existe pelo menos um $a \in A$ tal que $f(a) = b$;
- c) *Bijetora* ou *Isomorfismo* se é injetora e sobrejetora.

Observação:

Portanto, uma função é:

- a) Injetora se cada elemento do contra-domínio é imagem de, no máximo, um elemento do domínio;
- b) Sobrejetora se todo elemento do contra-domínio é imagem de pelo menos um elemento do domínio;
- c) Bijetora ou isomorfismo se todo elemento do contra-domínio é imagem de exatamente um elemento do domínio.

É comum usar os termos injeção, sobrejeção e bijeção ao invés de função injetora, função sobrejetora e função bijetora, respectivamente. Adicionalmente, se existe um isomorfismo (função bijetora) entre dois conjuntos, estes são ditos *conjuntos isomorfos*.

EXEMPLO:

Função Injetora, Sobrejetora, Bijetora, Isomorfismo.

- a) A função *inclusão*: $\mathbb{N} \rightarrow \mathbb{Z}$ tal que $\text{inclusão}(a) = a$ é injetora;
- b) A função *módulo*: $\mathbb{Z} \rightarrow \mathbb{N}$ tal que $\text{módulo}(a) = |a|$ é sobrejetora;
- c) A função $f: \mathbb{Z} \rightarrow \mathbb{N}$ tal que $f(a) = 2a$ se $a \geq 0$ e $f(a) = |2a| - 1$ se $a < 0$ é bijetora, ou seja, é um isomorfismo. Portanto, os conjuntos $\mathbb{Z}$ e $\mathbb{N}$ são isomorfos. Note-se que, os inteiros não-negativos são associados aos naturais pares e os inteiros negativos aos naturais ímpares.

Cardinalidade de Conjuntos

A cardinalidade de um conjunto é uma medida de seu tamanho e é definida usando funções bijetoras.

Definição:

Cardinalidade Finita, Infinita

A *Cardinalidade* de um conjunto A , representada por $\#A$ é:

a) **Finita** se existe uma bijeção entre A e o conjunto $\{1, 2, 3, \dots, n\}$, para algum $n \in \mathbb{N}$. Neste caso, afirma-se que $\#A = n$ (como fica o caso em que $n = 0$?);

b) **Infinita** se existe uma bijeção entre A e um subconjunto próprio de A .

Portanto, um conjunto é finito (ou seja, possui uma cardinalidade finita) se for possível representá-lo por extensão. Um conjunto A é infinito se for possível retirar alguns elementos de A e, mesmo assim, estabelecer uma bijeção com A .

EXEMPLO:

Cardinalidade de $\mathbb{Z}$

A função $f: \mathbb{Z} \rightarrow \mathbb{N}$ tal que $f(a) = 2a$ se $a \geq 0$ e $f(a) = |2a| - 1$ se $a < 0$ é bijetora. Claramente, $\mathbb{N}$ é subconjunto próprio de $\mathbb{Z}$. Então $\mathbb{Z}$ é infinito.

É importante destacar que nem todos os conjuntos infinitos possuem a mesma cardinalidade, o que contradiz a noção intuitiva da maioria das pessoas. De especial interesse é o cardinal do conjunto dos números naturais $\mathbb{N}$, denotado por $\aleph_0$. O símbolo $\aleph$ (lê-se "alef") é a primeira letra do alfabeto hebraico.

Definição:

Conjunto Contável, Não-Contável

Um conjunto infinito A é dito:

a) **Contável** ou **Contavelmente Infinito**, se existe uma bijeção entre o conjunto A e um subconjunto infinito de $\mathbb{N}$;

b) **Não-Contável**, caso contrário.

A bijeção que define se um conjunto A é contável é denominada **enumeração** de A . Portanto, um conjunto é contável se for possível enumerar seus elementos como uma sequência na forma $a_0, a_1, a_2, \dots$. O cardinal de qualquer conjunto contável é $\aleph_0$.

EXEMPLO:

Conjunto Contável, Não-Contável

Os conjuntos $\mathbb{Z}$ e $\mathbb{Q}$ são contáveis. Os conjuntos $\mathbb{I}$ e $\mathbb{R}$ são não-contáveis. De fato, prova-se que o cardinal de $\mathbb{I}$ e $\mathbb{R}$ é $2^{\aleph_0}$.

Lógica

No texto que segue é suposto que o leitor está familiarizado com os conceitos básicos relativos à Lógica Booleana. Entende-se por **Lógica Booleana** como o estudo dos princípios e métodos usados para distinguir sentenças verdadeiras de falsas.

Definição:

Proposição

- a) Uma **Proposição** é uma sentença declarativa a qual possui valor lógico **verdadeiro** ou **falso** (não-verdadeiro);
- b) Considere um conjunto universo U . Uma **Proposição Sobre U** é uma proposição cujo valor lógico depende de um elemento $x \in U$ considerado.

Observação:

Os valores lógicos verdadeiro e falso são usualmente denotados por V e F , respectivamente. Uma proposição p a qual descreve alguma propriedade de um elemento $x \in U$ é usualmente denotada por $p(x)$. Toda a proposição p sobre U induz uma partição de U em duas classes de equivalência, como segue:

- a) $\{x \mid p(x) \text{ é verdadeira}\}$, denominado **conjunto verdade** de p
- b) $\{x \mid p(x) \text{ é falsa}\}$, denominado **conjunto falsidade** de p

Definição:

Tautologia, Contradição

Seja p uma proposição sobre o conjunto universo U . Então:

- a) p é dita uma **Tautologia** se $p(x)$ é verdadeira para qualquer $x \in U$
- b) p é dita uma **Contradição** se $p(x)$ é falsa para qualquer $x \in U$

EXEMPLO:

Proposição, Tautologia, Contradição

- a) A sentença $3 + 4 > 5$ é uma proposição;
- b) Para a proposição $n! < 10$ sobre $\mathbb{N}$, tem-se que $\{0, 1, 2, 3\}$ é o conjunto verdade e $\{n \in \mathbb{N} \mid n > 3\}$ é o conjunto falsidade;
- c) A proposição $n + 1 > n$ sobre $\mathbb{N}$ é uma tautologia;
- d) A proposição " $2n$ é ímpar" sobre $\mathbb{N}$ é uma contradição.

Observação:

Uma **operação** ou um **operador** sobre um conjunto A é uma função da forma $op: A^n \rightarrow A$. Portanto, um **operador lógico**, também denominado **conectivo (lógico)**, é um operador sobre o conjunto das proposições. Uma proposição que não contém operadores é denominada **proposição atômica** ou simplesmente **átomo**. O conjunto de todas as proposições lógicas é denotado por P .

Uma **tabela verdade** é uma tabela que descreve os valores lógicos de uma proposição em termos das possíveis combinações dos valores lógicos das proposições componentes.

Definição:

Operadores Lógicos

Os seguintes **Operadores** ou **Conetivos** sobre o conjunto das proposições lógicas **P** são definidos conforme a tabela verdade ilustrada abaixo:

- a) **Negação**. Operador denotado pelo símbolo $\neg$
- b) **E**. Operador denotado pelo símbolo $\wedge$
- c) **Ou**. Operador denotado pelo símbolo $\vee$
- d) **Se-Então**. Operador denotado pelo símbolo $\rightarrow$
- e) **Se-Somente-Se**. Operador denotado pelo símbolo $\leftrightarrow$

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$
V	V	F	V	V	V	V
V	F	F	F	V	F	F
F	V	V	F	V	V	F
F	F	V	F	F	V	V

Note-se que todos os operadores definidos acima são binários e denotados de forma infixada (operador no meio dos operandos), excetuando-se o de Negação o qual é unário e denotado de forma prefixada.

Os operadores lógicos $\rightarrow$ e $\leftrightarrow$ induzem importantes relações entre proposições.

Definição:

Relação de Implicação, Equivalência

As seguintes relações são induzidas pelos operadores $\rightarrow$ e $\leftrightarrow$ sobre **P**:

- a) A relação $\Rightarrow$, denominada **Relação de Implicação** ou simplesmente **Implicação**, é definida pelo conjunto:

$$\{(p, q) \in P^2 \mid p \rightarrow q \text{ é uma tautologia}\}$$

b) A relação $\Leftrightarrow$, denominada *Relação de Equivalência* ou simplesmente *Equivalência*, é definida pelo conjunto:

$$\{ (p, q) \in P^2 \mid p \Leftrightarrow q \text{ é uma tautologia} \}$$

É fácil verificar que $\Rightarrow$ e $\Leftrightarrow$ são relações de ordem e de equivalência, respectivamente.

EXEMPLO:

Relação de Implicação, Equivalência

Os seguintes pares de proposições pertencem às relações de implicação ou de equivalência, como pode ser verificado pelas tabelas verdade abaixo:

a) *Adição*.

$$p \Rightarrow p \vee q$$

b) *Simplificação*.

$$p \wedge q \Rightarrow p$$

c) *Contraposição*.

$$p \rightarrow q \Leftrightarrow \neg q \rightarrow \neg p$$

d) *Redução ao Absurdo*.

$$p \rightarrow q \Leftrightarrow (p \wedge \neg q) \rightarrow F$$

p	q	$p \vee q$	$p \rightarrow (p \vee q)$	$p \wedge q$	$(p \wedge q) \rightarrow p$
V	V	V	V	V	V
V	F	V	V	F	V
F	V	V	V	F	V
F	F	F	V	F	V

p	q	$\neg p$	$\neg q$	$p \rightarrow q$	$\neg q \rightarrow \neg p$	$(p \rightarrow q) \Leftrightarrow (\neg q \rightarrow \neg p)$
V	V	F	F	V	V	V
V	F	F	V	F	F	V
F	V	V	F	V	V	V

F	F	V	V	V	V	V
---	---	---	---	---	---	---

p	q	$\neg q$	$p \rightarrow q$	$p \wedge \neg q$	$((p \wedge \neg q) \rightarrow F)$	$(p \rightarrow q) \leftrightarrow ((p \wedge \neg q) \rightarrow F)$
V	V	F	V	F	V	V
V	F	V	F	V	F	V
F	V	F	V	F	V	V
F	F	V	V	F	V	V

Técnicas de Demonstração

Um **teorema** é uma proposição do tipo $p \rightarrow q$ a qual prova-se ser verdadeira sempre, ou seja, que $p \Rightarrow q$. As proposições p e q são denominadas **hipótese** e **tese**, respectivamente. É usual denominar por **corolário** um teorema que é uma consequência quase direta de um outro já demonstrado (ou seja, cuja prova é trivial ou imediata). Adicionalmente, um teorema auxiliar que possui um resultado importante para a prova de um outro é usualmente denominado por **lema**.

Dado um teorema a ser demonstrado, é fundamental, antes de iniciar a demonstração, identificar claramente a hipótese e a tese. Por exemplo, considere o seguinte teorema:

a intersecção distribui-se sobre a união, ou seja,

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$

Uma reescrita identificando claramente a hipótese e a tese é como segue:

se A , B e C são conjuntos quaisquer,

$$\text{então } A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$

É usual um teorema ser apresentado na forma $p \leftrightarrow q$ como, por exemplo, (suponha que A é um conjunto qualquer):

A é contável se, e somente se,

existe uma função bijetora entre A e o conjunto dos números pares

Sugere-se como **exercício** verificar que:

$$p \leftrightarrow q \Leftrightarrow (p \rightarrow q) \wedge (q \rightarrow p)$$

Assim, neste caso, deve ser demonstrada a "ida" e a "volta", ou seja, que:

se um conjunto A é contável,

então existe uma função bijetora entre A e o conjunto dos números pares

e

se existe uma função bijetora entre A e o conjunto dos números pares,

então A é contável

Para um determinado teorema $p \rightarrow q$ existem diversas técnicas para provar (demonstrar) que, de fato, $p \Rightarrow q$. As seguintes técnicas destacam-se:

- a) Direta;
- b) Contraposição;
- c) Redução ao absurdo;
- d) Indução.

Prova Direta

A **prova direta** simplesmente pressupõe verdadeira a hipótese e, a partir desta, prova ser verdadeira a tese.

EXEMPLO:

Prova Direta

Considere o teorema:

a intersecção distribui-se sobre a união, ou seja,

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$

exemplificado acima. Inicialmente, lembre-se que:

- por definição, $X = Y$ se, e somente se, $X \subseteq Y$ e $Y \subseteq X$;
- por definição, $X \subseteq Y$ se, e somente se, todos os elementos de X também são elementos de Y .

Adicionalmente, é fácil verificar, usando tabela verdade, que o operador lógico $\wedge$ se distribui sobre o $\vee$, ou seja, para quaisquer proposições p , q e r , tem-se que:

$$p \wedge (q \vee r) \Leftrightarrow (p \wedge q) \vee (p \wedge r)$$

Para provar que $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$, deve-se provar que:

$$A \cap (B \cup C) \subseteq (A \cap B) \cup (A \cap C)$$

$$(A \cap B) \cup (A \cap C) \subseteq A \cap (B \cup C)$$

Suponha que A , B e C são conjuntos quaisquer.

Caso 1. $A \cap (B \cup C) \subseteq (A \cap B) \cup (A \cap C)$. Suponha $x \in A \cap (B \cup C)$. Então:

$$x \in A \cap (B \cup C) \Rightarrow \text{definição de intersecção}$$

$$x \in A \wedge x \in (B \cup C) \Rightarrow \text{definição de união}$$

$$x \in A \wedge (x \in B \vee x \in C) \Rightarrow \text{distributividade do } \wedge \text{ sobre o } \vee$$

$$(x \in A \wedge x \in B) \vee (x \in A \wedge x \in C) \Rightarrow \text{definição de intersecção}$$

$$x \in (A \cap B) \vee x \in (A \cap C) \Rightarrow \text{definição de união}$$

$$x \in (A \cap B) \cup (A \cap C)$$

$$\text{Portanto, } A \cap (B \cup C) \subseteq (A \cap B) \cup (A \cap C)$$

Caso 2. $(A \cap B) \cup (A \cap C) \subseteq A \cap (B \cup C)$. Suponha $x \in (A \cap B) \cup (A \cap C)$. Então:

$$x \in (A \cap B) \cup (A \cap C) \Rightarrow \text{definição de união}$$

$$x \in (A \cap B) \vee x \in (A \cap C) \Rightarrow \text{definição de intersecção}$$

$$(x \in A \wedge x \in B) \vee (x \in A \wedge x \in C) \Rightarrow \text{distributividade do } \wedge \text{ sobre o } \vee$$

$$x \in A \wedge (x \in B \vee x \in C) \Rightarrow \text{definição de união}$$

$$x \in A \wedge x \in (B \cup C) \Rightarrow \text{definição de intersecção}$$

$$x \in A \cap (B \cup C)$$

$$\text{Portanto, } (A \cap B) \cup (A \cap C) \subseteq A \cap (B \cup C)$$

Logo, $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$

Prova por Contraposição

A *prova por contraposição* baseia-se no seguinte resultado:

$$p \rightarrow q \Leftrightarrow \neg q \rightarrow \neg p$$

EXEMPLO:

Prova por Contraposição

Para demonstrar o seguinte teorema:

$$n! > n + 1 \rightarrow n > 2$$

pode-se, equivalentemente, demonstrar por contraposição que:

$$n \leq 2 \rightarrow n! \leq n + 1$$

Note-se que é muito simples provar que $n \leq 2 \rightarrow n! \leq n + 1$ pois é suficiente testar para os casos $n = 0$, $n = 1$ e $n = 2$.

Prova por Redução ao Absurdo

A *prova por redução ao absurdo* ou simplesmente *prova por absurdo* baseia-se no seguinte resultado:

$$p \rightarrow q \Leftrightarrow (p \wedge \neg q) \rightarrow F$$

Ou seja, para demonstrar que $p \rightarrow q$ a técnica consiste em supor a hipótese p , supor a negação da tese $\neg q$ e concluir uma contradição a qual, em geral, é $q \wedge \neg q$. Note-se que, a técnica de demonstração denominada *por contra-exemplo*, é uma demonstração por absurdo. De fato, em uma demonstração por absurdo, a construção da contradição $q \wedge \neg q$ é, em geral, a apresentação de um contra-exemplo.

EXEMPLO:

Prova por Redução ao Absurdo

Considere o seguinte teorema:

0 é o único elemento neutro da adição em $\mathbb{N}$

ou seja, reescrevendo na forma de $p \rightarrow q$:

se 0 é elemento neutro da adição em $\mathbb{N}$,

então 0 é o único elemento neutro da adição em $\mathbb{N}$

Uma prova por redução ao absurdo é como segue:

a) Suponha que 0 é o elemento neutro da adição em $\mathbb{N}$ e que não é o único elemento neutro da adição em $\mathbb{N}$. Seja e um elemento neutro da adição em $\mathbb{N}$ tal que $e \neq 0$;

b) Então:

- como 0 é elemento neutro, para qualquer $n \in \mathbb{N}$, tem-se que $n = 0 + n$. Em particular, para $n = e$, tem-se que $e = 0 + e$;
- como e é elemento neutro, para qualquer $n \in \mathbb{N}$, tem-se que $n = n + e$. Em particular, para $n = 0$, tem-se que $0 = 0 + e$;
- portanto, como $e = 0 + e$ e $0 = 0 + e$, pela transitividade da igualdade tem-se que $e = 0$, o que é uma contradição, pois foi suposto que $e \neq 0$.

Logo, é absurdo supor que o elemento neutro da adição em $\mathbb{N}$ não é único.

Prova por Indução

A prova por indução é de fundamental importância neste trabalho, pois é usada com frequência. A prova por indução é usada em proposições que dependem dos números naturais.

Definição:

Princípio da Indução Matemática

Seja $p(n)$ uma proposição sobre $\mathbb{N}$. O *Princípio da Indução Matemática* é como segue:

- a) $p(0)$ é verdadeira;
- b) Para qualquer $k \in \mathbb{N}$, $p(k) \rightarrow p(k + 1)$ é verdadeira;
- c) Então, para qualquer $n \in \mathbb{N}$, $p(n)$ é verdadeira.

Neste caso, $p(0)$, $p(k)$ e a proposição $p(k) \rightarrow p(k + 1)$ denominam-se *base de indução*, *hipótese de indução* e *passo de indução*, respectivamente.

Observação:

Em uma demonstração por indução, deve-se demonstrar a base de indução $p(0)$ e, fixado um k , supor verdadeira a hipótese de indução $p(k)$ e demonstrar o passo de indução.

Na realidade, o princípio da indução matemática pode ser aplicado a qualquer proposição que dependa de um conjunto para o qual exista uma bijeção com os naturais.

O princípio da indução matemática pode ser usado também em definições. Uma definição de uma construção usando este princípio é denominada *definição indutiva* ou *recursiva*. Neste caso, afirma-se que a construção é *indutivamente* ou *recursivamente definida*.

EXEMPLO:

Prova por Indução

Considere o seguinte teorema:

para qualquer $n \in \mathbb{N}$, tem-se que

$$1 + 2 + \dots + n = (n^2 + n)/2$$

Uma prova por indução é como segue:

- a) *Base de Indução*. Seja $n = 0$. Então:

$$(0^2 + 0)/2 = (0 + 0)/2 = 0/2 = 0$$

Portanto, $1 + 2 + \dots + n = (n^2 + n)/2$ é verdadeira para $n = 0$. Note-se que, $1 + 2 + \dots + n = 0 + 1 + 2 + \dots + n$ (pois 0 é o elemento neutro da adição) e, portanto, o somatório até $n = 0$ é perfeitamente definido;

b) *Hipótese de Indução*. Suponha que, para algum $n \in \mathbb{N}$, tem-se que $1 + 2 + \dots + n = (n^2 + n)/2$

c) *Passo de Indução*. Prova para $1 + 2 + \dots + n + (n + 1)$

$$1 + 2 + \dots + n + (n + 1) =$$

$$(1 + 2 + \dots + n) + (n + 1) =$$

$$(n^2 + n)/2 + (n + 1) =$$

$$(n^2 + n)/2 + (2n + 2)/2 =$$

$$(n^2 + n + 2n + 2)/2 =$$

$$((n^2 + 2n + 1) + (n + 1))/2 =$$

$$((n + 1)^2 + (n + 1))/2$$

$$\text{Portanto, } 1 + 2 + \dots + n + (n + 1) = ((n + 1)^2 + (n + 1))/2$$

Logo, para qualquer $n \in \mathbb{N}$, tem-se que $1 + 2 + \dots + n = (n^2 + n)/2$

Alfabetos, Palavras, Linguagens e Gramáticas

O Dicionário Aurélio define linguagem como "o uso da palavra articulada ou escrita como meio de expressão e comunicação entre pessoas". Entretanto, esta definição não é suficientemente precisa para permitir o desenvolvimento matemático de uma teoria sobre linguagens. Assim, faremos a seguir algumas definições formais necessárias aos estudos posteriores.

Definição:

Alfabeto

Um *Alfabeto* é um conjunto finito de *Símbolos*.

Portanto, um conjunto vazio também é considerado um alfabeto. Um *símbolo* (ou *caractere*) é uma entidade abstrata básica a qual não é definida formalmente. Letras e dígitos são exemplos de símbolos frequentemente usados.

Definição:

Palavra, Cadeia de Caracteres ou Sentença

Uma *Palavra*, *Cadeia de Caracteres* ou *Sentença* sobre um alfabeto é uma seqüência finita de símbolos (do alfabeto) justapostos.

A *palavra vazia*, representada pelo símbolo ϵ , é uma palavra sem símbolo. Se Σ representa um alfabeto, então Σ^* denota o conjunto de todas as palavras possíveis sobre Σ . Analogamente, Σ^+ representa o conjunto de todas as palavras sobre Σ excetuando-se a palavra vazia, ou seja, $\Sigma^+ = \Sigma^* - \{\epsilon\}$.

Definição:

Tamanho ou Comprimento

O *Tamanho* ou *Comprimento* de uma palavra w , representado por $|w|$, é o número de símbolos que compõem a palavra.

Definição:

Prefixo, Sufixo, Subpalavra

Um *Prefixo* (respectivamente, *Sufixo*) de uma palavra é qualquer seqüência de símbolos inicial (respectivamente, final) da palavra. Uma *Subpalavra* de uma palavra é qualquer seqüência de símbolos contígua da palavra.

EXEMPLO:

Palavra, Prefixo, Sufixo

- a) $abcb$ é uma palavra sobre o alfabeto $\{a, b, c\}$
- b) Se $\Sigma = \{a, b\}$, então $\Sigma^+ = \{a, b, aa, ab, ba, bb, aaa, \dots\}$ e $\Sigma^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$
- c) $|abcb| = 4$ e $|\epsilon| = 0$
- d) $\epsilon, a, ab, abc, abcb$ são os prefixos da palavra $abcb$ e $\epsilon, b, cb, bcb, abcb$ são os respectivos sufixos;
- e) Qualquer prefixo ou sufixo de uma palavra é uma subpalavra.

Definição:

Linguagem Formal

Uma *Linguagem Formal* é um conjunto de palavras sobre um alfabeto.

EXEMPLO:

Linguagem

Suponha o alfabeto $\Sigma = \{a, b\}$. Então:

- a) O conjunto vazio e o conjunto formado pela palavra vazia são linguagens sobre Σ (obviamente $\{\} \neq \{\epsilon\}$);

b) O conjunto de palíndromos (palavras que têm a mesma leitura da esquerda para a direita e vice-versa) sobre Σ é um exemplo de linguagem infinita. Assim, ε , a , b , aa , bb , aaa , aba , bab , bbb , $aaaa$, ... são palavras desta linguagem.

Definição:

Concatenação

A **Concatenação** é uma operação binária, definida sobre uma linguagem, a qual associa a cada par de palavras uma palavra formada pela justaposição da primeira com a segunda. Uma concatenação é denotada pela justaposição dos símbolos que representam as palavras componentes. A operação de concatenação satisfaz às seguintes propriedades (suponha v , w , t palavras):

a) **Associatividade**.

$$v(wt) = (vw)t$$

b) **Elemento Neutro à Esquerda e à Direita**.

$$\varepsilon w = w = w\varepsilon$$

Uma operação de concatenação definida sobre uma linguagem L não é, necessariamente, fechada sobre L , ou seja, a concatenação de duas palavras de L não é, necessariamente, uma palavra de L .

EXEMPLO:

Concatenação

Considere a linguagem L de palíndromos sobre $\{a, b\}$. A concatenação das palavras aba e bbb resulta na palavra $ababbb$ a qual não é palíndromo. Portanto, a operação de concatenação não é fechada sobre L .

Definição:

Concatenação Sucessiva

A **Concatenação Sucessiva** de uma palavra (com ela mesma), representada na forma de um expoente w^n onde w é uma palavra e n indica o número de concatenações sucessivas, é definida indutivamente a partir da concatenação binária, como segue:

a) **Caso 1.** $w \neq \varepsilon$

$$w^0 = \varepsilon$$

$$w^n = w^{n-1}w, \text{ para } n > 0$$

b) Caso 2. $w = \varepsilon$

$$w^n = \varepsilon, \text{ para } n > 0$$

$$w^n \text{ é indefinida para } n = 0$$

Note-se que a concatenação sucessiva é indefinida para ε^0 .

EXEMPLO:

Concatenação Sucessiva

Sejam w uma palavra e a um símbolo. Então:

$$w^3 = www$$

$$w^1 = w$$

$$a^5 = aaaaa$$

$$a^n = aaa...a \text{ (o símbolo } a \text{ repetido } n \text{ vezes)}$$

Definição:

Gramática

Uma **Gramática** é uma quádrupla ordenada $G = (V, T, P, S)$ onde:

V conjunto finito de símbolos **variáveis** ou **não-terminais**;

T conjunto finito de símbolos **terminais** disjunto de V ;

P conjunto finito de pares, denominados **regras de produção** tal que a primeira componente é palavra de $(V \cup T)^+$ e a segunda componente é palavra de $(V \cup T)^*$;

S elemento de V denominado **variável inicial**.

Observação:

Uma regra de produção (α, β) é representada por $\alpha \rightarrow \beta$. As regras de produção definem as condições de geração das palavras da linguagem. Uma seqüência de regras de produção da forma $\alpha \rightarrow \beta_1, \alpha \rightarrow \beta_2, \dots, \alpha \rightarrow \beta_n$ (mesma componente no lado esquerdo) pode ser abreviada como uma única produção na forma:

$$\alpha \rightarrow \beta_1 | \beta_2 | \dots | \beta_n$$

A aplicação de uma regra de produção é denominado derivação de uma palavra. A aplicação sucessiva de regras de produção permite derivar as palavras da linguagem representada pela gramática.

Definição:

Derivação

Seja $G = (V, T, P, S)$ uma gramática. Uma **Derivação** é um par da relação denotada por $\Rightarrow$ com domínio em $(V \cup T)^+$ e contra-domínio em $(V \cup T)^*$. Um par (α, β) é representado de forma infixada, como segue:

$$\alpha \Rightarrow \beta$$

A relação $\Rightarrow$ é indutivamente definida como segue:

- para toda a produção da forma $S \rightarrow \beta$ (a primeira componente é o símbolo inicial de G) tem-se que:

$$S \Rightarrow \beta$$

- para todo o par $\alpha \Rightarrow \beta$, onde $\beta = \beta_u \beta_v \beta_w$, se $\beta_v \rightarrow \beta_t$ é regra de P então:

$$\beta \Rightarrow \beta_u \beta_t \beta_w$$

Observação:

Portanto, uma derivação é a substituição de uma subpalavra de acordo com uma regra de produção. Quando for desejado explicitar a regra de produção $p \in P$ que define a derivação $\alpha \Rightarrow \beta$, a seguinte notação é usada:

$$\alpha \Rightarrow^p \beta$$

Sucessivos passos de derivação são definidos como segue:

$\Rightarrow^*$ fecho transitivo e reflexivo da relação $\Rightarrow$, ou seja, zero ou mais passos de derivações sucessivos;

$\Rightarrow^+$ fecho transitivo da relação $\Rightarrow$, ou seja, um ou mais passos de derivações sucessivos;

$\Rightarrow^i$ exatos i passos de derivações sucessivos, onde i é um número natural.

Gramática é considerado um formalismo de geração, pois permite derivar ("gerar") todas as palavras da linguagem que representa.

Definição:

Linguagem Gerada

Seja $G = (V, T, P, S)$ uma gramática. A *Linguagem Gerada* pela gramática G , denotada por $L(G)$ ou $GERA(G)$, é composta por todas as palavras de símbolos terminais deriváveis a partir do símbolo inicial S , ou seja:

$$L(G) = \{ w \in T^* \mid S \Rightarrow^+ w \}$$

EXEMPLO:

Gramática, Derivação, Linguagem Gerada

A gramática $G = (V, T, P, S)$ onde:

$$V = \{ S, D \}$$

$$T = \{ 0, 1, 2, \dots, 9 \}$$

$$P = \{ S \rightarrow D, S \rightarrow DS, D \rightarrow 0 \mid 1 \mid \dots \mid 9 \}$$

gera, sintaticamente, o conjunto dos números naturais. Note-se que se distingue os zeros à esquerda. Por exemplo, distingue 123 de 0123 (sugere-se como *exercício* o desenvolvimento de uma gramática que não distinga zeros à esquerda). Como ilustração, uma derivação do número 243 é como segue (existe mais alguma derivação do número 243?):

$$S \Rightarrow DS \Rightarrow 2S \Rightarrow 2DS \Rightarrow 24S \Rightarrow 24D \Rightarrow 243$$

Logo, pode-se indicar que:

$$S \Rightarrow^* 243$$

$$S \Rightarrow^+ 243$$

$$S \Rightarrow^6 243$$

EXEMPLO:

Gramática, Derivação, Linguagem Gerada

A gramática:

$$G = (\{ S, X, Y, A, B, F \}, \{ a, b \}, P, S), \text{ onde:}$$

$$P = \{ S \rightarrow XY,$$

$$X \rightarrow XaA \mid XbB \mid F$$

$$Aa \rightarrow aA, Ab \rightarrow bA, AY \rightarrow Ya,$$

$$Ba \rightarrow aB, Bb \rightarrow bB, BY \rightarrow Yb,$$

$$Fa \rightarrow aF, Fb \rightarrow bF, FY \rightarrow \varepsilon \}$$

gera a linguagem:

$$\{ ww \mid w \text{ é palavra de } \{ a, b \}^* \}$$

Como ilustração, uma derivação da palavra **baba** é como segue (existe mais alguma derivação da palavra **baba**?):

$$\begin{aligned} S &\Rightarrow XY \Rightarrow XaAY \Rightarrow XaYa \Rightarrow XbBaYa \Rightarrow XbaBYa \Rightarrow XbaYba \Rightarrow \\ &FbaYba \Rightarrow bFaYba \Rightarrow baFYba \Rightarrow baba \end{aligned}$$

A gramática apresentada gera o primeiro **w** após **X** e o segundo **w** após **Y**, como segue:

- a cada símbolo terminal gerado após **X**, é gerada uma variável correspondente;
- esta variável "caminha" na palavra até passar por **Y**, quando deriva o correspondente terminal;
- para encerrar, **X** deriva a variável **F** a qual "caminha" até encontrar **Y** quando **FY** deriva a palavra vazia. Lembre-se ϵ é o elemento neutro da concatenação e, portanto, $ba\epsilon ba = baba$.

Definição:

Gramáticas Equivalentes

Duas gramáticas **G1** e **G2** são ditas **Gramáticas Equivalentes** se, e somente se, $GERA(G1) = GERA(G2)$.



Linguagens Regulares

O estudo das *Linguagens Regulares* ou *Tipo 3*, pode ser abordado através de formalismos:

- operacional ou reconhecedor: *Autômato Finito*, o qual pode ser Determinístico, Não-Determinístico ou com Movimentos Vazios;
- axiomático ou gerador: *Gramática Regular*;
- denotacional: *Expressão Regular*.

O formalismo Expressão Regular também pode ser considerado como um formalismo gerador.

De acordo com a Hierarquia de Chomsky, trata-se da classe de linguagens mais simples, sendo possível desenvolver algoritmos de reconhecimento ou de geração de pouca complexidade, grande eficiência e de fácil implementação.

Sistema de Estados Finitos

Um *Sistema de Estados Finitos* é um modelo matemático de sistema com entradas e saídas discretas. Pode assumir um número *finito* e *pré-definido* de estados. Cada estado resume somente as informações do passado necessárias para determinar as ações para a próxima entrada.

Um forte motivacional para o estudo de Sistemas de Estados Finitos é o fato de poderem ser associados a diversos tipos de sistemas naturais e construídos.

Um exemplo clássico e de simples entendimento é um elevador. Trata-se de um sistema que não memoriza as requisições anteriores. Cada "estado" sumariza as informações "andar corrente" e "direção de movimento". As entradas para o sistema são requisições pendentes.

Analísadores Léxicos e Processadores de Texto (ou algumas ferramentas de Processadores de Texto) também são exemplos de sistemas de estados finitos, onde cada estado, basicamente, memoriza a estrutura do prefixo da palavra em análise.

Entretanto, nem todos os Sistemas de Estados Finitos são adequados para serem estudados por esta abordagem. Um contra-exemplo é o cérebro humano. Existem evidências de que um neurônio pode ser representado por um número finito de bits. O cérebro é composto por cerca de 2^{35} células. Portanto, a princípio, é possível representá-lo por um número finito de estados. Entretanto, o elevado número de combinações de células (e, conseqüentemente, de estados) determina uma abordagem pouco eficiente.

Outro contra-exemplo é o computador. Os estados determinados pelos processadores e memórias podem ser representados como um sistema de estados finitos. Entretanto, o estudo adequado da noção de computabilidade exige uma memória sem limite predefinido. Nas próximas páginas, é apresentado um outro formalismo de autômato, a Máquina de Turing, mais adequado ao estudo da computabilidade. Note-se que, o estudo da computabilidade e solucionabilidade de problemas não é um dos objetivos desta publicação.

Autômato Finito

Um Autômato Finito Determinístico ou simplesmente Autômato Finito pode ser visto como uma máquina composta, basicamente, de três partes:

- Fita.** Dispositivo de entrada que contém a informação a ser processada;
- Unidade de Controle.** Reflete o estado corrente da máquina. Possui uma unidade de leitura (cabeça da fita) a qual acessa uma célula da fita de cada vez e movimenta-se exclusivamente para a direita;
- Programa** ou **Função de Transição.** Função que comanda as leituras e define o estado da máquina.

A fita é finita (à esquerda e à direita), sendo dividida em células, onde cada uma armazena um símbolo. Os símbolos pertencem a um alfabeto de entrada. Não é possível gravar sobre a fita (e não existe memória auxiliar). Inicialmente, a palavra a ser processada (ou seja, a informação de entrada para a máquina) ocupa toda a fita.

A unidade de controle possui um número finito e predefinido de estados. A unidade de leitura lê o símbolo de uma célula de cada vez. Após a leitura, a **cabeça da fita** move-se uma célula para a direita. Inicialmente, a cabeça está posicionada na célula mais à esquerda da fita, como ilustrado na [figura abaixo](#).

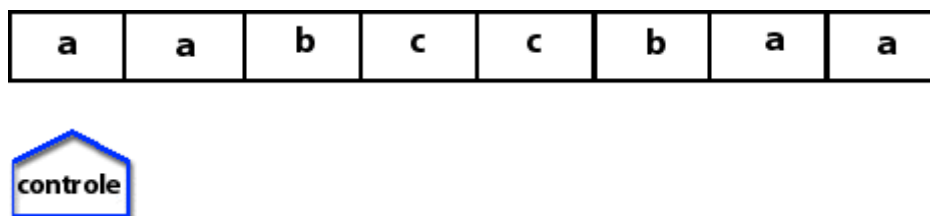


Figura Autômato finito como uma máquina com controle finito

O programa é uma função parcial que, dependendo do estado corrente e do símbolo lido, determina o novo estado do autômato. Deve-se reparar que o Autômato Finito não

possui memória de trabalho. Portanto, para armazenar as informações passadas necessárias ao processamento, deve-se usar o conceito de estado.

Definição:

Autômato Finito Determinístico

Um *Autômato Finito Determinístico* (AFD) ou simplesmente *Autômato Finito* (AF) M é uma 5-upla:

$$M = (\Sigma, Q, \delta, q_0, F)$$

onde:

Σ *alfabeto de símbolos de entrada*;

Q *conjunto de estados* possíveis do autômato o qual é finito;

δ *função programa* ou *função de transição*:

$$\delta: Q \times \Sigma \rightarrow Q$$

a qual é uma função parcial;

q_0 *estado inicial* tal que q_0 é elemento de Q ;

F *conjunto de estados finais* tal que F está contido em Q .

A *função programa* pode ser representada como um grafo finito direto, como ilustrado na figura abaixo.

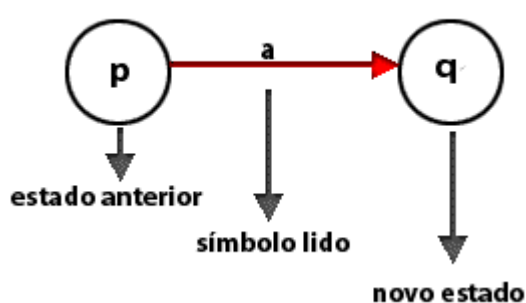


Figura Representação da função programa como um grafo

Neste caso, os estados iniciais e finais são representados como ilustrado na figura abaixo.



Figura Representação de um estado inicial (esq.) e final (dir.) como nodos de grafos

O processamento de um Autômato Finito M , para uma palavra de entrada w , consiste na sucessiva aplicação da função programa para cada símbolo de w (da esquerda para a direita) até ocorrer uma condição de parada.

EXEMPLO:

Autômato Finito

Considere a linguagem:

$$L1 = \{w \mid w \text{ possui } aa \text{ ou } bb \text{ como subpalavra}\}$$

O Autômato Finito:

$$M1 = (\{a, b\}, \{q0, q1, q2, qf\}, \delta1, q0, \{qf\})$$

onde $\delta1$ é como abaixo, representada na forma de uma tabela, reconhece a linguagem $L1$.

$\delta1$	a	b
q0	q1	q2
q1	qf	q2
q2	q1	qf
qf	qf	qf

O algoritmo apresentado usa os estados $q1$ e $q2$ para "memorizar" o símbolo anterior. Assim, $q1$ representa "símbolo anterior é a" e $q2$ representa "símbolo anterior é b". Após identificar dois a ou dois b consecutivos, o autômato assume o estado qf (final) e varre o sufixo da palavra de entrada, sem qualquer controle lógico, somente para terminar o processamento. O processamento do Autômato Finito $M1$ para a entrada $w = abba$, a qual é aceita.

Um **Autômato Finito** sempre pára ao processar qualquer entrada pois, como qualquer palavra é finita e como um novo símbolo da entrada é lido a cada aplicação da função programa, não existe a possibilidade de ciclo (*loop*) infinito. A parada de um processamento pode ser de duas maneiras: aceitando ou rejeitando uma entrada w . As condições de parada são as seguintes:

- Após processar o último símbolo da fita, o Autômato Finito assume um estado final: o autômato pára e a entrada w é aceita;
- Após processar o último símbolo da fita, o Autômato Finito assume um estado não-final: o autômato pára e a entrada w é rejeitada;
- A função programa é indefinida para o argumento (estado corrente e símbolo lido): a máquina pára e a palavra de entrada w é rejeitada.

Definição:

Função Programa Estendida

Seja $M = (\Sigma, Q, \delta, q_0, F)$ um Autômato Finito Determinístico. A *Função Programa Estendida* denotada por:

$$\hat{\delta}: Q \times \Sigma^* \rightarrow Q$$

é a função programa $\delta: Q \times \Sigma \rightarrow Q$ estendida para palavras e é indutivamente definida como segue:

$$\hat{\delta}(q, \varepsilon) = q$$

$$\hat{\delta}(q, aw) = \hat{\delta}(\hat{\delta}(q, a), w)$$

EXEMPLO:

Função Programa Estendida

Considere o Autômato Finito $M_1 = (\{a, b\}, \{q_0, q_1, q_2, q_f\}, \delta_1, q_0, \{q_f\})$. Então, a função programa estendida aplicada à palavra **abaa** a partir do estado inicial q_0 é como segue:

$\hat{\delta}(q_0, abaa) =$	função estendida sobre abaa
$\hat{\delta}(\hat{\delta}(q_0, a), baa) =$	processa <u>a</u> baa
$\hat{\delta}(q_1, baa) =$	função estendida sobre baa
$\hat{\delta}(\hat{\delta}(q_1, b), aa) =$	processa <u>b</u> aa
$\hat{\delta}(q_2, aa) =$	função estendida sobre aa
$\hat{\delta}(\hat{\delta}(q_2, a), a) =$	processa <u>a</u> a
$\hat{\delta}(q_1, a) =$	função estendida sobre a
$\hat{\delta}(\hat{\delta}(q_1, a), \varepsilon) =$	processa <u>a</u> baa
$\hat{\delta}(q_f, \varepsilon) = q_f$	função estendida sobre ε : fim da indução

e, portanto, a palavra é aceita.

A linguagem aceita por um Autômato Finito $M = (\Sigma, Q, \delta, q_0, F)$, denotada por **ACEITA**(M) ou **L**(M), é o conjunto de todas as palavras pertencentes a Σ^* aceitas por M, ou seja:

$$\text{ACEITA}(M) = \{w \mid \hat{\delta}(q_0, w) \in F\}$$

Analogamente, **REJEITA**(M) é o conjunto de todas as palavras pertencentes a Σ^* rejeitadas por M. As seguintes afirmações são verdadeiras:

- $\text{ACEITA}(M) \cap \text{REJEITA}(M) = \emptyset$
- $\text{ACEITA}(M) \cup \text{REJEITA}(M) = \Sigma^*$
- o complemento de **ACEITA**(M) é **REJEITA**(M)

- o complemento de $REJEITA(M)$ é $ACEITA(M)$

Definição:

Autômatos Finitos Equivalentes

Dois Autômatos Finitos M_1 e M_2 são ditos *Autômatos Finitos Equivalentes* se, e somente se:

$$ACEITA(M_1) = ACEITA(M_2)$$

EXEMPLO:

Autômato Finito

Considere as linguagens:

$$L_2 = \{ \} \quad \text{e} \quad L_3 = \Sigma^*$$

Os Autômatos Finitos:

$$M_2 = (\{ a, b \}, \{ q_0 \}, \delta_2, q_0, \{ \}) \quad \text{e} \quad M_3 = (\{ a, b \}, \{ q_0 \}, \delta_3, q_0, \{ q_0 \})$$

onde δ_2 e δ_3 são como abaixo, representadas na forma de uma tabela, são tais que $ACEITA(M_2) = L_2$ e $ACEITA(M_3) = L_3$:

δ_2	a	b
q_0	q_0	q_0
δ_3	a	b
q_0	q_0	q_0

Relativamente aos autômatos M_2 e M_3 , sugere-se, como *exercício*, o esclarecimento das seguintes questões:

- Existe alguma diferença entre as funções δ_2 e δ_3 ?
- O que, exatamente, diferencia M_2 de M_3 ?



Figura Grafos dos Autômatos Finitos

Definição:

Linguagem Regular ou Tipo 3

Uma linguagem aceita por um Autômato Finito Determinístico é uma *Linguagem Regular* ou *Tipo 3*.

EXEMPLO:

Autômato Finito

Considere a linguagem:

$$L_4 = \{w \mid w \text{ possui um número par de } a \text{ e } b\}$$

O Autômato Finito:

$$M_4 = (\{a, b\}, \{q_0, q_1, q_2, q_3\}, \delta_4, q_0, \{q_0\})$$

ilustrado na *figura abaixo*, é tal que $ACEITA(M_4) = L_4$.

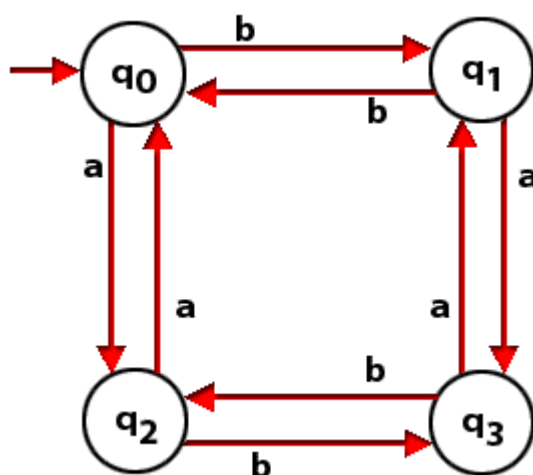


Figura Grafo do Autômato Finito Determinístico

Autômato Finito Não-Determinístico

O não-determinismo é uma importante generalização dos modelos de máquinas, sendo de fundamental importância no estudo da teoria da computação e da teoria das linguagens formais. Nem sempre a facilidade de não-determinismo aumenta o poder de reconhecimento de linguagens de uma classe de autômatos. Por exemplo, conforme será mostrado adiante, qualquer Autômato Finito Não-Determinístico pode ser simulado por um Autômato Finito Determinístico.

A facilidade de *não-determinismo* para autômatos finitos é interpretada como segue: a função programa, ao processar uma entrada composta pelo estado corrente e símbolo lido, tem como resultado um conjunto de novos estados. Visto como uma máquina

composta por fita, unidade de controle e programa, pode-se afirmar que um Autômato Não-Determinístico assume um conjunto de estados alternativos, como se houvesse uma multiplicação da unidade de controle, uma para cada alternativa, processando independentemente, sem compartilhar recursos com as demais. Assim, o processamento de um caminho não influi no estado, símbolo lido e posição da cabeça dos demais caminhos alternativos

Definição:

Autômato Finito Não-Determinístico

Um *Autômato Finito Não-Determinístico (AFN)* M é uma 5-upla:

$$M = (\Sigma, Q, \delta, q_0, F)$$

onde:

Σ *alfabeto de símbolos de entrada*;

Q *conjunto de estados possíveis* do autômato o qual é finito;

δ *função programa* ou *função de transição*:

$$\delta: Q \times \Sigma \rightarrow 2^Q$$

a qual é uma função parcial;

q_0 *estado inicial* tal que q_0 é elemento de Q ;

F *conjunto de estados finais* tal que F está contido em Q .

Excetuando-se pela função programa, as componentes Σ , Q , q_0 e F são como na definição do AFD. A função programa pode ser representada como um grafo finito direto, como ilustrado na *figura abaixo*, supondo que:

$$\delta(q, a) = \{p_1, p_2, \dots, p_n\}$$

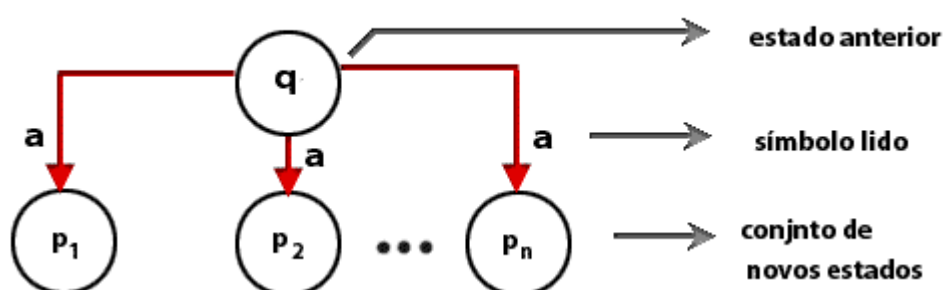


Figura Representação da função programa não-determinística como um grafo

O processamento de um Autômato Finito Não-Determinístico M para um conjunto de estados, ao ler um símbolo, é a união dos resultados da função programa aplicada a cada estado alternativo.

Definição:

Função Programa Estendida

Seja $M = (\Sigma, Q, \delta, q_0, F)$ um Autômato Finito Não-Determinístico. A *Função Programa Estendida* de M denotada por:

$$\hat{\delta}: 2^Q \times \Sigma^* \rightarrow 2^Q$$

é a função programa $\delta: Q \times \Sigma \rightarrow 2^Q$ estendida para palavras e é indutivamente definida como segue:

$$\hat{\delta}(P, \varepsilon) = P$$

$$\hat{\delta}(P, aw) = \hat{\delta}(\cup_{q \in P} \delta(q, a), w)$$

Para um dado conjunto de estados $\{q_1, q_2, \dots, q_n\}$ e para um dado símbolo a :

$$\hat{\delta}(\{q_1, q_2, \dots, q_n\}, a) = \delta(q_1, a) \cup \delta(q_2, a) \cup \dots \cup \delta(q_n, a)$$

A linguagem aceita por um Autômato Finito Não-Determinístico $M = (\Sigma, Q, \delta, q_0, F)$, denotada por $ACEITA(M)$ ou $L(M)$, é o conjunto de todas as palavras pertencentes a Σ^* tais que existe pelo menos um caminho alternativo que aceita a palavra, ou seja:

$$ACEITA(M) = \{w \mid \text{existe } q \in \delta(q_0, w) \text{ tal que } q \in F\}$$

Analogamente, $REJEITA(M)$ é o conjunto de todas as palavras pertencentes a Σ^* rejeitadas por todos os caminhos alternativos de M (a partir de q_0).

EXEMPLO:

Autômato Finito Não-Determinístico

Considere a linguagem:

$$L_5 = \{w \mid w \text{ possui } aa \text{ ou } bb \text{ como subpalavra}\}$$

O Autômato Finito Não-Determinístico:

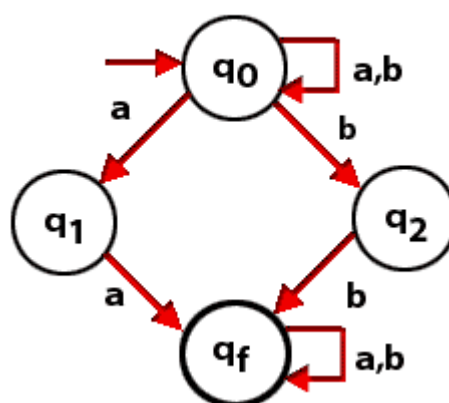
$$M_5 = (\{a, b\}, \{q_0, q_1, q_2, q_f\}, \delta_5, q_0, \{q_f\})$$

onde δ_5 é como abaixo, representada na forma de tabela, é tal que $ACEITA(M_5) = L_5$:

δ	a	b
q0	{q0, q1}	{q0, q2}
q1	{qf}	-
q2	-	{qf}
qf	{qf}	{qf}

O autômato pode ser representado pelo grafo ilustrado na [figura abaixo](#). O algoritmo apresentado realiza uma varredura sobre a palavra de entrada. A cada ocorrência de **a** (ou **b**) uma alternativa é iniciada, para verificar se o símbolo seguinte também é **a** (ou **b**). Assim, existem três caminhos alternativos de processamento:

- ciclo em q0 realiza uma varredura em toda a entrada;
- caminho q0/q1/qf garante a ocorrência de **aa**
- caminho q0/q2/qf garante a ocorrência de **bb**



EXEMPLO:

Autômato Finito Não-Determinístico

Considere a linguagem:

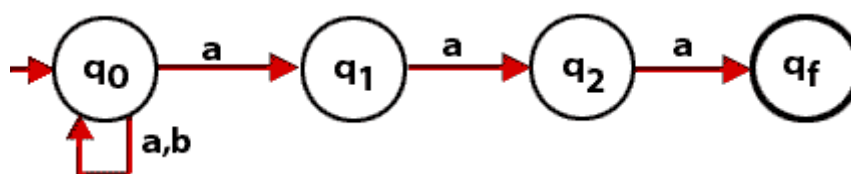
$$L_6 = \{w \mid w \text{ possui } aaa \text{ como sufixo}\}$$

O Autômato Finito Não-Determinístico:

$$M_6 = (\{a, b\}, \{q_0, q_1, q_2, q_f\}, \delta_6, q_0, \{q_f\})$$

ilustrado na [figura abaixo](#), é tal que $ACEITA(M_6) = L_6$.

Embora a facilidade de não-determinismo seja, aparentemente, um significativo acréscimo ao Autômato Finito, na realidade não aumenta seu poder computacional. Assim, para cada AFN, é possível construir um AFD equivalente que realiza o mesmo processamento. O contrário também é verdadeiro.



Teorema:

Equivalência entre AFD e AFN

A Classe dos Autômatos Finitos Determinísticos é equivalente à Classe dos Autômatos Finitos Não-Determinísticos.

Prova: A prova consiste em mostrar que, a partir de um AFN M qualquer, é possível construir um AFD M' que realiza o mesmo processamento (ou seja, M' simula M). A demonstração apresenta um algoritmo para converter um AFN qualquer em um AFD equivalente. A idéia central do algoritmo é a construção de estados de M' que simulem as diversas combinações de estados alternativos de M . O contrário (construir um não-determinístico a partir de um determinístico) não necessita ser mostrado, pois decorre trivialmente das definições (por quê?).

Seja $M = (\Sigma, Q, \delta, q_0, F)$ um AFN qualquer. Seja $M' = (\Sigma, Q', \delta', \langle q_0 \rangle, F')$ um AFD construído a partir de M como segue:

Q' conjunto de todas as combinações, sem repetições, de estados de Q as quais são denotadas por $\langle q_1 q_2 \dots q_n \rangle$, onde q_i pertence a Q , para i em $\{1, 2, \dots, n\}$. Note-se que a ordem dos elementos não distingue mais combinações. Por exemplo, $\langle q_u q_v \rangle = \langle q_v q_u \rangle$;

δ' tal que $\delta'(\langle q_1 \dots q_n \rangle, a) = \langle p_1 \dots p_m \rangle$ se, e somente se, $\delta(\{q_1, \dots, q_n\}, a) = \{p_1, \dots, p_m\}$. Ou seja, um estado de M' representa uma imagem dos estados de todos os caminhos alternativos de M ;

$\langle q_0 \rangle$ estado inicial;

F' conjunto de todos os estados $\langle q_1 q_2 \dots q_n \rangle$ pertencentes a Q' tal que alguma componente q_i pertence a F , para i em $\{1, 2, \dots, n\}$.

A demonstração de que o AFD M' simula o processamento do AFN M é por indução no tamanho da palavra. Deve-se mostrar que (suponha w uma palavra qualquer de Σ^*):

$$\delta'(\langle q_0 \rangle, w) = \langle q_1 \dots q_u \rangle \quad \text{sse} \quad \delta(\{q_0\}, w) = \{q_1, \dots, q_u\}$$

a) **Base de indução.** Seja w tal que $|w| = 0$. Então:

$$\delta'(\langle q_0 \rangle, \epsilon) = \langle q_0 \rangle \quad \text{sse} \quad \delta(\{q_0\}, \epsilon) = \{q_0\}$$

o que é verdadeiro, por definição de função programa estendida;

b) **Hipótese de indução.** Seja w tal que $|w| = n$ e $n \geq 1$. Suponha verdadeiro que:

$$\delta'(\langle q_0 \rangle, w) = \langle q_1 \dots q_u \rangle \quad \text{sse} \quad \delta(\{q_0\}, w) = \{q_1, \dots, q_u\}$$

c) *Passo de Indução*. Seja w tal que $|w| = n + 1$ e $n \geq 1$.

$$\delta'(\langle q_0 \rangle, wa) = \langle p_1 \dots p_v \rangle \quad \text{sse} \quad \delta(\{q_0\}, wa) = \{p_1, \dots, p_v\}$$

o que equivale, por hipótese de indução:

$$\delta'(\langle q_1 \dots q_u \rangle, a) = \langle p_1 \dots p_v \rangle \quad \text{sse} \quad \delta(\{q_1, \dots, q_u\}, a) = \{p_1, \dots, p_v\}$$

o que é verdadeiro, por definição de δ' .

Logo, M' simula M para qualquer entrada w pertencente a Σ^* .

EXEMPLO:

Construção de um AFD a partir de um AFN

Considere o AFN $M_6 = (\{a, b\}, \{q_0, q_1, q_2, q_f\}, \delta_6, q_0, \{q_f\})$. E o correspondente AFD $M_6' = (\{a, b\}, Q', \delta_6', \langle q_0 \rangle, F')$, é como segue:

$$Q' = \{\langle q_0 \rangle, \langle q_1 \rangle, \langle q_2 \rangle, \langle q_f \rangle, \langle q_0q_1 \rangle, \langle q_0q_2 \rangle, \dots, \langle q_0q_1q_2q_f \rangle\}$$

$$F' = \{\langle q_f \rangle, \langle q_0q_f \rangle, \langle q_1q_f \rangle, \dots, \langle q_0q_1q_2q_f \rangle\}$$

δ_6' é tal que (na tabela que segue são explicitados somente os estados para os quais a função programa é definida):

δ_6'	a	b
$\langle q_0 \rangle$	$\langle q_0q_1 \rangle$	$\langle q_0 \rangle$
$\langle q_0q_1 \rangle$	$\langle q_0q_1q_2 \rangle$	$\langle q_0 \rangle$
$\langle q_0q_1q_2 \rangle$	$\langle q_0q_1q_2q_f \rangle$	$\langle q_0 \rangle$
$\langle q_0q_1q_2q_f \rangle$	$\langle q_0q_1q_2q_f \rangle$	$\langle q_0 \rangle$

O grafo que representa M_6' é ilustrado na figura abaixo onde os estados p_0 , p_1 , p_2 e p_f denotam $\langle q_0 \rangle$, $\langle q_0q_1 \rangle$, $\langle q_0q_1q_2 \rangle$ e $\langle q_0q_1q_2q_f \rangle$, respectivamente.

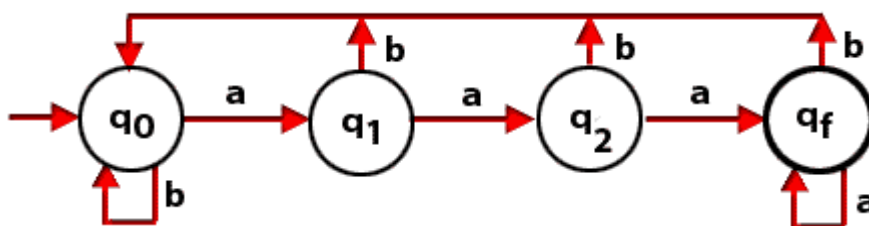


Figura Grafo do AFD construído a partir de um AFN

Autômato Finito com Movimentos Vazios

Movimentos vazios constituem uma generalização dos modelos de máquinas não-determinística. Um *movimento vazio* é uma transição sem leitura de símbolo algum da fita. Pode ser interpretado como um *não-determinismo interno* ao autômato o qual é encapsulado, ou seja, excetuando-se por uma eventual mudança de estados, nada mais pode ser observado sobre um movimento vazio. Uma das vantagens dos Autômatos Finitos com Movimentos Vazios no estudo das linguagens formais é o fato de facilitar algumas construções e demonstrações relacionadas com os autômatos.

No caso dos Autômatos Finitos, a facilidade de movimentos vazios não aumenta o poder de reconhecimento de linguagens. Conforme será mostrado adiante, qualquer Autômato Finito com Movimentos Vazios pode ser simulado por um Autômato Finito Não-Determinístico.

Definição:

Autômato Finito com Movimentos Vazios

Um *Autômato Finito Não-Determinístico e com Movimentos Vazios* (AFN_{ϵ}) ou simplesmente *Autômato Finito com Movimentos Vazios* (AF_{ϵ}) M é uma 5-upla:

$$M = (\Sigma, Q, \delta, q_0, F)$$

onde:

Σ *alfabeto de símbolos de entrada* ;

Q *conjunto de estados* possíveis do autômato o qual é finito;

δ *função programa* ou *função de transição*:

$$\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$$

a qual é uma função parcial;

q_0 *estado inicial* tal que q_0 é elemento de Q ;

F *conjunto de estados finais* tal que F está contido em Q

Portanto, excetuando-se pela função programa, as componentes Σ , Q , F e q_0 são como na definição de AFN. Um *movimento vazio* (ou *transição vazia*) é representado pela aplicação da função programa, em um dado estado q ao símbolo especial ϵ , ou seja, $\delta(q, \epsilon)$. A função programa com movimentos vazios pode ser interpretada como um grafo finito direto, como ilustrado na *figura abaixo*, supondo que:

$$\delta(q, \epsilon) = \{p_0\}, \delta(q, a_1) = \{p_1\}, \dots, \delta(q, a_n) = \{p_n\}$$

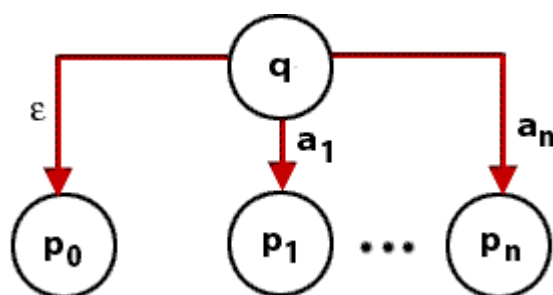


Figura Representação como um grafo da função programa com movimentos vazios

O processamento de um AF_{ϵ} é análogo ao de um AFN. Adicionalmente, o processamento de uma transição para uma entrada vazia também é não-determinística. Assim, um AF_{ϵ} ao processar uma entrada vazia assume simultaneamente os estados destino e origem. Ou seja, a origem de um movimento vazio sempre é um caminho alternativo.

EXEMPLO:

Autômato Finito com Movimentos Vazios

Considere a linguagem:

$$L_7 = \{w \mid \text{qualquer símbolo } a \text{ antecede qualquer símbolo } b\}$$

O Autômato Finito com Movimentos Vazios:

$$M_7 = (\{a, b\}, \{q_0, q_f\}, \delta_7, q_0, \{q_f\})$$

ilustrado na, onde δ_7 é como abaixo, representada na forma de tabela, é tal que $ACEITA(M_7) = L_7$:

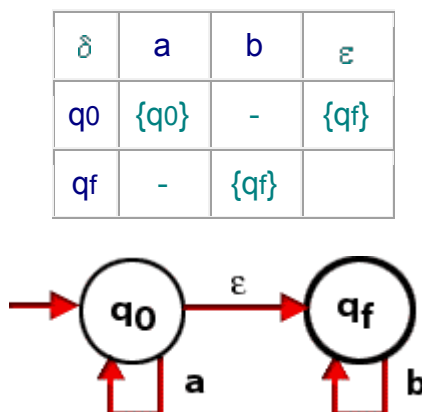


Figura Grafo do Autômato Finito com Movimentos Vazios

Definição:

Função Fecho Vazio, Fecho Vazio Estendida

Seja $M = (\Sigma, Q, \delta, q_0, F)$ um Autômato Finito com Movimentos Vazios.

a) A *Função Fecho Vazio* denotada por:

$$\text{FECHO-}\varepsilon: Q \rightarrow 2^Q$$

é indutivamente definida como segue:

$$\text{FECHO-}\varepsilon(q) = \{ q \}, \text{ se } \delta(q, \varepsilon) \text{ é indefinido;}$$

$$\text{FECHO-}\varepsilon(q) = \{ q \} \cup \delta(q, \varepsilon) \cup (\cup p \in \delta(q, \varepsilon) \text{ FECHO-}\varepsilon(p)), \text{ caso contrário.}$$

b) A *Função Fecho Vazio Estendida* denotada por:

$$\underline{\text{FECHO-}}\varepsilon: 2^Q \rightarrow 2^Q$$

é a função fecho vazio estendida para conjunto de estados e é tal que:

$$\underline{\text{FECHO-}}\varepsilon(P) = \cup_{q \in P} \text{FECHO-}\varepsilon(q)$$

EXEMPLO:

Função Fecho Vazio, Fecho Vazio Estendida

Considere o Autômato Finito com Movimentos Vazios $M7 = (\{ a, b \}, \{ q0, qf \}, \delta7, q0, \{ qf \})$. Então:

$$\text{FECHO-}\varepsilon(q0) = \{ q0, qf \}$$

$$\text{FECHO-}\varepsilon(qf) = \{ qf \}$$

$$\underline{\text{FECHO-}}\varepsilon(\{ q0, qf \}) = \{ q0, qf \}$$

Definição:

Função Programa Estendida

Seja $M = (\Sigma, Q, \delta, q0, F)$ um Autômato Finito com Movimentos Vazios. A *Função Programa Estendida* de M denotada por:

$$\delta: 2^Q \times \Sigma^* \rightarrow 2^Q$$

é a função programa $\delta: Q \times (\Sigma \cup \{ \varepsilon \}) \rightarrow 2^Q$ estendida para um conjunto finito de estados e para uma palavra e é indutivamente definida como segue:

$$\delta(P, \varepsilon) = \text{FECHO-}\varepsilon(P)$$

$$\delta(P, wa) = \text{FECHO-}\varepsilon(R) \text{ onde } R = \{ r \mid r \in \delta(s, a) \text{ e } s \in \delta(P, w) \}$$

A linguagem aceita por um *Autômato Finito com Movimentos Vazios* $M = (\Sigma, Q, \delta, q0, F)$, denotada por $\text{ACEITA}(M)$ ou $L(M)$, é o conjunto de todas as palavras pertencentes a Σ^* tais que existe pelo menos um caminho alternativo que aceita a palavra, ou seja:

$$ACEITA(M) = \{ w \mid \text{existe } q \in \delta(q_0, w) \text{ tal que } q \in F \}$$

Analogamente, **REJEITA (M)** é o conjunto de todas as palavras pertencentes a Σ^* rejeitadas por todos os caminhos alternativos de **M** (a partir de q_0).

EXEMPLO:

Função Programa Estendida

Considere a linguagem:

$$L_8 = \{ w \mid w \text{ possui como sufixo } a \text{ ou } bb \text{ ou } ccc \}$$

O Autômato Finito com Movimentos Vazios:

$$M_8 = (\{ a, b, c \}, \{ q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_f \}, \delta_8, q_0, \{ q_f \})$$

ilustrado na **figura abaixo**, é tal que $ACEITA(M_8) = L_8$. Relativamente ao fecho vazio, tem-se que, por exemplo, $FECHO-\varepsilon(\{ q_0 \}) = \{ q_0, q_1, q_2, q_4 \}$. Exemplificando a função estendida, tem-se que:

$$\hat{\delta}(\{ q_0 \}, abb) = FECHO-\varepsilon(\{ r \mid r \in \delta(s, b) \text{ e } s \in \hat{\delta}(\{ q_0 \}, ab) \}), \text{ onde:}$$

$$\hat{\delta}(\{ q_0 \}, ab) = FECHO-\varepsilon(\{ r \mid r \in \delta(s, b) \text{ e } s \in \hat{\delta}(\{ q_0 \}, a) \}), \text{ onde:}$$

$$\hat{\delta}(\{ q_0 \}, a) = FECHO-\varepsilon(\{ r \mid r \in \delta(s, a) \text{ e } s \in \hat{\delta}(\{ q_0 \}, \varepsilon) \})$$

Como $\hat{\delta}(\{ q_0 \}, \varepsilon) = FECHO-\varepsilon(\{ q_0 \}) = \{ q_0, q_1, q_2, q_4 \}$ tem-se que:

$$\hat{\delta}(\{ q_0 \}, a) = \{ q_0, q_1, q_2, q_4, q_f \}$$

$$\hat{\delta}(\{ q_0 \}, ab) = \{ q_0, q_1, q_2, q_3, q_4 \}$$

$$\hat{\delta}(\{ q_0 \}, abb) = \{ q_0, q_1, q_2, q_3, q_4, q_f \}$$

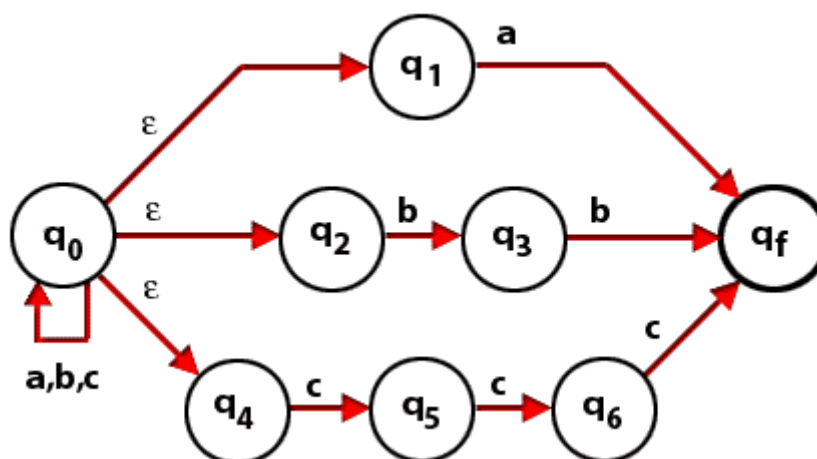


Figura Grafo do Autômato Finito com Movimentos Vazios

Teorema:

Equivalência entre AFN e AF ϵ

A Classe dos Autômatos Finitos com Movimentos Vazios é equivalente à Classe dos Autômatos Finitos Não-Determinísticos.

Prova: A prova consiste em mostrar que, a partir de um AF ϵ qualquer, é possível construir um AFN que realiza o mesmo processamento. A demonstração apresenta um algoritmo para converter um AF ϵ em um AFN equivalente. A idéia central do algoritmo é a construção de uma função programa sem movimentos vazios onde o conjunto de estados destino de cada transição não vazia é estendido com todos os estados possíveis de serem atingidos por transições vazias. O contrário (construir um AF ϵ a partir de um AFN) não necessita ser mostrado, pois decorre trivialmente das definições.

Seja $M = (\Sigma, Q, \delta, q_0, F)$ um AF ϵ qualquer. Seja $M' = (\Sigma, Q, \delta', q_0, F')$ um AFN construído a partir de M como segue:

δ' tal que $\delta': Q \times \Sigma \rightarrow 2^Q$ onde $\delta'(q, a) = \delta(\{q\}, a)$

F' conjunto de todos os estados q pertencentes a Q tal que algum elemento do $\text{FECHO-}\epsilon(q)$ pertence a F .

A demonstração de que o AFN M' simula o AF ϵ M é feita por indução no tamanho da palavra e é sugerida como exercício.

EXEMPLO:

Construção de um AFN a partir de um AF ϵ

O Autômato Finito com Movimentos Vazios $M_9 = (\{a, b\}, \{q_0, q_1, q_2\}, \delta_9, q_0, \{q_2\})$, onde δ_9 é como segue:

δ_9	a	b	ϵ
q0	{q0}	-	{q1}
q1	-	{q1}	{q2}
q2	{q2}	-	-

é representado pelo grafo ilustrado na figura abaixo (qual a linguagem aceita por M_9 ?).

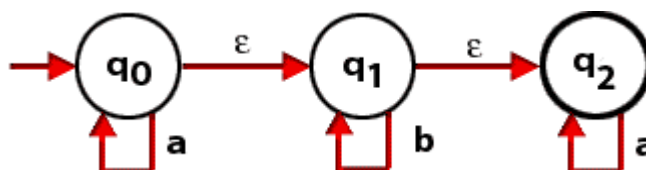


Figura Grafo do Autômato Finito com Movimentos Vazios

O correspondente AFN $M_9' = (\{a, b\}, \{q_0, q_1, q_2\}, \delta_9', q_0, F')$, é como segue (por simplicidade, FECHO- ε é abreviado para $F\varepsilon$) e é ilustrado na figura abaixo:

$F' = \{q_0, q_1, q_2\}$, pois:

$$F\varepsilon(q_0) = \{q_0, q_1, q_2\}$$

$$F\varepsilon(q_1) = \{q_1, q_2\}$$

$$F\varepsilon(q_2) = \{q_2\}$$

Na construção de δ_9' note-se que:

$$\delta_9(\{q_0\}, \varepsilon) = \{q_0, q_1, q_2\}$$

$$\delta_9(\{q_1\}, \varepsilon) = \{q_1, q_2\}$$

$$\delta_9(\{q_2\}, \varepsilon) = \{q_2\}$$

Assim, δ_9' é tal que:

$$\delta_9'(q_0, a) = \delta_9(\{q_0\}, a) = F\varepsilon(\{r \mid r \in \delta(s, a) \text{ e } s \in \delta_9(\{q_0\}, \varepsilon)\}) = \{q_0, q_1, q_2\}$$

$$\delta_9'(q_0, b) = \delta_9(\{q_0\}, b) = F\varepsilon(\{r \mid r \in \delta(s, b) \text{ e } s \in \delta_9(\{q_0\}, \varepsilon)\}) = \{q_1, q_2\}$$

$$\delta_9'(q_1, a) = \delta_9(\{q_1\}, a) = F\varepsilon(\{r \mid r \in \delta(s, a) \text{ e } s \in \delta_9(\{q_1\}, \varepsilon)\}) = \{q_2\}$$

$$\delta_9'(q_1, b) = \delta_9(\{q_1\}, b) = F\varepsilon(\{r \mid r \in \delta(s, b) \text{ e } s \in \delta_9(\{q_1\}, \varepsilon)\}) = \{q_1, q_2\}$$

$$\delta_9'(q_2, a) = \delta_9(\{q_2\}, a) = F\varepsilon(\{r \mid r \in \delta(s, a) \text{ e } s \in \delta_9(\{q_2\}, \varepsilon)\}) = \{q_2\}$$

$$\delta_9'(q_2, b) = \delta_9(\{q_2\}, b) = F\varepsilon(\{r \mid r \in \delta(s, b) \text{ e } s \in \delta_9(\{q_2\}, \varepsilon)\}) \text{ é indefinida.}$$

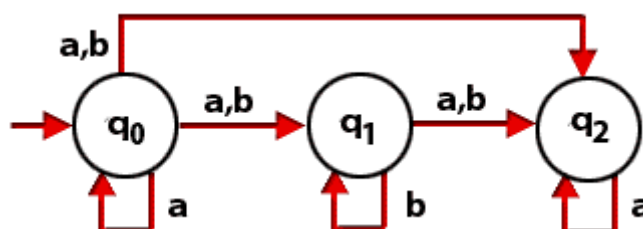


Figura Grafo do Autômato Finito Não-Determinístico equivalente

Expressão Regular

Toda Linguagem Regular pode ser descrita por uma expressão simples, denominada Expressão Regular. Trata-se de um formalismo denotacional, também considerado gerador, pois pode-se inferir como construir ("gerar") as palavras de uma linguagem.

Uma Expressão Regular é definida a partir de conjuntos (linguagens) básicos e operações de concatenação e união. As Expressões Regulares são consideradas adequadas para a comunicação homem $\times$ homem e, principalmente, para a comunicação homem $\times$ máquina.

Definição:

Expressão Regular

Uma **Expressão Regular** (ER) sobre um alfabeto Σ é indutivamente definida como segue:

- a) $\emptyset$ é uma ER e denota a linguagem vazia;
- b) ϵ é uma ER e denota a linguagem contendo exclusivamente a palavra vazia, ou seja, $\{\epsilon\}$
- c) Qualquer símbolo x pertencente a Σ é uma ER e denota a linguagem contendo a palavra unitária x , ou seja, $\{x\}$
- d) Se r e s são ER e denotam as linguagens R e S , respectivamente, então:
 - d.1) $(r + s)$ é ER e denota a linguagem $R \cup S$
 - d.2) (rs) é ER e denota a linguagem $RS = \{uv \mid u \in R \text{ e } v \in S\}$
 - d.3) (r^*) é ER e denota a linguagem R^*

EXEMPLO:

Expressão Regular

Na tabela abaixo, são apresentadas Expressões Regulares e as correspondentes linguagens:

Expressão Regular	Linguagem Representada
aa	somente a palavra aa
ba^*	todas as palavras que iniciam por b , seguido por zero ou mais a
$(a + b)^*$	todas as palavras sobre $\{a, b\}$
$(a + b)^*aa(a + b)^*$	todas as palavras contendo aa como subpalavra
$a^*ba^*ba^*$	todas as palavras contendo exatamente dois b
$(a + b)^*(aa + bb)$	todas as palavras que terminam com aa ou bb
$(a + \epsilon)(b + ba)^*$	todas as palavras que não possuem dois a consecutivos

Teorema:

Expressão Regular $\rightarrow$ Linguagem Regular

Se r é uma Expressão Regular, então $GERA(r)$ é uma Linguagem Regular.

Prova: Por definição, uma linguagem é Regular se, e somente se, é possível construir um Autômato Finito (Determinístico, Não-Determinístico ou com Movimentos Vazios), que reconheça a linguagem. Assim, é necessário mostrar que, dada uma Expressão Regular r qualquer, é possível construir um Autômato Finito M tal que $ACEITA(M) = GERA(r)$. Na construção do correspondente $AF_{\varepsilon} M$ apresentada a seguir, a demonstração de que $ACEITA(M) = GERA(r)$ é por indução no número de operadores.

a) **Base de indução.** Seja r uma ER com zero operadores. Então r só pode ser da forma:

$$r = \emptyset$$

$$r = \varepsilon$$

$$r = x \text{ (} x \text{ pertencente a } \Sigma \text{)}$$

Os Autômatos Finitos:

$$M_1 = (\emptyset, \{q_0\}, \delta_1, q_0, \emptyset)$$

$$M_2 = (\emptyset, \{q_f\}, \delta_2, q_f, \{q_f\})$$

$$M_3 = (\{x\}, \{q_0, q_f\}, \delta_3, q_0, \{q_f\})$$

ilustrados na [figura abaixo](#), aceitam as linguagens acima, respectivamente;

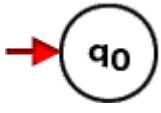
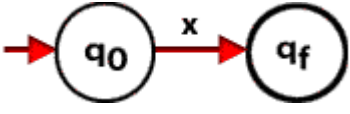
ER	AF correspondente
$r = \emptyset$	
$r = \varepsilon$	
$r = x$	

Figura Autômatos finitos correspondentes às Expressões Regulares com zero operadores

b) *Hipótese de Indução*. Seja r uma ER com até $n > 0$ operadores. Suponha que é possível definir um Autômato Finito que aceita a linguagem gerada por r ;

c) *Passo de Indução*. Seja r uma ER com $n + 1$ operadores. Então r pode ser representada por um dos seguintes casos, onde r_1 e r_2 possuem conjuntamente no máximo n operadores:

$$r = r_1 + r_2$$

$$r = r_1 r_2$$

$$r = r_1^*$$

Portanto, por hipótese de indução é possível construir os autômatos:

$$M_1 = (\Sigma_1, Q_1, \delta_1, q_{01}, \{q_{f1}\}) \quad \text{e} \quad M_2 = (\Sigma_2, Q_2, \delta_2, q_{02}, \{q_{f2}\})$$

tais que $ACEITA(M_1) = GERA(r_1)$ e $ACEITA(M_2) = GERA(r_2)$. Note-se que, sem perda de generalidade, é possível assumir que M_1 e M_2 possuem exatamente um estado final (ver exercícios). Adicionalmente, suponha que os conjuntos de alfabetos e de estados dos dois autômatos são disjuntos (se não forem disjuntos, é suficiente realizar uma simples renomeação). Os Autômatos Finitos com Movimentos Vazios que aceitam a linguagem $GERA(r)$ para cada caso são como segue:

c.1) $r = r_1 + r_2$. O autômato:

$$M = (\Sigma_1 \cup \Sigma_2, Q_1 \cup Q_2 \cup \{q_0, q_f\}, \delta, q_0, \{q_f\})$$

ilustrado na figura abaixo é tal que, a partir do estado inicial q_0 , realiza transições vazias para os estados q_{01} e q_{02} . Assim, M_1 e M_2 processam de forma não-determinística e, portanto, é suficiente um dos módulos aceitar a entrada para o autômato M aceitar;

c.2) $r = r_1 r_2$. O autômato:

$$M = (\Sigma_1 \cup \Sigma_2, Q_1 \cup Q_2, \delta, q_{01}, \{q_{f2}\})$$

ilustrado na figura abaixo, ao processar os módulos M_1 e M_2 em seqüência, aceita uma entrada se, e somente se, o prefixo pertencer a $ACEITA(M_1)$ e o sufixo pertencer a $ACEITA(M_2)$;

c.3) $r = r_1^*$. O autômato:

$$M = (\Sigma_1, Q_1 \cup \{q_0, q_f\}, \delta, q_0, \{q_f\})$$

ilustrado na figura abaixo, é tal que a transição vazia de q_0 para q_{01} garante a aceitação da palavra vazia e o ciclo de q_{f1} para q_{01} permite o sucessivo processamento de M_1 para assegurar o reconhecimento de duas ou mais concatenações sucessivas.

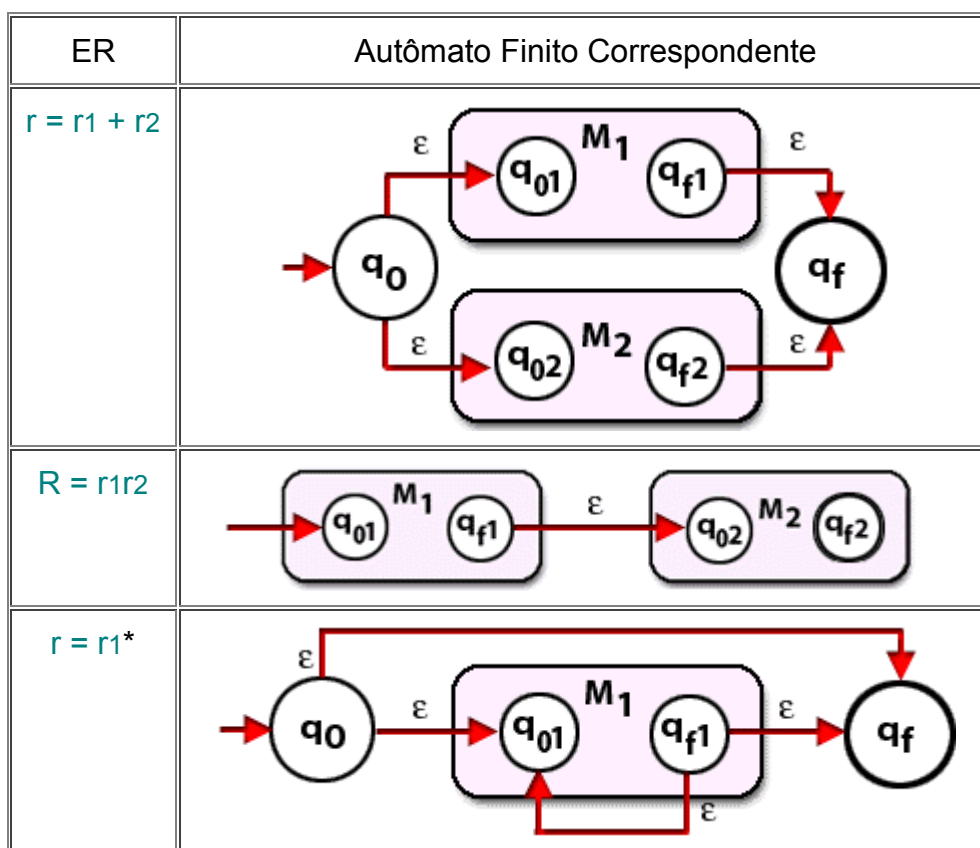


Figura Autômatos finitos correspondentes às Expressões Regulares com $n+1$ operadores

Observação:

É interessante observar que, na prova do Teorema Expressão Regular $\rightarrow$ Linguagem Regular, algumas possíveis alterações na construção do autômato resultante podem gerar resultados indesejados, como por exemplo (por quê?):

- no caso $r = r_1 + r_2$, não introduzir os estados q_0 e q_f e identificar ("unificar") os estados iniciais e finais de M_1 com os correspondentes de M_2 ;
- no caso $r = r_1^*$, não introduzir o estado q_f e manter q_{f1} como o estado final do autômato resultante. Neste caso, a transição vazia que parte de q_0 teria q_{f1} como imagem.

EXEMPLO:

Construção de um AF_ϵ a partir de uma Expressão Regular

A construção dos módulos de um AF_ϵ que aceita a linguagem gerada pela ER $a^*(aa + bb)$ é como ilustrada na figura abaixo e o autômato resultante é ilustrado na figura abaixo (por simplicidade, a identificação dos estados é omitida).

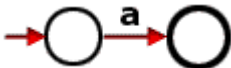
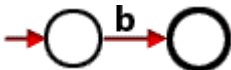
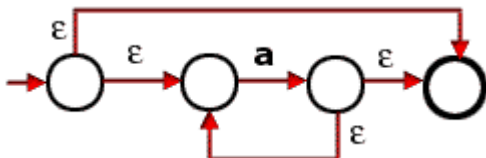
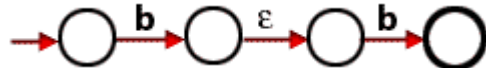
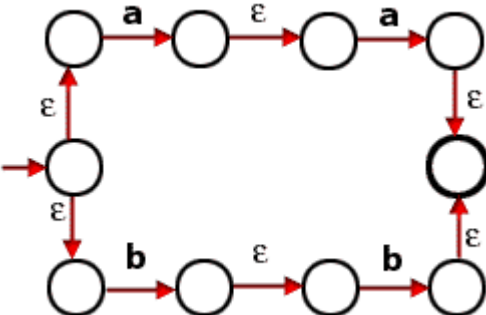
ER	Autômato Finito correspondente
A	
B	
a^*	
Aa	
Bb	
$(aa + bb)$	

Figura Expressões Regulares e os correspondentes Autômatos Finitos

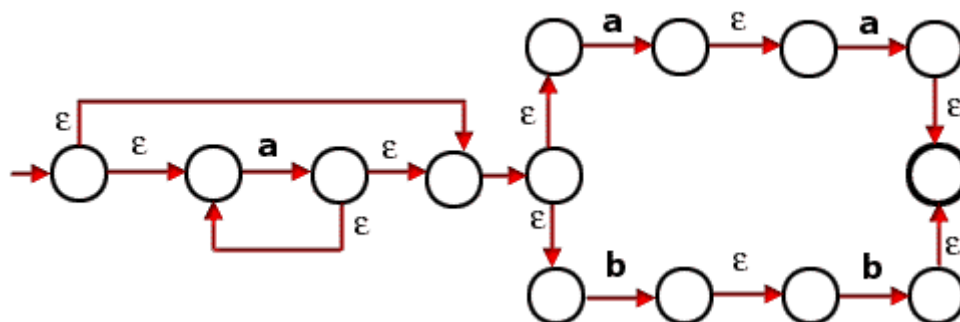


Figura Autômato finito resultante

Teorema:

Linguagem Regular $\rightarrow$ Expressão Regular

Se L é uma Linguagem Regular, então existe uma Expressão Regular r tal que $GERA(r) = L$.

Gramática Regular

Usando o conceito geral de gramáticas como introduzido no Capítulo - [Introdução e Conceitos Básicos](#), é possível definir Linguagens Regulares e Não-Regulares (conforme será visto adiante, é fácil definir Linguagens Não-Regulares). Entretanto, é possível estabelecer restrições nas regras de produção, de tal forma a definir exatamente a Classe de Linguagens Regulares

Definição:

Gramáticas Lineares

Seja $G = (V, T, P, S)$ uma gramática e sejam A e B elementos de V e w uma palavra de T^* . Então G é uma:

- a) *Gramática Linear à Direita* (GLD). Se todas as regras de produção são da forma:

$$A \rightarrow wB \quad \text{ou} \quad A \rightarrow w$$

- b) *Gramática Linear à Esquerda* (GLE). Se todas as regras de produção são da forma:

$$A \rightarrow Bw \quad \text{ou} \quad A \rightarrow w$$

- c) *Gramática Linear Unitária à Direita* (GLUD). Se todas as regras de produção são como na linear à direita e, adicionalmente, $|w| \leq 1$

- d) *Gramática Linear Unitária à Esquerda* (GLUE). Se todas as regras de produção são como na linear à esquerda e, adicionalmente, $|w| \leq 1$

Observação:

Note-se que, nas Gramáticas Lineares, o lado direito de uma produção é constituído por, no máximo, uma variável. Adicionalmente, esta variável, se existir, sempre antecede (linear à esquerda) ou sucede (linear à direita) qualquer subpalavra de terminais.

Teorema:

Equivalência das Gramáticas Lineares

Seja L uma linguagem. Então:

L é gerada por uma GLD se, e somente se,

L é gerada por uma GLE se, e somente se,

L é gerada por uma GLUD se, e somente se,

L é gerada por uma GLUE.

Definição:

Gramática Regular

Uma *Gramática Regular* é qualquer Gramática Linear.

Uma linguagem gerada por uma Gramática Regular G é representada por $L(G)$ ou $GERA(G)$.

EXEMPLO:

Gramática Regular

A linguagem $a(ba)^*$ é gerada pelas seguintes Gramáticas Regulares:

a) *Linear à Direita*. $G = (\{S, A\}, \{a, b\}, P, S)$ onde P possui as seguintes regras de produção:

$$S \rightarrow aA$$

$$A \rightarrow baA \mid \epsilon$$

b) *Linear à Esquerda*. $G = (\{S\}, \{a, b\}, P, S)$ onde P possui as seguintes regras de produção:

$$S \rightarrow Sba \mid a$$

c) *Linear Unitária à Direita*. $G = (\{S, A, B\}, \{a, b\}, P, S)$ onde P possui as seguintes regras de produção:

$$S \rightarrow aA$$

$$A \rightarrow bB \mid \epsilon$$

$$B \rightarrow aA$$

d) *Linear Unitária à Esquerda*. $G = (\{S, A\}, \{a, b\}, P, S)$ onde P possui as seguintes regras de produção:

$$S \rightarrow Aa \mid a$$

$$A \rightarrow Sb$$

EXEMPLO:

Gramática Regular

A linguagem $(a + b)^*(aa + bb)$ é gerada pelas seguintes Gramáticas Regulares:

a) *Linear à Direita*. $G = (\{S, A\}, \{a, b\}, P, S)$ onde P possui as seguintes regras de produção:

$$S \rightarrow aSibSIA$$

$$A \rightarrow aa|bb$$

b) *Linear à Esquerda*. $G = (\{S, A\}, \{a, b\}, P, S)$ onde P possui as seguintes regras de produção:

$$S \rightarrow Aaa|Abb$$

$$A \rightarrow Aa|Ab|\varepsilon$$

Teorema:

Gramática Regular $\rightarrow$ Linguagem Regular.

Se L é uma linguagem gerada por uma Gramática Regular, então L é uma Linguagem Regular.

Prova: Para mostrar que uma linguagem é Regular, é suficiente construir um Autômato Finito que a reconheça. Suponha $G = (V, T, P, S)$ uma Gramática Linear Unitária à Direita. Então o $AF\varepsilon M = (\Sigma, Q, \delta, q_0, F)$ construído abaixo simula as derivações de G , ou seja, $ACEITA(M) = GERA(G)$:

$$\Sigma = T$$

$$Q = V \cup \{q_f\}$$

$$F = \{q_f\}$$

$$q_0 = S$$

δ é construída como segue (suponha A e B variáveis e a terminal):

Tipo da Produção	Transição Gerada
$A \rightarrow \varepsilon$	$\delta(A, \varepsilon) = q_f$
$A \rightarrow a$	$\delta(A, a) = q_f$
$A \rightarrow B$	$\delta(A, \varepsilon) = B$
$A \rightarrow aB$	$\delta(A, a) = B$

A demonstração que, de fato, $ACEITA(M) = GERA(G)$ é no número de derivações, como segue (suponha α elemento de $(T \cup V)^*$ e w elemento de T^*):

a) *Base de indução*. Suponha que $S \Rightarrow^1 \alpha$. Então, se:

a.1) $\alpha = \varepsilon$, existe uma regra $S \rightarrow \varepsilon$. Por construção de M , $\delta(S, \varepsilon) = q_f$

a.2) $\alpha = a$, existe uma regra $S \rightarrow a$. Por construção de M , $\delta(S, a) = q_f$

a.3) $\alpha = A$, existe uma regra $S \rightarrow A$. Por construção de M , $\delta(S, \varepsilon) = A$

a.4) $\alpha = aA$, existe uma regra $S \rightarrow aA$. Por construção de M , $\delta(S, a) = A$

b) *Hipótese de indução*. Suponha que $S \Rightarrow^n \alpha$, $n > 1$ tal que, se:

b.1) $\alpha = w$, então $\delta(S, w) = q_f$

b.2) $\alpha = wA$, então $\delta(S, w) = A$

c) *Passo de Indução*. Suponha que $S \Rightarrow^{n+1} \alpha$. Então, obrigatoriamente ocorre somente a hipótese b.2) acima e $S \Rightarrow^n wA \Rightarrow^1 \alpha$. Portanto, se:

c.1) $\alpha = w\varepsilon = w$, existe uma regra $A \rightarrow \varepsilon$. Assim:

$$\delta(S, w\varepsilon) = \delta(\delta(S, w), \varepsilon) = \delta(A, \varepsilon) = q_f$$

c.2) $\alpha = wb$, existe uma regra $A \rightarrow b$. Assim:

$$\delta(S, wb) = \delta(\delta(S, w), b) = \delta(A, b) = q_f$$

c.3) $\alpha = wB$, existe uma regra $A \rightarrow B$. Assim:

$$\delta(S, w\varepsilon) = \delta(\delta(S, w), \varepsilon) = \delta(A, \varepsilon) = B$$

c.4) $\alpha = wbB$, existe uma regra $A \rightarrow bB$. Assim:

$$\delta(S, wb) = \delta(\delta(S, w), b) = \delta(A, b) = B$$

EXEMPLO:

Construção de um AF_ε a partir de uma Gramática Regular

Considere a seguinte Gramática Linear Unitária à Direita:

$$G = (\{S, A, B\}, \{a, b\}, P, S)$$

onde P é tal que:

$$S \rightarrow aA$$

$$A \rightarrow bB \mid \varepsilon$$

$$B \rightarrow aA$$

O AF_ε M que reconhece a linguagem gerada pela gramática G é:

$$M = (\{a, b\}, \{S, A, B, q_f\}, \delta, S, \{q_f\})$$

o qual é ilustrado na [figura abaixo](#) e onde δ é construída como segue:

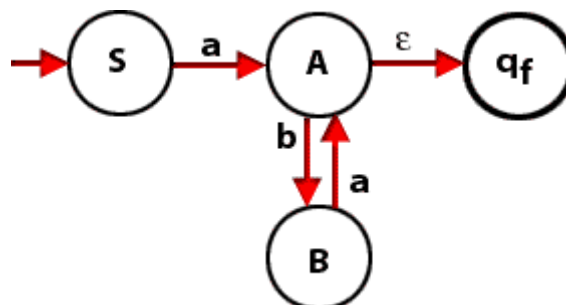


Figura Autômato finito construído a partir de uma Gramática Regular

Produção	Transição
$S \rightarrow aA$	$\delta(S, a) = A$
$A \rightarrow bB$	$\delta(A, b) = B$
$A \rightarrow \epsilon$	$\delta(A, \epsilon) = q_f$
$B \rightarrow aA$	$\delta(B, a) = A$

Teorema:

Linguagem Regular $\rightarrow$ Gramática Regular

Se L é Linguagem Regular, então existe G , Gramática Regular que gera L .

Prova: Se L é uma Linguagem Regular, então existe um AFD $M = (\Sigma, Q, \delta, q_0, F)$ tal que $ACEITA(M) = L$. A idéia central da demonstração é construir uma Gramática Linear à Direita G a partir de M tal que $GERA(G) = ACEITA(M)$, ou seja, cuja derivação simula a função programa estendida. Seja $G = (V, T, P, S)$ tal que:

$$V = Q \cup \{S\}$$

$$T = \Sigma$$

e P é construído como segue (suponha q_i, q_k elementos de Q , q_f elemento de F e a elemento de Σ):

Transição	Produção
-	$S \rightarrow q_0$
-	$q_f \rightarrow \epsilon$
$\delta(q_i, a) = q_k$	$q_i \rightarrow aq_k$

A demonstração que, de fato, $GERA(G) = ACEITA(M)$ é no tamanho da palavra, como segue (suponha w elemento de Σ^*):

a) *Base de Indução*. Seja w tal que $|w| = 0$. Por definição, tem-se que $S \rightarrow q_0$ é produção. Se $\varepsilon \in ACEITA(M)$, então q_0 é estado final e, por definição, tem-se que $q_0 \rightarrow \varepsilon$ é produção. Logo:

$$S \Rightarrow q_0 \Rightarrow \varepsilon$$

b) *Hipótese de Indução*. Seja w tal que $|w| = n$ ($n \geq 1$) e $\delta(q_0, w) = q$. Assim, se:

b.1) q não é estado final, então suponha que $S \Rightarrow wq$

b.2) q é estado final, então suponha que $S \Rightarrow wq \Rightarrow w$ (este caso não é importante para o passo de indução);

c) *Passo de Indução*. Seja w tal que $|wa| = n + 1$ e $\delta(q_0, wa) = p$. Então:

$$\delta(\delta(q_0, w), a) = \delta(q, a) = p$$

Portanto, obrigatoriamente ocorre somente a hipótese b.1) acima e, se:

c.1) p não é estado final, então $S \Rightarrow wq \Rightarrow wqap$

c.2) p é estado final, então $S \Rightarrow wq \Rightarrow wqap \Rightarrow wa$

EXEMPLO:

Construção de uma Gramática Regular a partir de um AFD

Considere o Autômato Finito Determinístico:

$$M = (\{a, b, c\}, \{q_0, q_1, q_2\}, \delta, q_0, \{q_0, q_1, q_2\})$$

ilustrado na figura abaixo.

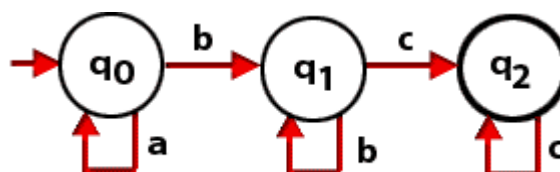


Figura Grafo do Autômato Finito Determinístico

A correspondente Gramática Regular construída conforme o algoritmo da prova do Teorema Linguagem Regular $\rightarrow$ Gramática Regular é:

$$G = (\{q_0, q_1, q_2, S\}, \{a, b, c\}, S, P)$$

onde P é como segue:

Transição	Produção
-	$S \rightarrow q_0$
-	$q_0 \rightarrow \epsilon$
-	$q_1 \rightarrow \epsilon$
-	$q_2 \rightarrow \epsilon$
$\delta(q_0, a) = q_0$	$q_0 \rightarrow aq_0$
$\delta(q_0, b) = q_1$	$q_0 \rightarrow bq_1$
$\delta(q_1, b) = q_1$	$q_1 \rightarrow bq_1$
$\delta(q_1, c) = q_2$	$q_1 \rightarrow cq_2$
$\delta(q_2, c) = q_2$	$q_2 \rightarrow cq_2$

Propriedades das Linguagens Regulares

Uma das principais características das Linguagens Regulares é o fato de serem representadas por formalismos de pouca complexidade, grande eficiência e fácil implementação. Entretanto, por ser uma classe relativamente simples, é restrita e limitada, sendo fácil definir Linguagens Não-Regulares. Assim, algumas questões sobre Linguagens Regulares necessitam ser analisadas:

- Como determinar se uma linguagem é Regular?
- Como verificar se uma Linguagem Regular é infinita ou finita (ou até mesmo vazia)?
- É possível analisar duas Linguagens Regulares e concluir se são iguais ou diferentes?
- A Classe das Linguagens Regulares é fechada para operações de união, concatenação e intersecção?

No estudo que segue, a análise de cada propriedade é desenvolvida para um dos formalismos estudados. Para os demais formalismos, é suficiente traduzi-los usando os algoritmos apresentados nos correspondentes teoremas como ilustrado na [figura abaixo](#) (note-se que, a construção de uma ER a partir de um AFD não foi provada).

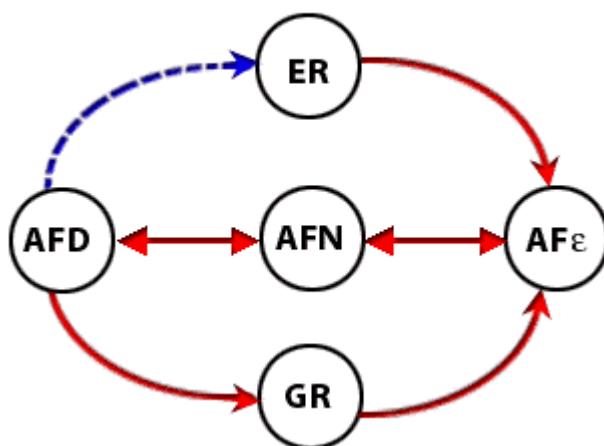


Figura Tradução dos formalismos das Linguagens Regulares

Lema:

Bombeamento para as Linguagens Regulares

Se L é uma Linguagem Regular, então:

existe uma constante n tal que,

para qualquer palavra w de L onde $|w| \geq n$,

w pode ser definida como $w = uvz$ onde $|uv| \leq n$, $|v| \geq 1$ e,

para todo $i \geq 0$, uv^iz é palavra de L .

Prova: Seja L uma Linguagem Regular. Então:

existe um AFD $M = (\Sigma, Q, \delta, q_0, F)$ tal que $ACEITA(M) = L$

seja n o cardinal de Q

seja $w = a_1a_2...a_m$ uma palavra de L tal que $m \geq n$

suponha que $\delta(q_0, a_1) = q_1, \delta(q_1, a_2) = q_2, \dots, \delta(q_{m-1}, a_m) = q_m$

como $m \geq n$ existem r, s onde $0 \leq r < s \leq n$ tais que:

$$q_r = q_s, \delta(q_0, a_1...a_r) = q_r, \delta(q_r, a_{r+1}...a_s) = q_s \text{ e } \delta(q_s, a_{s+1}...a_m) = q_m$$

sejam $u = a_1...a_r$, $v = a_{r+1}...a_s$ e $z = a_{s+1}...a_m$

como $r < s \leq n$, então $|v| \geq 1$ e $|uv| \leq n$

como $q_r = q_s$, então v é reconhecida em um ciclo.

Portanto, uv^iz é palavra de L , para todo $i \geq 0$

Observação:

O *Lema do Bombeamento para as Linguagens Regulares*, é útil no estudo das propriedades. Resumidamente, a idéia básica é a seguinte:

- se uma linguagem é Regular, então é aceita por um Autômato Finito Determinístico o qual possui um número finito e predefinido de n estados;
- se o autômato reconhece uma entrada w de comprimento maior ou igual a n , obrigatoriamente o autômato assume algum estado q mais de uma vez e, portanto, existe um ciclo na função programa que passa por q ;
- logo, w pode ser dividida em três subpalavras $w = uvz$ tal que $|uv| \leq n$, $|v| \geq 1$ e onde v é a parte de w reconhecida pelo ciclo. Portanto, claramente uv^iz , para qualquer $i \geq 0$, é aceita pelo autômato (ou seja, é palavra da linguagem), executando o ciclo i vezes

EXEMPLO:

Bombeamento para as Linguagens Regulares

Considere o autômato ilustrado na figura abaixo (qual a linguagem aceita?). Exemplificando o *Lema do Bombeamento*, tem-se que:

$$n = 4$$

no caso particular de $w = abbbba$, tem-se que:

$$q_r = q_s = q_1$$

$$u = a, v = bb, z = ba$$

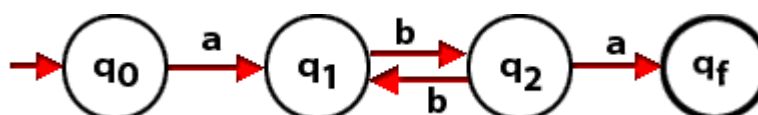


Figura Autômato Finito Determinístico

Linguagens Regulares e Não-Regulares

Para mostrar que uma determinada linguagem é Regular, é suficiente representá-la usando um dos formalismos apresentados (Autômato Finito, Expressão Regular ou Gramática Regular). Entretanto, a prova de que é Não-Regular necessita ser desenvolvida para cada caso. Uma ferramenta útil é o *Lema do Bombeamento*.

EXEMPLO:

Linguagem Não-Regular

A seguinte linguagem sobre $\{a, b\}$ não é Regular:

$$L = \{w \mid w \text{ possui o mesmo número de símbolos } a \text{ e } b\}$$

A prova que segue usa o **Lema do Bombeamento** e é por absurdo. Suponha que L é Regular. Então:

existe um AFD M com n estados que aceita L

seja $w = a^n b^n$

Pelo **Lema do Bombeamento**, w pode ser definida como $w = uvz$ onde:

$$|uv| \leq n, |v| \geq 1 \text{ e,}$$

para todo $i \geq 0$, $uv^i z$ é palavra de L

o que é um absurdo, pois, como $|uv| \leq n$, uv é composta exclusivamente por símbolos a . Neste caso, por exemplo, $uv^2 z$ não pertence a L , pois não possui o mesmo número de símbolos a e b .

Teorema:

Operações Fechadas sobre as Linguagens Regulares

A Classe das Linguagens Regulares é fechada para as seguintes operações:

- união;
- concatenação;
- complemento;
- intersecção.

Prova: A prova referente aos casos de união e concatenação decorrem trivialmente da definição de Expressão Regular (por quê?). Nos demais casos, tem-se que:

a) **Complemento**. Seja L uma Linguagem Regular sobre Σ^* . Seja:

$$M = (\Sigma, Q, \delta, q_0, F)$$

um AFD tal que $ACEITA(M) = L$. A idéia do que segue consiste em inverter as condições de **ACEITA/REJEITA** de M para reconhecer L' . Entretanto, como M pode rejeitar por indefinição é necessário modificar o autômato, garantindo que somente irá parar ao terminar de ler toda a entrada. Assim, a inversão das condições **ACEITA/REJEITA** pode ser obtida transformando os estados finais em não-finais e vice-versa. A construção do AFD:

$$M' = (\Sigma, Q', \delta', q_0, F')$$

tal que $ACEITA(M') = L'$ é como segue (suponha $a \in \Sigma$ e $q \in Q$):

$$Q' = Q \cup \{d\}$$

$$F' = Q' - F$$

δ' é como δ , com as seguintes transições adicionais, para todo $a \in \Sigma$:

$$\delta'(q, a) = d \quad \text{se} \quad \delta(q, a) \text{ não é definida}$$

$$\delta'(d, a) = d$$

Claramente, o Autômato Finito M' construído acima é tal que $ACEITA(M') = L'$

b) **Intersecção**. Sejam L_1 e L_2 Linguagens Regulares. Como a seguinte igualdade é válida:

$$L_1 \cap L_2 = (L_1' \cup L_2)'$$

e como já foi verificado que a Classe das Linguagens Regulares é fechada para as operações de complemento e união, então também é fechada para a intersecção.

Teorema:

Linguagem Regular Vazia, Finita ou Infinita

Se L é uma Linguagem Regular aceita por um Autômato Finito $M = (\Sigma, Q, \delta, q_0, F)$ com n estados (o cardinal de Q é n), então L é:

- a) Vazia se, e somente se, M não aceita qualquer palavra w tal que $|w| < n$
- b) Finita se, e somente se, M não aceita uma palavra w tal que $n \leq |w| < 2n$
- c) Infinita se, e somente se, M aceita uma palavra w tal que $n \leq |w| < 2n$

Infinita ($\Leftarrow$). Processa M para toda a palavra w de comprimento $n \leq |w| < 2n$. Se o autômato aceita alguma palavra então L é infinita. De fato, se $w \in L$ é tal que $n \leq |w| < 2n$, então, pelo **Bombeamento**, $w = uvz$ onde $|uv| \leq n$, $|v| \geq 1$ e, para todo $i \geq 0$, $uv^i z$ é palavra de L . Logo, L é infinita.

Infinita ($\rightarrow$). Se L é infinita, então obviamente existe w tal que $|w| \geq n$. Assim:

- se $|w| < 2n$, então a prova está completa;
- suponha que não existe palavra de comprimento menor que $2n$. Então suponha que $|w| \geq 2n$ mas de comprimento menor ou igual a qualquer outra palavra t tal que $|t| \geq 2n$. Pelo **Bombeamento**, $w = uvz$ onde $|uv| \leq n$, $|v| \geq 1$ e, para todo $i \geq 0$, $uv^i z$ é palavra de L . Em, particular, $1 \leq |v| \leq n$ e uz é palavra de L , o que é um absurdo, pois:
- se $|uz| \geq 2n$, então contradiz a suposição de que w é palavra de menor comprimento tal que $|w| \geq 2n$
- se $|uz| < 2n$, então $n \leq |uz| < 2n$ (pois $|uvz| \geq 2n$, $1 \leq |v| \leq n$) e, portanto, contradiz a suposição de que não existe w tal que $n \leq |w| < 2n$

Vazia. Processa M para todas as palavras de comprimento menor que n . Se rejeita todas as palavras, a linguagem é vazia. O detalhamento da prova é simples usando o **Bombeamento** e é sugerida como exercício.

Finita. É consequência direta do caso onde L é infinita, provado acima. Lembre-se que:

$$(p \leftrightarrow q) \Leftrightarrow (\neg p \leftrightarrow \neg q)$$

Ou seja, processa M para toda palavra w de comprimento $n \leq |w| < 2n$. Se o autômato rejeita todas as palavras, então L é finita.

EXEMPLO:

Linguagem Regular Infinita

Considere o autômato ilustrado na figura abaixo (qual a linguagem aceita?). Exemplificando o Teorema Linguagem Regular Vazia, Finita ou Infinita onde a linguagem é infinita se, e somente se, o autômato aceita uma palavra w tal que $n \leq |w| < 2n$, note-se que a menor palavra aceita cujo comprimento maior ou igual a 3 é $aabaa$ a qual possui comprimento 5. Ou seja, $3 \leq |aabaa| < 6$.

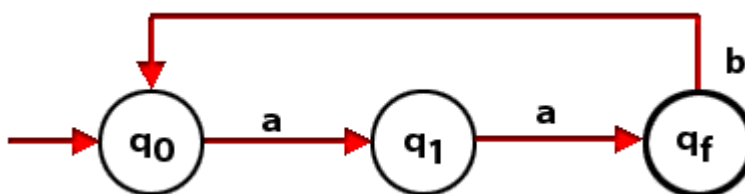


Figura Autômato Finito Determinístico

Teorema:

Igualdade de Linguagens Regulares

Se M_1 e M_2 são Autômatos Finitos, então existe um algoritmo para determinar se $ACEITA(M_1) = ACEITA(M_2)$.

Prova: Sejam M_1 e M_2 Autômatos Finitos tais que $ACEITA(M_1) = L_1$ e $ACEITA(M_2) = L_2$. Portanto, é possível construir um Autômato Finito M_3 tal que $ACEITA(M_3) = L_3$ onde:

$$L_3 = (L_1 \cap L_2') \cup (L_1' \cap L_2)$$

Claramente, $L_1 = L_2$ se, e somente se, L_3 é vazia. Como visto no Teorema Linguagem Regular Vazia, Finita ou Infinita, existe um algoritmo para verificar se uma Linguagem Regular é vazia.

Eficiência de um Autômato Finito como Algoritmo de Reconhecimento

A implementação de um simulador de Autômato Finito Determinístico (sugerida como exercício) consiste, basicamente, em um algoritmo que controla a mudança de estado do autômato a cada símbolo lido da entrada. Assim, o tempo de processamento necessário para aceitar ou rejeitar é diretamente proporcional ao tamanho da entrada. Em termos de Complexidade (parte da Teoria da Computação que estuda os recursos

necessários ao processamento), este procedimento pertence à mais rápida classe de algoritmos.

Deve-se destacar que o tempo de processamento não depende do autômato de reconhecimento. Ou seja, qualquer Autômato Finito Determinístico que reconheça a linguagem terá a mesma eficiência. Entretanto, uma otimização possível é a redução do número de estados. Conforme é visto adiante, existe um algoritmo para construir o Autômato Finito Determinístico mínimo (com o menor número de estados) de qualquer Linguagem Regular.

Minimização de um Autômato Finito

O objetivo da minimização é gerar um Autômato Finito equivalente com o menor número de estados possível. Esta definição de Autômato Mínimo é universalmente aceita e adotada na maioria das soluções práticas. Entretanto, em algumas aplicações especiais, a minimização do número de estados não implica necessariamente no menor custo de implementação. Um exemplo típico seria o desenho de circuitos eletrônicos, quando pode ser desejável introduzir estados intermediários em determinadas posições (do circuito), de forma a melhorar a eficiência, ou simplesmente facilitar as ligações físicas. Portanto, o algoritmo de minimização nestes casos deve ser modificado, prevendo as variáveis específicas da aplicação.

O Autômato Finito Mínimo é único (a menos de isomorfismo). Assim, dois autômatos distintos que aceitam a mesma linguagem ao serem minimizados geram o mesmo Autômato Mínimo, diferenciando-se, eventualmente, na identificação dos estados. Neste caso, os conjuntos dos estados são isomorfos.

Basicamente, o algoritmo de minimização unifica os estados equivalentes. Dois estados q e p são ditos equivalentes se, e somente se, para qualquer palavra w pertencente a Σ^* , $\delta(q, w)$ e $\delta(p, w)$ resultam simultaneamente em estados finais, ou não-finais. Ou seja, o processamento de uma entrada qualquer a partir de estados equivalentes gera, em qualquer caso, o mesmo resultado aceita/rejeita.

Na definição que segue, lembre-se que, para um dado conjunto A , $\#A$ denota o cardinal de A .

Definição:

Autômato Mínimo

Um *Autômato Mínimo* de uma Linguagem Regular L é um Autômato Finito Determinístico $M = (\Sigma, Q, \delta, q_0, F)$ tal que $ACEITA(M) = L$ e que, para qualquer outro Autômato Finito Determinístico $M' = (\Sigma, Q', \delta', q_0', F')$ tal que $ACEITA(M') = L$, tem-se que $\#Q' \geq \#Q$.

Definição:

Pré-Requisitos do Algoritmo de Minimização

Um Autômato Finito a ser minimizado deve satisfazer aos seguintes pré-requisitos:

- a) Deve ser determinístico;
- b) Não pode ter estados inacessíveis (não-atingíveis a partir do estado inicial);
- c) A função programa deve ser total (a partir de qualquer estado são previstas transições para todos os símbolos do alfabeto).

Caso o autômato não satisfaça algum dos pré-requisitos, é necessário gerar um autômato equivalente, como segue:

- a) Gerar um autômato determinístico equivalente, usando os algoritmos introduzidos nas correspondentes demonstrações do teoremas (de $AF\epsilon$ para AFN e de AFN para AFD);
- b) Eliminar os estados inacessíveis e sua correspondentes transições. Trata-se de um algoritmo relativamente simples e é sugerido como exercício;
- c) Para transformar a função programa em total, é suficiente introduzir um novo estado não final d e incluir as transições não previstas, tendo d como estado destino. Por fim, incluir um ciclo em d para todos os símbolos do alfabeto.

Definição:

Algoritmo de Minimização

Suponha um Autômato Finito Determinístico $M = (\Sigma, Q, \delta, q_0, F)$ que satisfaz aos pré-requisitos de minimização. Os passos do *Algoritmo de Minimização* são os seguintes:

- a) *Tabela*. Construir uma tabela relacionando os estados distintos, onde cada par de estados ocorre somente uma vez, como ilustrado na *figura abaixo*;

q_1					
q_2					
...					
q_n					
d					
	q_0	q_1	...	q_{n-1}	q_n

Figura Tabela de estados do algoritmo de minimização de Autômatos Finitos

b) *Marcação dos estados trivialmente não-equivalentes*. Marcar todos os pares do tipo {estado final, estado não-final}, pois, obviamente, estados finais não são equivalentes a não-finais;

c) *Marcação dos estados não-equivalentes*. Para cada par $\{q_u, q_v\}$ não-marcado e para cada símbolo $a \in \Sigma$, suponha que $\delta(q_u, a) = p_u$ e $\delta(q_v, a) = p_v$ e:

- se $p_u = p_v$, então q_u é equivalente a q_v para o símbolo a e não deve ser marcado;
- se $p_u \neq p_v$ e o par $\{p_u, p_v\}$ não está marcado, então $\{q_u, q_v\}$ é incluído em uma lista a partir de $\{p_u, p_v\}$ para posterior análise;
- se $p_u \neq p_v$ e o par $\{p_u, p_v\}$ está marcado, então:
- $\{q_u, q_v\}$ não é equivalente e deve ser marcado;
- se $\{q_u, q_v\}$ encabeça uma lista de pares, então marcar todos os pares da lista (e, recursivamente, se algum par da lista encabeça outra lista);

d) *Unificação dos estados equivalentes*. Os estados dos pares não-marcados são equivalentes e podem ser unificados como segue:

- a equivalência de estados é transitiva;
- pares de estados não-finais equivalentes podem ser unificados como um único estado não-final;
- pares de estados finais equivalentes podem ser unificados como um único estado final;
- se algum dos estados equivalentes é inicial, então o correspondente estado unificado é inicial;

e) *Exclusão dos estados inúteis*. Por fim, os estados chamados inúteis devem ser excluídos. Um estado q é inútil se é não-final e a partir de q não é possível atingir um estado final. Deve-se reparar que o estado d (se incluído) sempre é inútil (o algoritmo para excluir os estados inúteis é simples e é sugerido como *exercício*).

EXEMPLO:

Minimização

Considere o Autômato Finito Determinístico ilustrado na [figura abaixo](#) (qual a linguagem aceita?).

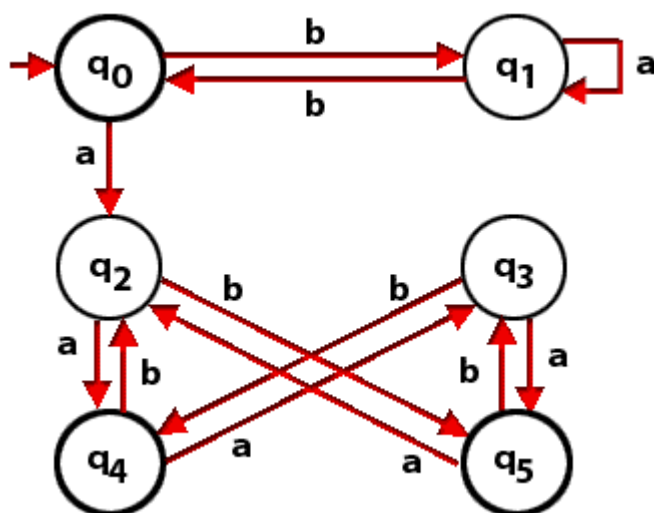


Figura Autômato finito a ser minimizado

O autômato satisfaz os pré-requisitos de minimização (e, conseqüentemente, não é necessário incluir o estado d). Os passos do algoritmo são como segue:

- Construção da tabela, como ilustrado na figura abaixo;
- Marcação dos pares do tipo { estado final, estado não-final }, como ilustrado na mesma figura abaixo;

q_1	×				
q_2	×				
q_3	×				
q_4		×	×	×	
q_5		×	×	×	
	q_0	q_1	q_2	q_3	q_4

Figura Tabela de estados e os pares do tipo {estados finais, estados não-finais} marcados

- Análise dos pares de estado não-marcados, onde a tabela resultante é ilustrada na figura a seguir, sendo o símbolo $\otimes$ usado para representar os pares marcados neste etapa:

c.1) Análise do par { q_0 , q_4 }:

$$\delta(q_0, a) = q_2 \quad \delta(q_0, b) = q_1$$

$$\delta(q_4, a) = q_3 \quad \delta(q_4, b) = q_2$$

Como $\{q_1, q_2\}$ e $\{q_2, q_3\}$ são não-marcados, então $\{q_0, q_4\}$ é incluído nas listas encabeçadas por $\{q_1, q_2\}$ e $\{q_2, q_3\}$;

c.2) Análise do par $\{q_0, q_5\}$:

$$\delta(q_0, a) = q_2 \quad \delta(q_0, b) = q_1$$

$$\delta(q_5, a) = q_2 \quad \delta(q_5, b) = q_3$$

Como $\{q_1, q_3\}$ é não-marcado (e como $\{q_2, q_2\}$ é trivialmente equivalente), então $\{q_0, q_5\}$ é incluído na lista encabeçada por $\{q_1, q_3\}$;

c.3) Análise do par $\{q_1, q_2\}$:

$$\delta(q_1, a) = q_1 \quad \delta(q_1, b) = q_0$$

$$\delta(q_2, a) = q_4 \quad \delta(q_2, b) = q_5$$

Como $\{q_1, q_4\}$ é marcado, então $\{q_1, q_2\}$ também é marcado.

Como $\{q_1, q_2\}$ encabeça uma lista, o par $\{q_0, q_4\}$ também é marcado;

c.4) Análise do par $\{q_1, q_3\}$:

$$\delta(q_1, a) = q_1 \quad \delta(q_1, b) = q_0$$

$$\delta(q_3, a) = q_5 \quad \delta(q_3, b) = q_4$$

Como $\{q_1, q_5\}$ bem como $\{q_0, q_4\}$ são marcados, então $\{q_1, q_3\}$ também é marcado.

Como $\{q_1, q_3\}$ encabeça uma lista, o par $\{q_0, q_5\}$ também é marcado;

c.5) Análise do par $\{q_2, q_3\}$:

$$\delta(q_2, a) = q_4 \quad \delta(q_2, b) = q_5$$

$$\delta(q_3, a) = q_5 \quad \delta(q_3, b) = q_4$$

Como $\{q_4, q_5\}$ é não-marcado, então $\{q_2, q_3\}$ é incluído na lista encabeçada por $\{q_4, q_5\}$;

q_1	$\times$					$\rightarrow \{q_0, q_5\}$
q_2	$\times$	$\otimes$				$\rightarrow \{q_0, q_4\}$
q_3	$\times$	$\otimes$				$\rightarrow \{q_0, q_4\} \rightarrow \{q_4, q_5\}$
q_4	$\otimes$	$\times$	$\times$	$\times$		
q_5	$\otimes$	$\times$	$\times$	$\times$		$\rightarrow \{q_2, q_3\}$
	q_0	q_1	q_2	q_3	q_4	

Figura Tabela de estados resultante

c.6) Análise do par $\{q_4, q_5\}$:

$$\delta(q_4, a) = q_3 \quad \delta(q_4, b) = q_2$$

$$\delta(q_5, a) = q_2 \quad \delta(q_5, b) = q_3$$

Como $\{q_2, q_3\}$ é não-marcado, então $\{q_4, q_5\}$ é incluído na lista encabeçada por $\{q_2, q_3\}$;

d) Como os pares $\{q_2, q_3\}$ e $\{q_4, q_5\}$ são não-marcados, as seguintes unificações podem ser feitas:

- q_{23} representa a unificação dos estados não-finais q_2 e q_3 ;
- q_{45} representa a unificação dos estados finais q_4 e q_5 .

O Autômato Mínimo resultante possui quatro estados e é ilustrado na figura abaixo.

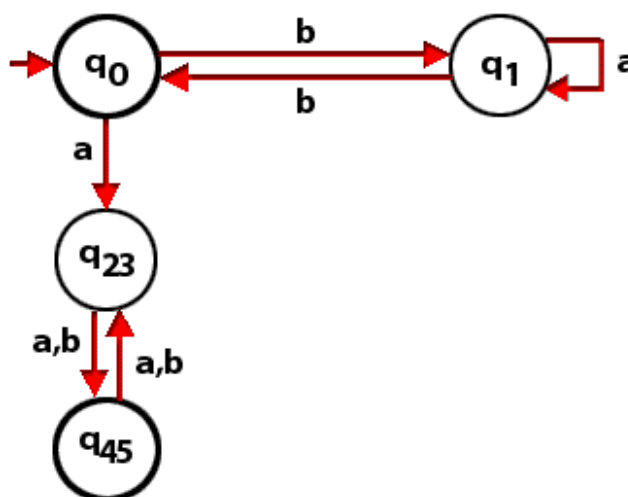


Figura Autômato finito mínimo

Teorema:

Autômato Mínimo

O Autômato Finito Determinístico construído usando o algoritmo de minimização apresentado é o autômato com menor número de estados para a linguagem.

Teorema:

Unicidade do Autômato Mínimo

O Autômato Finito Determinístico mínimo de uma linguagem é único, a menos de isomorfismo.

Autômato Finito com Saída

O conceito básico de Autômato Finito possui aplicações restritas, pois a informação de saída é limitada à lógica binária aceita/rejeita. Sem alterar a classe de linguagens reconhecidas, é possível estender a definição de Autômato Finito incluindo a geração de uma palavra de saída. As saídas podem ser associadas às transições (Máquina de Mealy) ou aos estados (Máquina de Moore). Em ambas as máquinas, a saída não pode ser lida, ou seja, não pode ser usada como memória auxiliar, e é como segue:

- é definida sobre um alfabeto especial, denominado Alfabeto de Saída (pode ser igual ao alfabeto de entrada);
- a saída é armazenada em uma fita independente da de entrada;
- a cabeça da fita de saída move uma célula para direita a cada símbolo gravado;
- o resultado do processamento do Autômato Finito é o seu estado final (condição de aceita/rejeita) e a informação contida na fita de saída.

As Máquinas de Mealy e Moore são modificações sobre o Autômato Finito Determinístico. A extensão da definição prevendo as facilidades de Não-Determinismo e Movimentos Vazios, é sugerida como exercício.

Máquina de Mealy

A Máquina de Mealy é um Autômato Finito modificado de forma a gerar uma palavra de saída para cada transição.

Definição:

Máquina de Mealy

Uma *Máquina de Mealy* M é Autômato Finito Determinístico com saídas associadas às transições. É representada por uma 6-upla:

$$M = (\Sigma, Q, \delta, q_0, F, \Delta)$$

onde:

Σ alfabeto de símbolos de entrada;

Q *conjunto de estados* possíveis do autômato o qual é finito;

δ *função programa* ou *função de transição*:

$$\delta: Q \times \Sigma \rightarrow Q \times \Delta^*$$

a qual é uma função parcial;

q_0 *estado inicial* do autômato tal que q_0 é elemento de Q ;

F *conjunto de estados finais* tal que F está contido em Q ;

Δ *alfabeto de símbolos de saída*.

Observação:

As componentes Σ , Q , q_0 e F são como no Autômato Finito Determinístico. A função programa pode ser representada como um grafo finito direto como nos AFD, adicionando, na etiqueta de cada transição, a saída associada, quando diferente da palavra vazia.

O processamento de uma Máquina de Mealy, para uma palavra de entrada w , consiste na sucessiva aplicação da função programa para cada símbolo de w (da esquerda para a direita) até ocorrer uma condição de parada. A palavra vazia como saída da função programa indica que nenhuma gravação é realizada e, obviamente, não move a cabeça da fita de saída. Se todas as transições geram saída vazia, então a Máquina de Mealy processa como se fosse um Autômato Finito. A definição formal da função programa estendida de uma Máquina de Mealy é sugerida como exercício.

EXEMPLO:

Máquina de Mealy

Uma aplicação comum e freqüentemente recomendada para os autômatos com saída é o projeto de diálogo entre um programa (de computador) e o seu usuário. Basicamente, um diálogo pode ser de dois tipos:

- comandado pelo programa;
- comandado pelo usuário.

Em qualquer caso, uma das principais dificuldades do projetista é a visualização do conjunto de eventos e ações que definam um diálogo adequado para as diversas situações possíveis.

O exemplo que segue é uma Máquina de Mealy que trata algumas situações típicas de um diálogo que cria e atualiza arquivos. A seguinte simbologia é adotada no grafo da função de transição:

- $\langle \dots \rangle$: entrada fornecida pelo usuário (em um teclado, por exemplo);
- "...": saída gerada pelo programa (em um vídeo, por exemplo);
- [...]: ação interna ao programa, sem comunicação com o usuário;

- (...): resultado de uma ação interna ao programa; é usado como entrada no grafo.

A Máquina de Mealy é $M = (\Sigma, \{q_0, q_1, \dots, q_8, q_f\}, \delta, q_0, \{q_f\}, \Delta)$, onde $\Sigma = \Delta$ e representam o conjunto de símbolos (palavras do português) válidos no diálogo.

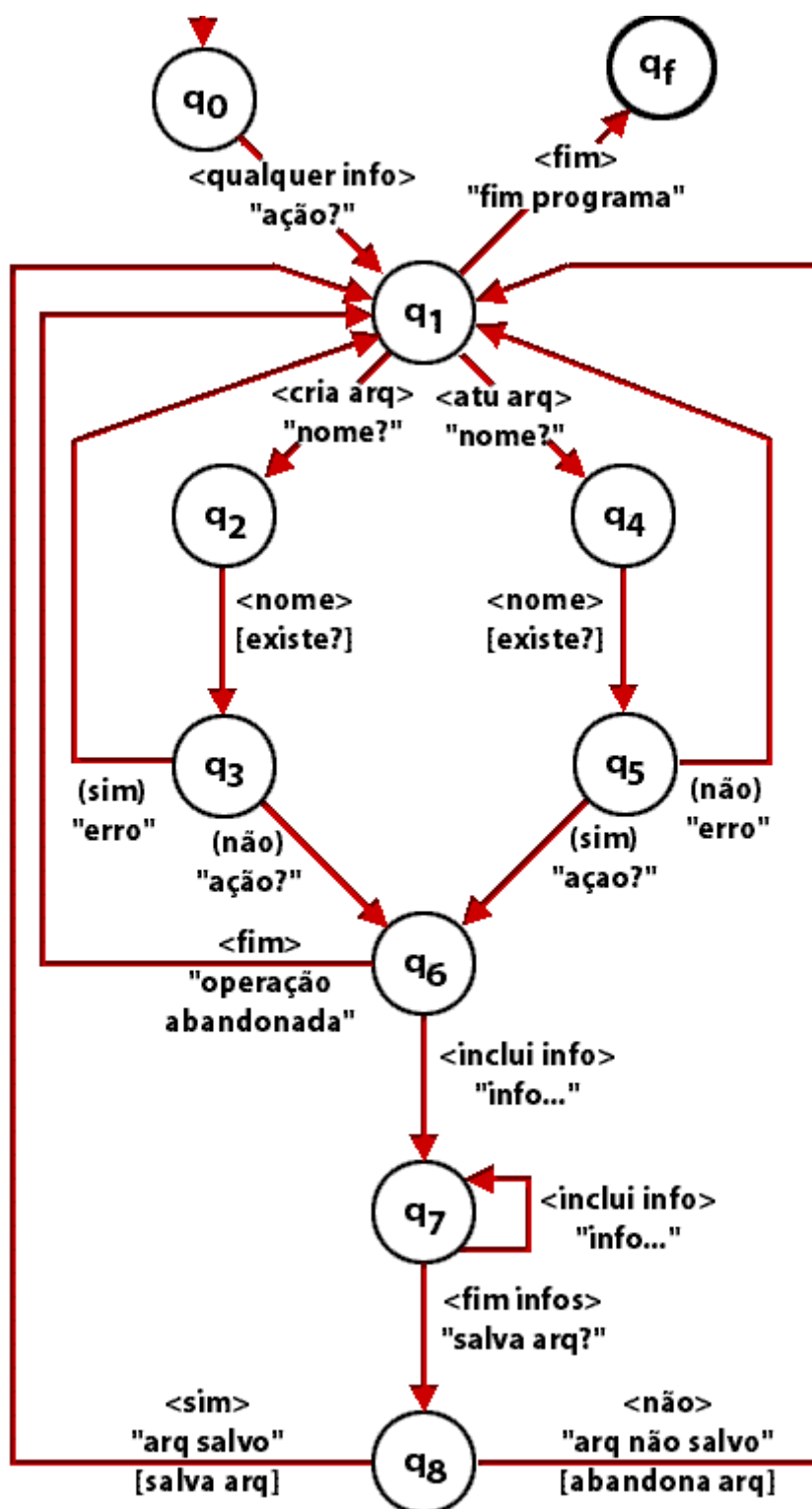


Figura Grafo da Máquina de Mealy

Máquina de Moore

A Máquina de Moore possui uma segunda função, que gera uma palavra de saída (que pode ser vazia) para cada estado da máquina.

Definição:

Máquina de Moore

Uma *Máquina de Moore* M é um Autômato Finito Determinístico com saídas associadas aos estados. É representada por uma 7-upla:

$$M = (\Sigma, Q, \delta, q_0, F, \Delta, \delta S)$$

onde:

Σ *alfabeto de símbolos de entrada*;

Q *conjunto de estados* possíveis do autômato o qual é finito;

δ *função programa* ou *função de transição*:

$$\delta: Q \times \Sigma \rightarrow Q$$

a qual é uma função parcial;

q_0 *estado inicial* tal que q_0 é elemento de Q ;

F *conjunto de estados finais* tal que F está contido em Q ;

Δ *alfabeto de símbolos de saída*;

δS *função de saída*:

$$\delta S: Q \rightarrow \Delta^*$$

a qual é uma função total.

Observação:

As componentes Σ , Q , δ , q_0 e F são como no Autômato Finito Determinístico e Δ é como na Máquina de Mealy. A função programa pode ser representada como um grafo finito direto como nos AFD, adicionando, na etiqueta de cada estado, a saída associada, quando diferente da palavra vazia.

O processamento de uma Máquina de Moore, para uma palavra de entrada w , consiste na sucessiva aplicação da função programa para cada símbolo de w (da esquerda para a direita) até ocorrer uma condição de parada. A palavra vazia resultado da função de saída indica que nenhuma gravação é realizada e, obviamente, não move a cabeça da fita de saída. Se todos os estados geram saída vazia, então a Máquina de Moore

processa como se fosse um Autômato Finito. A definição formal da função programa estendida de uma Máquina de Moore é sugerida como exercício.

EXEMPLO:

Máquina de Moore

Um exemplo comum de aplicação do conceito de Máquina de Moore é o desenvolvimento de Analisadores Léxicos de compiladores ou tradutores de linguagens em geral. Basicamente, um analisador léxico é um Autômato Finito (em geral, determinístico) que identifica os componentes básicos da linguagem como, por exemplo, números, identificadores, separadores, etc. Uma Máquina de Moore como um Analisador Léxico é como segue:

- um estado final é associado a cada unidade léxica;
- cada estado final possui uma saída (definida pela Função de Saída) que descreve ou codifica a unidade léxica identificada;
- para os demais estados (não-finais) a saída gerada é a palavra vazia.

Equivalência das Máquina de Moore e Mealy

A equivalência dos dois modelos de Autômato Finito com Saída não é válida para a entrada vazia. Neste caso, enquanto a Máquina de Moore gera a palavra correspondente ao estado inicial, a Máquina de Mealy não gera qualquer saída, pois não executa transição alguma. Entretanto, para os demais casos, a equivalência pode ser facilmente mostrada.

Teorema:

Máquina de Moore $\rightarrow$ Máquina de Mealy

Toda Máquina de Moore pode ser simulada por uma Máquina de Mealy, para entradas não vazias.

Prova: Suponha $MO = (\Sigma, Q, \delta_{MO}, q_0, F, \Delta, \delta_S)$, uma Máquina de Moore qualquer. Seja:

$$ME = (\Sigma, Q \cup \{q_e\}, \delta_{ME}, q_e, F, \Delta)$$

uma Máquina de Mealy onde a função δ_{ME} é definida como segue (suponha q um estado de Q e a um símbolo de Σ):

$$a) \delta_{ME}(q_e, a) = (\delta_{MO}(q_0, a), \delta_S(q_0)\delta_S(\delta_{MO}(q_0, a)))$$

$$b) \delta_{ME}(q, a) = (\delta_{MO}(q, a), \delta_S(\delta_{MO}(q, a)))$$

Em b), é construída a função programa da Máquina de Mealy, a partir das funções de transição e de saída da Máquina de Moore. O estado q_e introduzido em a) é referenciado somente na primeira transição a ser executada. Seu objetivo é garantir a geração da saída referente ao estado inicial q_0 de Moore (lembre-se que Mealy necessita executar a transição para gerar a saída).

Uma indução em $n \geq 0$, prova que, ao reconhecer a entrada $a_1 \dots a_n$, se M_0 passa pelos estados $q_0, q_1, \dots, q_n$ e gera as saídas $u_0, u_1, \dots, u_n$ então M_E passa pelos estados $q_e, q_0, q_1, \dots, q_n$ e gera as saídas $u_0 u_1, \dots, u_n$.

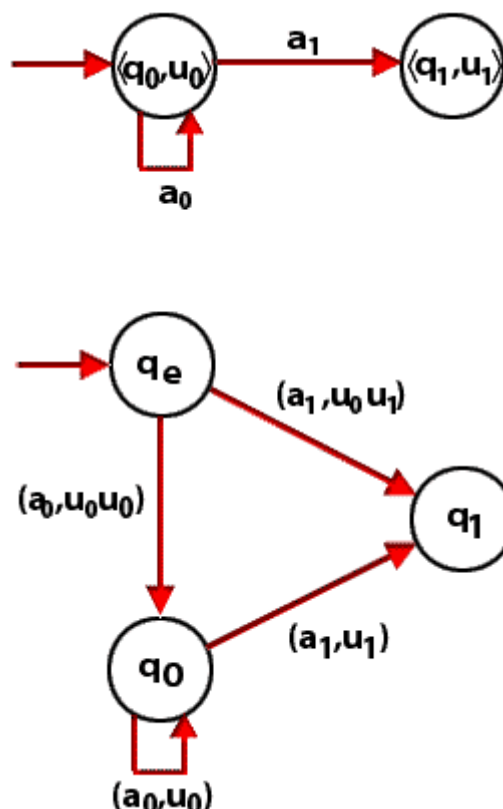


Figura Simulação da Máquina de Moore (acima) usando a Máquina de Mealy (abaixo)

Teorema:

Máquina de Mealy → Máquina de Moore

Toda Máquina de Mealy pode ser simulada por uma Máquina de Moore.

Prova: Suponha $M_E = (\Sigma, Q, \delta_{M_E}, q_0, F, \Delta)$, uma Máquina de Mealy qualquer. Seja $S(\delta_{M_E})$ a imagem da função programa δ_{M_E} restrita à componente da palavra de saída (ou seja, o conjunto de todas as saídas possíveis de M_E).

A construção da Máquina de Moore:

$$M_O = (\Sigma, (Q \times S(\delta_{M_E})) \cup \{q_0, \epsilon\}, \delta_{M_O}, \{q_0, \epsilon\}, F \times S(\delta_{M_E}), \Delta, \delta_S)$$

correspondente determina, em geral, um maior número de estados que a máquina M_E que está sendo simulada. Isto é necessário, pois se diversas transições com saídas diferentes atingem um mesmo estado, este necessita ser simulado por diversos estados, um para cada símbolo de saída, ou seja, em Moore os estados são construídos como um par ordenado, onde a segunda componente representa a palavra de saída, como ilustrado na figura abaixo.

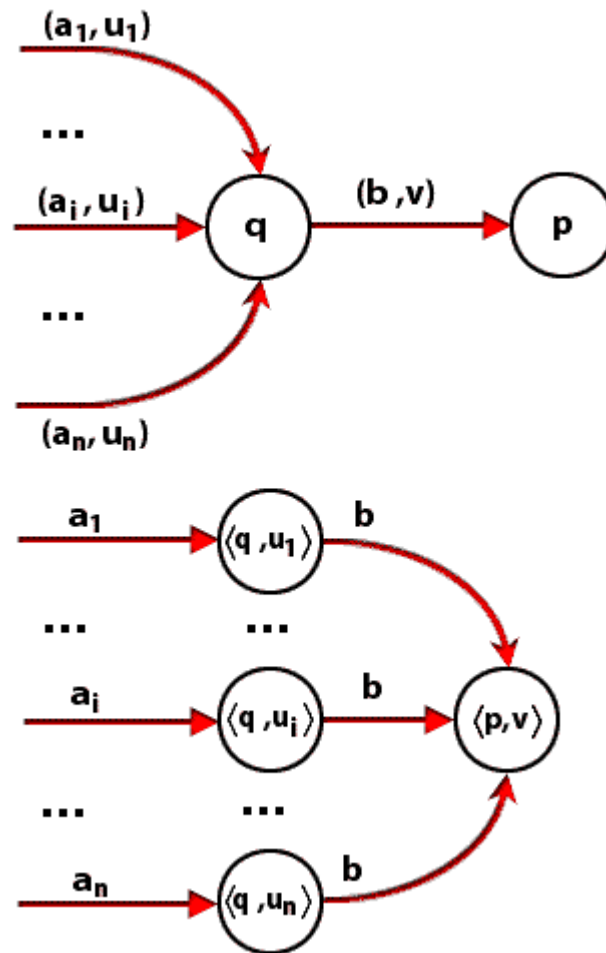


Figura Máquina de Moore (abaixo) que simula uma Máquina de Mealy (acima)

Assim, as funções da Máquina de Moore podem ser construídas a partir de δ_{ME} onde a função programa δ_{MO} é como segue:

- para a em Σ , se $\delta_{ME}(q_0, a) = (q, u)$, então:

$$\delta_{MO}(\langle q_0, \varepsilon \rangle, a) = \langle q, u \rangle$$

- para b em Σ e para q em Q , se $\delta_{ME}(q, b) = (p, v)$, então, para a_i em Σ e para q_i em Q tais que $\delta_{ME}(q_i, a_i) = (q, u_i)$, tem-se que:

$$\delta_{MO}(\langle q, u_i \rangle, b) = \langle p, v \rangle$$

e onde a função de saída δ_S é tal que, para o estado $\langle q, u \rangle$ de MO :

$$\delta_S(\langle q, u \rangle) = u$$

Uma indução em n , prova que, ao reconhecer a entrada $a_1 \dots a_n$, se ME passa pelos estados $q_0, q_1, \dots, q_n$ e gera as saídas $u_1, \dots, u_n$ então MO passa pelos estados $\langle q_0, \varepsilon \rangle, \langle q_1, u_1 \rangle, \dots, \langle q_n, u_n \rangle$ e gera as saídas $\varepsilon, u_1, \dots, u_n$.

EXEMPLO:**Máquina de Moore Simulando uma Máquina de Mealy**

Considere a Máquina de Mealy $ME = (\{a, \beta\}, \{q, p\}, \delta_{ME}, q, \{q, p\}, \{a, \beta\})$ que compacta os brancos de um texto onde a representa um símbolo qualquer do texto e β o símbolo branco, como ilustrado na figura abaixo (na etiqueta de uma transição, a primeira componente representa o símbolo lido e a segunda a palavra gravada).

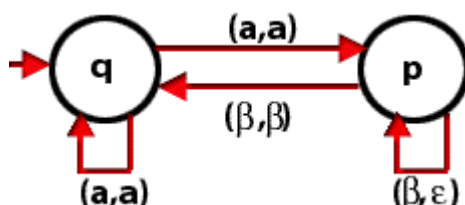


Figura Grafo da Máquina de Mealy

A Máquina de Moore construída conforme a prova do Teorema Máquina de Mealy $\rightarrow$ Máquina de Moore é $MO = (\{a, \beta\}, Q, \delta_{MO}, \langle q, \epsilon \rangle, F, \{a, \beta\}, \delta_S)$ tal que $Q = F = \{q, p\} \times \{\epsilon, a, \beta\}$, ilustrada na figura abaixo onde a segunda componente de cada estado representa a saída.

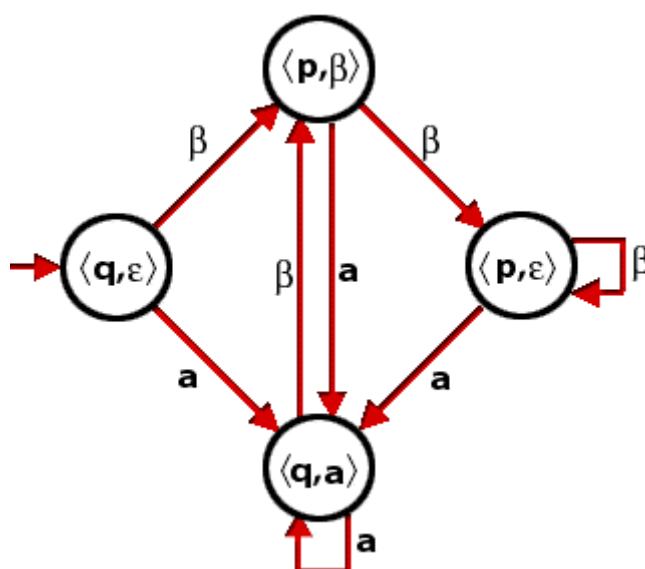


Figura Grafo da correspondente Máquina de Moore

Linguagens Livres do Contexto

A **Classe das Linguagens Livres do Contexto** ou **Tipo 2** contém propriamente a Classe das Linguagens Regulares. Seu estudo é de fundamental importância na informática pois:

- compreende um universo mais amplo de linguagens (comparativamente com as regulares) tratando, adequadamente, questões como parênteses balanceados, construções bloco-estruturadas, entre outras, típicas de linguagens de programação como Pascal, C, Algol, etc.;

- os algoritmos reconhecedores e geradores que implementam as Linguagens Livres do Contexto são relativamente simples e possuem uma boa eficiência;
- exemplos típicos de aplicações dos conceitos e resultados referentes às Linguagens Livres do Contexto são analisadores sintáticos, tradutores de linguagens e processadores de texto em geral.

O estudo da Classe das Linguagens Livres do Contexto é desenvolvido a partir de um formalismo axiomático ou gerador (gramática) e um operacional ou reconhecedor (autômato), como segue:

- a) **Gramática Livre do Contexto**. Gramática onde as regras de produção são definidas de forma mais livre que na Gramática Regular;
- b) **Autômato com Pilha**. Autômato cuja estrutura básica é análoga à do Autômato Finito, adicionando uma memória auxiliar tipo pilha (a qual pode ser lida ou gravada) e a facilidade de não-determinismo.

Gramática Livre do Contexto

As Linguagens Livres do Contexto são definidas a partir das Gramáticas Livres do Contexto.

Definição:

Gramática Livre do Contexto

Uma **Gramática Livre do Contexto** (GLC) G é uma gramática:

$$G = (V, T, P, S)$$

com a restrição de que qualquer regra de produção de P é da forma $A \rightarrow \alpha$, onde A é uma variável de V e α uma palavra de $(V \cup T)^*$.

Portanto, uma Gramática Livre do Contexto é uma gramática onde o lado esquerdo das produções contém exatamente uma variável.

Definição:

Linguagem Livre do Contexto ou Tipo 2

Uma linguagem é dita **Linguagem Livre do Contexto** (LLC) ou **Tipo 2** se for gerada por uma Gramática Livre do Contexto.

O nome "Livre do Contexto" se deve ao fato de representar a mais geral classe de linguagens cuja produção é da forma $A \rightarrow \alpha$. Ou seja, em uma derivação, a variável A deriva α sem depender ("Livre") de qualquer análise dos símbolos que antecedem ou sucedem A ("Contexto") na palavra que está sendo derivada. Assim, claramente, toda Linguagem Regular é Livre do Contexto.

EXEMPLO:*Linguagem Livre do Contexto - Duplo Balanceamento.*

Considere a linguagem:

$$L1 = \{ a^n b^n \mid n \geq 0 \}$$

A seguinte Gramática Livre do Contexto:

$$G1 = (\{ S \}, \{ a, b \}, P1, S), \text{ onde:}$$

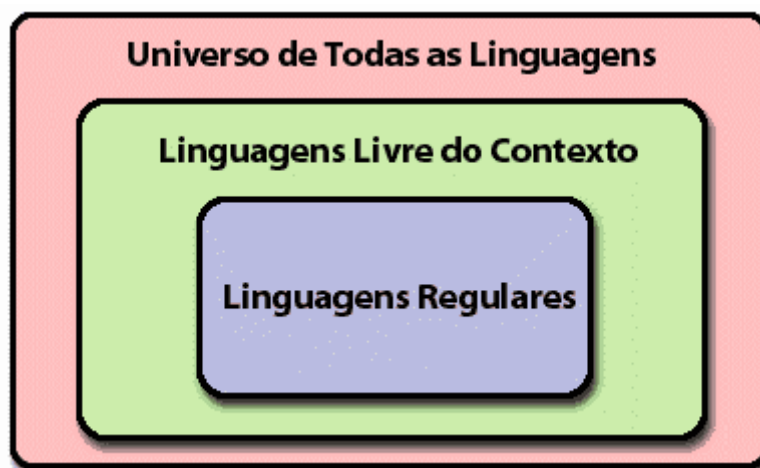
$$P1 = \{ S \rightarrow aSb \mid S \rightarrow \epsilon \}$$

é tal que $GERA(G1) = L1$. Por exemplo, a palavra **aabb** pode ser gerada pela seguinte seqüência de derivação:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aa\epsilon bb = aabb$$

Esta linguagem é um exemplo clássico e de fundamental importância no estudo das Linguagens Livres do Contexto, pois permite estabelecer analogia entre $a^n b^n$ e linguagens que possuem duplo balanceamento como, por exemplo:

- a) Linguagens bloco-estruturadas do tipo $BEGIN^n END^n$
- b) Linguagens com parênteses balanceados na forma $(^n)^n$

**EXEMPLO:***Linguagem Livre do Contexto - Expressões Aritméticas.*

A linguagem **L2** gerada pela Gramática Livre do Contexto abaixo é composta de expressões aritméticas contendo colchetes balanceados, dois operadores e um operando:

$$G2 = (\{ E \}, \{ +, *, [,], x \}, P2, E), \text{ onde:}$$

$$P_2 = \{ E \rightarrow E + E \mid E * E \mid [E] \mid x \}$$

Por exemplo, a expressão $[x+x]^*x$ pode ser gerada pela seguinte seqüência de derivação:

$$E \Rightarrow E * E \Rightarrow [E]^* E \Rightarrow [E + E]^* E \Rightarrow [x + E]^* E \Rightarrow [x + x]^* E \Rightarrow [x + x]^* x$$

É possível gerar a mesma expressão com outra seqüência de derivação? Quantas seqüências distintas são possíveis?

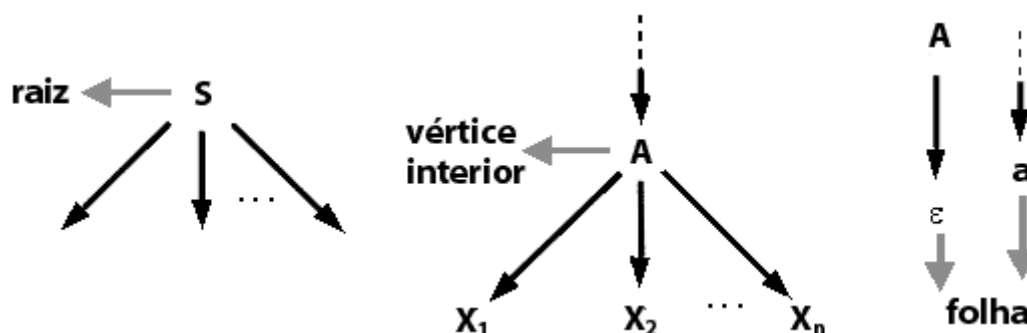
Árvore de Derivação

Em algumas aplicações como compiladores e processadores de textos, freqüentemente é conveniente representar a derivação de palavras na forma de árvore, partindo do símbolo inicial como a raiz, e terminando em folhas de terminais.

Definição:

Árvore de Derivação

Para uma determinada Gramática Livre do Contexto, a representação da derivação de palavras na forma de árvore, denominada *Árvore de Derivação*, é como segue (considere a Figura):



a) A *raiz* é o símbolo inicial da gramática;

b) Os *vértices interiores* obrigatoriamente são variáveis. Se A é um vértice interior e $X_1, X_2, \dots, X_n$ são os filhos de A , então $A \rightarrow X_1 X_2 \dots X_n$ é uma produção da gramática e os vértices $X_1, X_2, \dots, X_n$ estão ordenados da esquerda para a direita;

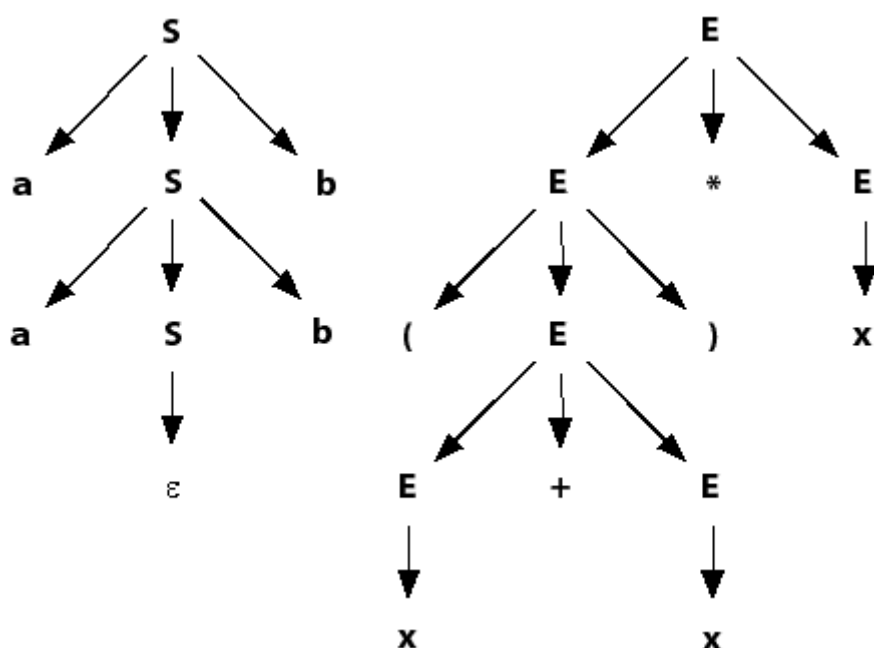
c) Um *vértice folha* é um símbolo terminal, ou o símbolo vazio. Neste caso, o vazio é o único filho de seu pai ($A \rightarrow \epsilon$).

EXEMPLO:

Árvore de Derivação

A palavra $aabb$ e a expressão $[x+x]^*x$ são geradas pelas árvores de derivação ilustradas na figura abaixo, à esquerda e à direita, respectivamente.

Uma única árvore de derivação pode representar derivações distintas de uma mesma palavra.

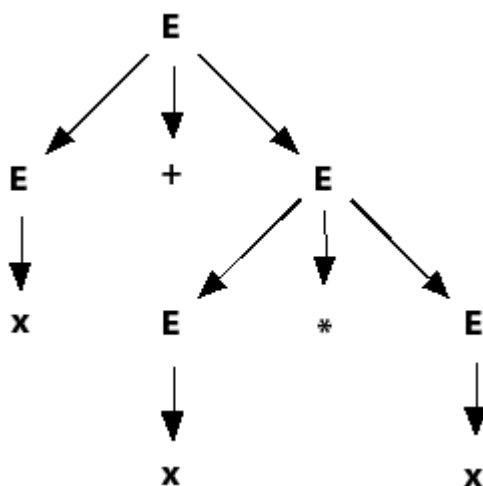


EXEMPLO:

Árvore de Derivação X Derivações

Na árvore representada na [figura abaixo](#), a palavra $x+x*x$ pode ser gerada por diversas derivações, como segue:

- a) $E \Rightarrow E+E \Rightarrow x+E \Rightarrow x+E*E \Rightarrow x+x*E \Rightarrow x+x*x$
- b) $E \Rightarrow E+E \Rightarrow E+E*E \Rightarrow E+E*x \Rightarrow E+x*x \Rightarrow x+x*x$
- c) $E \Rightarrow E+E \Rightarrow E+E*E \Rightarrow x+E*E \Rightarrow x+x*E \Rightarrow x+x*x$
- d) etc...



Definição:

Derivação mais à Esquerda (Direita)

A *Derivação mais à Esquerda (Direita)* de uma árvore de derivação é a sequência de produção aplicada sempre à variável mais à esquerda (*direita*).

Ambigüidade

Eventualmente, uma mesma palavra pode ser associada a duas ou mais árvores de derivação, determinando uma ambigüidade. Em muitas aplicações como, por exemplo, no desenvolvimento e otimização de alguns tipos de algoritmos de reconhecimento, é conveniente que a gramática usada não seja ambígua. Entretanto, nem sempre é possível eliminar ambigüidades. Na realidade, é fácil definir linguagens para as quais qualquer Gramática Livre do Contexto é ambígua.

Definição:

Gramática Ambígua

Uma Gramática Livre do Contexto é dita uma *Gramática Ambígua*, se existe uma palavra que possua duas ou mais árvores de derivação.

EXEMPLO:

Gramática Ambígua

A palavra $x+x*x$ pode ser gerada por árvores distintas. Portanto, a gramática em questão é ambígua (tente desenvolver uma gramática não-ambígua para esta linguagem).

Note-se que a palavra $x+x*x$ possui mais de uma derivação à esquerda (direita), como segue:

a) Derivação mais à esquerda:

$$E \Rightarrow E+E \Rightarrow x+E \Rightarrow x+E^*E \Rightarrow x+x^*E \Rightarrow x+x^*x$$

$$E \Rightarrow E^*E \Rightarrow E+E^*E \Rightarrow x+E^*E \Rightarrow x+x^*E \Rightarrow x+x^*x$$

b) Derivação mais à direita:

$$E \Rightarrow E+E \Rightarrow E+E^*E \Rightarrow E+E^*x \Rightarrow E+x^*x \Rightarrow x+x^*x$$

$$E \Rightarrow E^*E \Rightarrow E^*x \Rightarrow E+E^*x \Rightarrow E+x^*x \Rightarrow x+x^*x$$

Prova-se que, uma forma equivalente de definir ambigüidade de uma gramática é a existência de uma palavra com duas ou mais derivações mais à esquerda (direita).

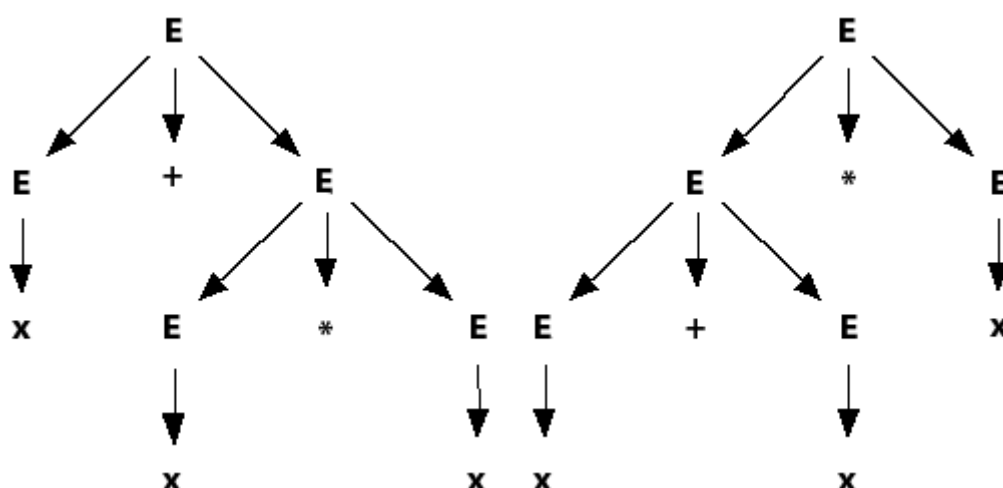


Figura Ambigüidade: árvores diferentes para uma mesma palavra

Definição:

Linguagem Inerentemente Ambígua

Uma linguagem é uma *Linguagem Inerentemente Ambígua* se qualquer Gramática Livre do Contexto que a define é ambígua.

EXEMPLO:

Linguagem Inerentemente Ambígua

A linguagem:

$$\{w \mid w = a^n b^n c^m d^m \text{ ou } w = a^n b^m c^m d^n, n \geq 1, m \geq 1\}$$

é inerentemente ambígua.

Simplificação de Gramáticas Livres do Contexto

É possível simplificar alguns tipos de produções sem reduzir o poder de geração das Gramáticas Livres do Contexto. Em geral, as simplificações de gramáticas são usadas na construção e otimização de algoritmos e na demonstração de teoremas.

As seguintes simplificações são apresentadas:

- exclusão de variáveis ou terminais não-usados para gerar palavras;
- exclusão de produções vazias da forma $A \rightarrow \epsilon$ (se a palavra vazia pertence à linguagem, é incluída uma produção vazia específica para tal fim);
- exclusão de produções da forma $A \rightarrow B$, ou seja, que simplesmente substituem uma variável por outra e, conseqüentemente, não adicionam qualquer informação de geração de palavras.

No texto que segue, são omitidas as provas de que os algoritmos introduzidos, de fato, atingem os objetivos propostos.

Símbolos Inúteis

A exclusão de símbolos inúteis (não-usados na geração de palavras de terminais) é realizada excluindo as produções que fazem referência a estes símbolos, bem como os próprios símbolos inúteis. Não é necessária qualquer modificação adicional nas produções da gramática. O algoritmo apresentado é dividido em duas etapas, como segue:

- a) *Qualquer variável gera palavra de terminais*. O algoritmo gera um novo conjunto de variáveis, analisando as produções da gramática a partir de terminais gerados. Inicialmente, considera todas as variáveis que geram terminais diretamente (exemplo: $A \rightarrow a$). A seguir, sucessivamente são adicionadas as variáveis que geram palavras de terminais indiretamente (exemplo: $B \rightarrow Ab$);
- b) *Qualquer símbolo é atingível a partir do símbolo inicial*. Após a execução da etapa acima, o algoritmo analisa as produções da gramática a partir do símbolo inicial. Inicialmente, considera exclusivamente o símbolo inicial. Após, sucessivamente as produções da gramática são aplicadas e os símbolos referenciados adicionados aos novos conjuntos.

Definição:

Algoritmo para Exclusão dos Símbolos Inúteis

Considere uma Gramática Livre do Contexto $G = (V, T, P, S)$. O *Algoritmo para Exclusão dos Símbolos Inúteis* é composto por duas etapas, como segue:

- a) *Etapa 1: garante que qualquer variável gera terminais*. A gramática resultante desta etapa é:

$$G_1 = (V_1, T, P_1, S)$$

onde V_1 é construído como segue:

$$V_1 = \emptyset;$$

repita

$$V_1 = V_1 \cup \{A \mid A \rightarrow \alpha \in P \text{ e } \alpha \in (T \cup V_1)^*\}$$

até que o cardinal de V_1 não aumente;

O conjunto P_1 possui os mesmos elementos que P excetuando-se as produções cujas variáveis não pertencem a V_1 ;

- b) *Etapa 2: garante que qualquer símbolo é atingível a partir do símbolo inicial*. A gramática resultante desta etapa é:

$$G_2 = (V_2, T_2, P_2, S)$$

onde V_2 e T_2 são construídos como segue:

$$T_2 = \emptyset;$$

$$V_2 = \{S\};$$

repita

$$V_2 = V_2 \cup \{A \mid X \rightarrow \alpha A \beta \in P_1, X \in V_2\};$$

$$T_2 = T_2 \cup \{a \mid X \rightarrow \alpha a \beta \in P_1, X \in V_2\}$$

até que os cardinais de V_2 e T_2 não aumentem;

O conjunto P_2 possui os mesmos elementos que P_1 excetuando-se as produções cujos símbolos não pertencem a V_2 ou T_2

Deve-se reparar que se as etapas acima forem executadas em ordem inversa (etapa 2 antes da 1), o algoritmo pode não gerar o resultado desejado. Para demonstrar, é suficiente apresentar um contra-exemplo, o que se sugere como **exercício** (lembre-se que, uma demonstração por contra-exemplo é, de fato, uma demonstração por absurdo).

EXEMPLO:

Exclusão dos Símbolos Inúteis

Considere a seguinte Gramática Livre do Contexto:

$$G = (\{S, A, B, C\}, \{a, b, c\}, P, S), \text{ onde:}$$

$$P = \{S \rightarrow aAa \mid bBb, A \rightarrow a \mid S, C \rightarrow c\}$$

A exclusão dos símbolos inúteis é como segue:

a) **Qualquer variável gera palavra de terminais**. A coluna "iteração" representa o número de ciclos do comando **repita-até**, e a coluna "variáveis" o conjunto de variáveis construído após a iteração:

iteração	Variáveis
início	$\emptyset$
1	$\{A, C\}$
2	$\{A, C, S\}$
3	$\{A, C, S\}$

O algoritmo pára na terceira iteração, pois nenhuma variável foi adicionada ao conjunto. A produção $S \rightarrow bBb$ é excluída, pois B não pertence ao novo conjunto de variáveis;

b) *Qualquer símbolo é atingido a partir de S.*

iteração	variáveis	Terminais
início	$\{S\}$	$\emptyset$
1	$\{S, A\}$	$\{a\}$
2	$\{S, A\}$	$\{a\}$

A produção $C \rightarrow c$ é excluída, pois C e c não pertencem aos novos conjuntos de variáveis e terminais, respectivamente. A gramática resultante é a seguinte:

$G = (\{S, A\}, \{a\}, P, S)$, onde:

$P = \{S \rightarrow aAa, A \rightarrow a \mid S\}$

Produções Vazias

A exclusão de produções vazias (da forma $A \rightarrow \varepsilon$) pode determinar modificações diversas nas produções da gramática. O algoritmo é dividido em três etapas, como segue:

- Variáveis que constituem produções vazias.* Considera, inicialmente, todas as variáveis que geram diretamente ε (ex.: $A \rightarrow \varepsilon$). A seguir, sucessivamente são determinadas as variáveis que indiretamente geram ε (ex.: $B \rightarrow A$);
- Exclusão de produções vazias.* Inicialmente, são consideradas todas as produções não-vazias. A seguir, cada produção cujo lado direito possui uma variável que gera a palavra vazia, determina uma produção adicional, sem esta variável;
- Inclusão de geração da palavra vazia, se necessário.* Se a palavra vazia pertence à linguagem, então é incluída uma produção para gerar a palavra vazia.

Definição:

Algoritmo para Exclusão das Produções Vazias

Considere uma Gramática Livre do Contexto $G = (V, T, P, S)$. O *Algoritmo para Exclusão das Produções Vazias* é composto por três etapas, como segue:

- Etapa 1: Conjunto de variáveis que constituem produções vazias.* O algoritmo para construir V_ε (conjunto das variáveis que geram ε) é o seguinte:

$V_\varepsilon = \{A \mid A \rightarrow \varepsilon\};$

repita

$$V_\varepsilon = V_\varepsilon \cup \{X \mid X \rightarrow X_1 \dots X_n \in P \text{ tal que } X_1, \dots, X_n \in V_\varepsilon\}$$

até que o cardinal de V_ε não aumente;

b) **Etapa 2:** *Conjunto de produções sem produções vazias.* A gramática resultante desta etapa é:

$$G_1 = (V, T, P_1, S)$$

onde P_1 é construído como segue:

$$P_1 = \{A \rightarrow \alpha \mid \alpha \neq \varepsilon\};$$

repita

para toda $A \rightarrow \alpha \in P_1$ e $X \in V_\varepsilon$ tal que $\alpha = \alpha_1 X \alpha_2$ e $\alpha_1 \alpha_2 \neq \varepsilon$

$$\text{faça } P_1 = P_1 \cup \{A \rightarrow \alpha_1 \alpha_2\}$$

até que o cardinal de P_1 não aumente;

c) **Etapa 3:** *inclusão de geração da palavra vazia, se necessário.* Se a palavra vazia pertence à linguagem, então a gramática resultante desta etapa é:

$$G_2 = (V, T, P_2, S) \text{ onde:}$$

$$P_2 = P_1 \cup \{S \rightarrow \varepsilon\}$$

EXEMPLO:

Exclusão das Produções Vazias

Considere a seguinte Gramática Livre do Contexto:

$$G = (\{S, X, Y\}, \{a, b\}, P, S), \text{ onde:}$$

$$P = \{S \rightarrow aXa \mid bXb \mid \varepsilon, X \rightarrow a \mid b \mid Y, Y \rightarrow \varepsilon\}$$

A exclusão das produções vazias é como segue:

a) **Conjunto de variáveis que geram ε .**

Iteração	V_ϵ
Início	$\{S, Y\}$
1	$\{S, Y, X\}$
2	$\{S, Y, X\}$

b) *Conjunto de produções sem produções vazias.*

Iteração	Produções
Inicial	$\{S \rightarrow aXa \mid bXb, X \rightarrow a \mid b \mid Y\}$
1	$\{S \rightarrow aXa \mid bXb \mid aa \mid bb, X \rightarrow a \mid b \mid Y\}$
2	$\{S \rightarrow aXa \mid bXb \mid aa \mid bb, X \rightarrow a \mid b \mid Y\}$

c) *Inclusão da geração da palavra vazia.* Como a palavra vazia pertence à linguagem, $S \rightarrow \epsilon$ é incluída no conjunto de produções.

A gramática resultante é a seguinte:

$G = (\{S, X, Y\}, \{a, b\}, P, S)$, onde:

$P = \{S \rightarrow aXa \mid bXb \mid aa \mid bb \mid \epsilon, X \rightarrow a \mid b\}$

Deve-se reparar que embora Y pertença ao conjunto de variáveis, a produção que possui Y no lado esquerdo foi excluída. Ou seja, a exclusão de produções vazias gerou um símbolo inútil. Veja adiante seção específica sobre a combinação de simplificações de gramáticas.

Produções da Forma $A \rightarrow B$

Uma produção da forma $A \rightarrow B$ não adiciona informação alguma em termos de geração de palavras, a não ser que a variável A pode ser substituída por B . Neste caso, se $B \rightarrow \alpha$, então a produção $A \rightarrow B$ pode ser substituída por $A \rightarrow \alpha$. A generalização desta ideia é o algoritmo proposto, dividido em duas etapas, como segue:

- Construção do fecho da cada variável.* Entende-se por fecho de uma variável o conjunto de variáveis que podem substituí-la transitivamente. Ou seja, se $A \rightarrow B$ e $B \rightarrow C$, então B e C pertencem ao fecho de A ;
- Exclusão das produções da forma $A \rightarrow B$.* Substitui as produções da forma $A \rightarrow B$ por produções da forma $A \rightarrow \alpha$, onde α é atingível a partir de A através de seu fecho.

Definição:

Algoritmo para Exclusão das Produções da Forma $A \rightarrow B$

Considere uma Gramática Livre do Contexto $G = (V, T, P, S)$. O *Algoritmo para Exclusão de Produções da Forma $A \rightarrow B$* é composto por duas etapas, como segue:

a) *Construção do fecho da cada variável.*

para toda $A \in V$

faça $\text{FECHO-A} = \{B \mid A \neq B \text{ e } A \Rightarrow^+ B\}$

usando exclusivamente produções da forma $X \rightarrow Y$;

b) *Exclusão das produções da forma $A \rightarrow B$.* A gramática resultante desta etapa é:

$$G_1 = (V, T, P_1, S)$$

onde P_1 é construído como segue:

$$P_1 = \{A \rightarrow \alpha \mid \alpha \notin V\};$$

para toda $A \in V$ e $B \in \text{FECHO-A}$

faça se $B \rightarrow \alpha \in P$ e $\alpha \notin V$

então $P_1 = P_1 \cup \{A \rightarrow \alpha\}$;

EXEMPLO:

Exclusão das Produções da Forma $A \rightarrow B$

Considere a seguinte Gramática Livre do Contexto:

$$G = (\{S, X\}, \{a, b\}, P, S), \text{ onde:}$$

$$P = \{S \rightarrow aXa \mid bXb, X \rightarrow a \mid b \mid S \mid \varepsilon\}$$

A exclusão da produção $X \rightarrow S$ é como segue:

a) *Construção do fecho de cada variável.*

$$\text{FECHO-S} = \emptyset$$

$$\text{FECHO-X} = \{S\}$$

b) *Exclusão das produções da forma $A \rightarrow B$.* Construção do conjunto de produções (a coluna iteração representa a execução do algoritmo para a variável referenciada):

Iteração	Produções
Inicial	$\{S \rightarrow aXa \mid bXb, X \rightarrow a \mid b \mid \varepsilon\}$
S	$\{S \rightarrow aXa \mid bXb, X \rightarrow a \mid b \mid \varepsilon\}$
X	$\{S \rightarrow aXa \mid bXb, X \rightarrow a \mid b \mid \varepsilon \mid aXa \mid bXb\}$

A gramática resultante é a seguinte:

$G = (\{S, X\}, \{a, b\}, P, S)$, onde:

$P = \{S \rightarrow aXa \mid bXb, X \rightarrow a \mid b \mid \varepsilon \mid aXa \mid bXb\}$

Simplificações Combinadas

Deve-se reparar que não é qualquer seqüência de simplificação de gramática que atinge os resultados desejados. Por exemplo, em uma gramática *sem* símbolos inúteis, mas *com* produções da forma $A \rightarrow B$, o algoritmo para excluir este tipo de produção pode gerar símbolos inúteis (por quê?). Portanto, caso os algoritmos sejam combinados, a seguinte seqüência de simplificação é recomendada:

- Exclusão de produções vazias;*
- Exclusão de produções da forma $A \rightarrow B$;*
- Exclusão de símbolos inúteis.*

Formas Normais

As formas normais estabelecem restrições rígidas na definição das produções, sem reduzir o poder de geração das Gramáticas Livres do Contexto. São usadas principalmente no desenvolvimento de algoritmos (com destaque para reconhecedores de linguagens) e na prova de teoremas.

As formas normais introduzidas são as seguintes:

- Forma Normal de Chomsky* onde as produções são da forma:

$$A \rightarrow BC \quad \text{ou} \quad A \rightarrow a$$

- Forma Normal de Greibach* onde as produções são da forma:

$$A \rightarrow a\alpha$$

onde α é uma palavra de variáveis.

Para cada caso, é apresentado um algoritmo de conversão de uma Gramática Livre do Contexto qualquer para a correspondente forma normal. As provas de que os algoritmos atingem os objetivos propostos são omitidas.

Definição:

Forma Normal de Chomsky

Uma Gramática Livre do Contexto é dita na *Forma Normal de Chomsky (FNC)* se todas as suas produções são da forma:

$$A \rightarrow BC \quad \text{ou} \quad A \rightarrow a$$

onde A , B e C são variáveis e a é um terminal.

O algoritmo a seguir transforma uma Gramática Livre do Contexto qualquer, cuja linguagem gerada não possua a palavra vazia, em uma gramática na Forma Normal de Chomsky. O algoritmo é dividido em três etapas, como segue:

- Simplificação da Gramática.* Simplifica a gramática excluindo as produções vazias (como a linguagem não possui a palavra vazia, todas as produções da forma $A \rightarrow \epsilon$ podem ser excluídas), produções da forma $A \rightarrow B$ (se o lado direito de alguma produção tiver somente um símbolo, este será terminal) e, opcionalmente, os símbolos inúteis;
- Transformação do lado direito das produções de comprimento maior ou igual a dois.* Garante que o lado direito das produções de comprimento maior ou igual a dois é composto exclusivamente por variáveis. A exclusão de um terminal a pode ser realizada substituindo-o por uma variável intermediária C_a e incluindo a produção $C_a \rightarrow a$;
- Transformação do lado direito das produções de comprimento maior ou igual a três, em produções com exatamente duas variáveis.* Garante que o lado direito das produções de comprimento maior do que um é composto exatamente por duas variáveis. Após a execução da etapa acima, o lado direito das produções da forma $A \rightarrow B_1 B_2 \dots B_n$ ($n \geq 2$) é composto exclusivamente por variáveis. Portanto, para concluir a transformação, é suficiente garantir que o lado direito é composto por exatamente duas variáveis. Isto é possível gerando $B_1 B_2 \dots B_n$ em diversas etapas, usando variáveis intermediárias.

Definição:

Algoritmo para Transformar uma GLC na FNC

Considere uma Gramática Livre do Contexto $G = (V, T, P, S)$, tal que $\epsilon \notin L(G)$. O *Algoritmo para Transformar uma GLC na Forma Normal de Chomsky* é como segue:

a) *Etapas 1: Simplificação da Gramática.* As seguintes simplificações:

- produções vazias;
- produções da forma $A \rightarrow B$;

- símbolos inúteis (opcional);

devem ser realizadas usando os algoritmos de simplificação introduzidos anteriormente, resultando na gramática:

$$G_1 = (V_1, T_1, P_1, S)$$

b) *Etapa 2: Transformação do lado direito das produções de comprimento maior ou igual a dois.* A gramática resultante desta etapa é:

$$G_2 = (V_2, T_1, P_2, S)$$

onde V_2 e P_2 são construídos como segue:

$$V_2 = V_1;$$

$$P_2 = P_1;$$

para toda $A \rightarrow X_1 X_2 \dots X_n \in P_2$ tal que $n \geq 2$

faça se para $r \in \{1, \dots, n\}$, X_r é um símbolo terminal

então (suponha $X_r = a$)

$$V_2 = V_2 \cup \{C_a\};$$

substitui a pela variável C_a em $A \rightarrow X_1 X_2 \dots X_n \in P_2$;

$$P_2 = P_2 \cup \{C_a \rightarrow a\};$$

c) *Etapa 3: Transformação do lado direito das produções de comprimento maior ou igual a três, em produções com exatamente duas variáveis.* A gramática resultante desta etapa é:

$$G_3 = (V_3, T_1, P_3, S)$$

onde V_3 e P_3 são construídos como segue:

$$V_3 = V_2;$$

$$P_3 = P_2;$$

para toda $A \rightarrow B_1 B_2 \dots B_n \in P_3$ tal que $n \geq 3$

faça $P_3 = P_3 - \{A \rightarrow B_1 B_2 \dots B_n\}$;

$$V_3 = V_3 \cup \{D_1, \dots, D_{n-2}\};$$

$$P_3 = P_3 \cup \{A \rightarrow B_1 D_1, D_1 \rightarrow B_2 D_2, \dots,$$

$$D_{n-3} \rightarrow B_{n-2} D_{n-2}, D_{n-2} \rightarrow B_{n-1} B_n\};$$

EXEMPLO:*Transformação de uma GLC na FNC*

Considere a gramática G_2 :

$$G_2 = (\{E\}, \{+, *, [,], x\}, P_2, E), \text{ onde:}$$

$$P_2 = \{E \rightarrow E+E \mid E^*E \mid [E] \mid x\}$$

- a) *Simplificação*. A gramática já está simplificada;
- b) *Variáveis do lado direito*. Excetuando-se a produção $E \rightarrow x$, as demais devem ser substituídas como segue:

$$E \rightarrow EC+E \mid EC^*E \mid C[EC]$$

$$C+ \rightarrow +$$

$$C^* \rightarrow *$$

$$C[\rightarrow [$$

$$C] \rightarrow]$$

- c) *Exatamente duas variáveis do lado direito*. As produções:

$$E \rightarrow EC+E \mid EC^*E \mid C[EC]$$

necessitam ser substituídas como segue:

$$E \rightarrow ED_1 \mid ED_2 \mid C[D_3]$$

$$D_1 \rightarrow C+E$$

$$D_2 \rightarrow C^*E$$

$$D_3 \rightarrow EC]$$

A gramática na Forma Normal de Chomsky resultante é a seguinte:

$$G'_2 = (\{E, C+, C^*, C[, C], D_1, D_2, D_3\}, \{+, *, [,], x\}, P'_2, E), \text{ onde:}$$

$$P'_2 = \{E \rightarrow ED_1 \mid ED_2 \mid C[D_3] \mid x,$$

$$D_1 \rightarrow C+E, D_2 \rightarrow C^*E, D_3 \rightarrow EC],$$

$$C+ \rightarrow +, C^* \rightarrow *, C[\rightarrow [, C] \rightarrow]\}$$

Definição:

Forma Normal de Greibach

Uma Gramática Livre do Contexto é dita na *Forma Normal de Greibach (FNG)* se todas as suas produções são da forma:

$$A \rightarrow a\alpha$$

onde A é uma variável, a é um terminal e α é uma palavra de variáveis.

O algoritmo a seguir transforma uma Gramática Livre do Contexto qualquer, cuja linguagem gerada não possua a palavra vazia, em uma gramática na Forma Normal de Greibach. O algoritmo é dividido nas seguintes etapas:

- a) *Simplificação da Gramática*. Análoga à correspondente etapa do algoritmo referente a Forma Normal de Chomsky;
- b) *Renomeação das variáveis em uma ordem crescente qualquer*. As variáveis da gramática são renomeadas em uma ordem crescente qualquer, como, por exemplo, $A_1, A_2, \dots, A_n$, onde n é o cardinal do conjunto de variáveis. Diferentes critérios de renomeação podem resultar em diferentes gramáticas na Forma Normal de Greibach. Entretanto, todas são equivalentes (geram a mesma linguagem);
- c) *Transformação de produções para a forma $A_r \rightarrow A_s\alpha$, onde $r \leq s$* . As produções são modificadas garantindo que a primeira variável do lado direito é maior ou igual que a do lado esquerdo, considerando a ordenação da etapa anterior. As produções $A_r \rightarrow A_s\alpha$ tais que $r > s$ são modificadas substituindo a variável A_s pelas suas correspondentes produções ($A_s \rightarrow \beta_1\gamma_1 \dots \beta_m\gamma_m$), resultando em $A_r \rightarrow \beta_1\alpha_1 \dots \beta_m\alpha_m$ e assim sucessivamente. Entretanto, como o conjunto de variáveis é finito, existe um limite para as produções crescentes, que pode ser a geração de um terminal ($A_r \rightarrow a\alpha$) ou de uma recursão ($A_r \rightarrow A_r\alpha$);
- d) *Exclusão das recursões da forma $A_r \rightarrow A_r\alpha$* . As recursões (à esquerda) podem existir originalmente na gramática, ou serem geradas pela etapa anterior. A eliminação da recursão à esquerda pode ser realizada introduzindo variáveis auxiliares e incluindo recursão à direita ($B_r \rightarrow \alpha B_r$);
- e) *Um terminal no início do lado direito de cada produção*. Após a execução da etapa anterior, todas as produções da forma $A_r \rightarrow A_s\alpha$ são tais que $r \leq s$. Conseqüentemente, as produções da maior variável A_n só podem iniciar por um terminal no lado direito. Assim, se em $A_{n-1} \rightarrow A_n\alpha$ for substituído A_n pelas suas correspondentes produções (exemplo: $A_n \rightarrow a\beta$), o lado direito das produções de A_{n-1} também iniciarão por um terminal (exemplo: $A_{n-1} \rightarrow a\beta\alpha$). A repetição do algoritmo para $A_{n-2}, \dots, A_1$ resultará em produções exclusivamente da forma $A_r \rightarrow a\alpha$;
- f) *Produções na forma $A \rightarrow a\alpha$ onde α é composta por variáveis*. É análoga a correspondente etapa do algoritmo relativo à Forma Normal de Chomsky.

Definição:

Algoritmo para Transformar uma GLC na FNG

Considere uma Gramática Livre do Contexto $G = (V, T, P, S)$, tal que $\varepsilon \notin L(G)$. O *Algoritmo para Transformar uma GLC na Forma Normal de Greibach* é como segue:

a) *Etapa 1: Simplificação da Gramática*. As seguintes simplificações:

- produções vazias;
- produções da forma $A \rightarrow B$;
- símbolos inúteis (opcional);

devem ser realizadas usando os algoritmos de simplificação introduzidos anteriormente, resultando na gramática:

$$G_1 = (V_1, T_1, P_1, S)$$

b) *Etapa 2: Renomeação das variáveis em uma ordem crescente qualquer*. A gramática resultante desta etapa é:

$$G_2 = (V_2, T_1, P_2, S)$$

onde V_2 e P_2 são construídos como segue (suponha que o cardinal de V_1 é n):

$V_2 = \{A_1, A_2, \dots, A_n\}$ é V_1 onde as variáveis são renomeadas;

P_2 é P_1 renomeando as variáveis nas produções;

c) *Etapas 3 e 4: Transformação de produções para a forma $A_r \rightarrow A_s \alpha$, onde $r \leq s$ e exclusão das recursões da forma $A_r \rightarrow A_r \alpha$* . A gramática resultante destas duas etapas realizadas em conjunto é:

$$G_3 = (V_3, T_1, P_3, S)$$

onde V_3 , P_3 são construídos como segue (suponha que o cardinal de V_2 é n):

$P_3 = P_2$

para r variando de 1 até n

faça

para s variando de 1 até $r-1$ *Etapa 3*

faça para toda $A_r \rightarrow A_s \alpha \in P_3$

faça excluir $A_r \rightarrow A_s \alpha$ de P_3 ;

para toda $A_s \rightarrow \beta \in P_3$

faça $P_3 = P_3 \cup \{A_r \rightarrow \beta \alpha\}$

para toda $A_r \rightarrow A_r \alpha \in P_3$ *Etapa 4*

faça excluir $A_r \rightarrow A_r \alpha$ de P_3 ;

$V_3 = V_3 \cup \{B_r\}$;

$P_3 = P_3 \cup \{B_r \rightarrow \alpha\} \cup \{B_r \rightarrow \alpha B_r\}$;

para toda $A_r \rightarrow \phi \in P_3$ tal que ϕ não inicia por A_r e

alguma $A_r \rightarrow A_r \alpha$ foi excluída

faça $P_3 = P_3 \cup \{A_r \rightarrow \phi B_r\}$;

d) *Etapa 5: Um terminal no início do lado direito de cada produção. A gramática resultante desta etapa é:*

$G_4 = (V_3, T_1, P_4, S)$

onde P_4 é construído como segue:

$P_4 = P_3$;

para r variando de $n-1$ até 1 e toda $A_r \rightarrow A_s \alpha \in P_4$

faça excluir $A_r \rightarrow A_s \alpha$ de P_4 ;

para toda $A_s \rightarrow \beta$ de P_4

faça $P_4 = P_4 \cup \{A_r \rightarrow \beta \alpha\}$;

Também é necessário garantir que as produções relativas às variáveis auxiliares B_r iniciam por um terminal do lado direito, como segue:

para toda $B_r \rightarrow A_s \beta$

faça excluir $B_r \rightarrow A_s \beta$ de P_4 ;

para toda $A_s \rightarrow a \alpha$

faça $P_4 = P_4 \cup \{B_r \rightarrow a \alpha \beta\}$;

e) *Etapa 6: Produções na forma $A \rightarrow a \alpha$ onde α é composta por variáveis. É análoga à correspondente etapa do algoritmo relativo à Forma Normal de Chomsky.*

EXEMPLO:**Transformação de uma GLC na FNG**

Considere a seguinte Gramática Livre do Contexto:

$$G = (\{S, A\}, \{a, b\}, P, S), \text{ onde:}$$

$$P = \{S \rightarrow AA \mid a, A \rightarrow SS \mid b\}$$

A transformação na correspondente Forma Normal de Greibach é como segue:

- a) **Simplificação.** A gramática já está simplificada;
- b) **Renomeação das variáveis em ordem crescente.** As variáveis S e A são renomeadas para A_1 e A_2 , respectivamente. As produções da gramática ficam como segue:

$$A_1 \rightarrow A_2 A_2 \mid a$$

$$A_2 \rightarrow A_1 A_1 \mid b$$

- c) $A_r \rightarrow A_{s\alpha}$, com $r \leq s$ e recursões $A_r \rightarrow A_{r\alpha}$. A produção $A_2 \rightarrow A_1 A_1$ necessita ser modificada. As produções da gramática ficam como segue:

$$A_1 \rightarrow A_2 A_2 \mid a$$

$$A_2 \rightarrow A_2 A_2 A_1 \mid a A_1 \mid b$$

A produção $A_2 \rightarrow A_2 A_2 A_1$ contém um recursão. Portanto, é necessário introduzir uma variável auxiliar B , como segue:

$$A_1 \rightarrow A_2 A_2 \mid a$$

$$A_2 \rightarrow a A_1 \mid b \mid a A_1 B \mid b B$$

$$B \rightarrow A_2 A_1 \mid A_2 A_1 B$$

- d) **Terminal no início de cada lado direito das produções.** O lado direito das produções da maior variável A_2 iniciam por um terminal. Substituindo A_2 no lado direito de $A_1 \rightarrow A_2 A_2$ pelas suas correspondentes derivações, as produções de A_1 também iniciarão por um terminal:

$$A_1 \rightarrow a A_1 A_2 \mid b A_2 \mid a A_1 B A_2 \mid b B A_2 \mid a$$

$$A_2 \rightarrow a A_1 \mid b \mid a A_1 B \mid b B$$

$$B \rightarrow A_2 A_1 \mid A_2 A_1 B$$

As produções referentes à variável B também são modificadas:

$$B \rightarrow aA_1A_1|bA_1|aA_1BA_1|bBA_1|aA_1A_1B|bA_1B|aA_1BA_1B|bBA_1B$$

e) $A \rightarrow a\alpha$, com α composta exclusivamente por variáveis. Nenhum procedimento é necessário, pois as produções já se encontram nesta forma.

A gramática na Forma Normal de Greibach resultante é a seguinte:

$$G = (\{A_1, A_2, B\}, \{a, b\}, P, A_1), \text{ onde:}$$

$$P = \{A_1 \rightarrow aA_1A_2|bA_2|aA_1BA_2|bBA_2|a,$$

$$A_2 \rightarrow aA_1|b|aA_1B|bB,$$

$$B \rightarrow aA_1A_1|bA_1|aA_1BA_1|bBA_1|aA_1A_1B|bA_1B|aA_1BA_1B|bBA_1B\}$$

Recursão à Esquerda

Em diversas situações, como no desenvolvimento de algoritmos reconhecedores, é desejável que a gramática que representa a linguagem não seja recursiva à esquerda.

Entende-se por recursão à esquerda a ocorrência da seguinte situação:

$$A \Rightarrow^+ A\alpha$$

Ou seja, uma variável deriva ela mesma, de forma direta ou indireta, como o símbolo mais à esquerda de uma subpalavra gerada.

A transformação de uma gramática qualquer em uma sem recursão à esquerda pode ser realizada executando as quatro primeiras etapas do algoritmo referente à Forma Normal de Greibach, que são as seguintes:

- a) *Etapa 1: Simplificação da Gramática;*
- b) *Etapa 2: Renomeação das variáveis em uma ordem crescente qualquer;*
- c) *Etapa 3: Qualquer produção é da forma $A_r \rightarrow A_s\alpha$, onde $r < s$;*
- d) *Etapa 4: Exclusão das recursões da forma $A_r \rightarrow A_r\alpha$.*

Autômato com Pilha

Analogamente às Linguagens Regulares, a Classe das Linguagens Livres do Contexto pode ser associada a um formalismo do tipo autômato, denominado Autômato com Pilha.

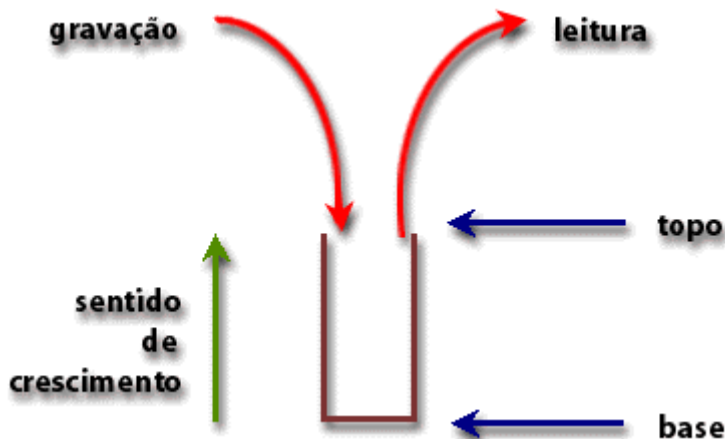
O Autômato com Pilha é análogo ao Autômato Finito, incluindo uma pilha como memória auxiliar e a facilidade de não-determinismo. A pilha é independente da fita de entrada e não possui limite máximo de tamanho ("infinita"). Estruturalmente, sua principal característica é que o último símbolo gravado é o primeiro a ser lido, como

ilustrado na Figura. A **base** de uma pilha é fixa e define o seu início. O **topo** é variável e define a posição do último símbolo gravado.

A facilidade de não-determinismo é importante e necessária, pois aumenta o poder computacional dos Autômatos com Pilha, permitindo reconhecer exatamente a Classe das Linguagens Livres do Contexto. Por exemplo, o reconhecimento da linguagem:

$$\{ww^r | w \text{ é palavra sobre } \{a, b\}\}$$

só é possível por um Autômato com Pilha Não-Determinístico.



Embora o poder computacional do Autômato com Pilha seja muito superior ao do Autômato Finito, ainda é restrito, não sendo possível reconhecer linguagens simples, como, por exemplo:

$$\{ww | w \text{ é palavra sobre } \{a, b\}\} \quad \text{ou} \quad \{a^n b^n c^n | n \geq 0\}$$

Um resultado interessante mostrado adiante é que qualquer Linguagem Livre do Contexto pode ser reconhecida por um Autômato com Pilha com somente um estado (ou três estados, dependendo da definição). Isto significa que a estrutura de pilha é suficiente como única memória, não sendo necessário usar os estados para "memorizar" informações passadas. Ou seja, a estrutura de estados no Autômato com Pilha poderia ser excluída sem reduzir o poder computacional.

O modelo Autômato com Pilha possui duas definições universalmente aceitas que diferem no critério de parada do autômato, como segue:

- o valor inicial da pilha é vazio e o autômato pára aceitando ao atingir um estado final;
- a pilha contém, inicialmente, um símbolo especial denominado símbolo inicial da pilha. Não existem estados finais e o autômato pára aceitando quando a pilha estiver vazia.

As duas definições são equivalentes (possuem o mesmo poder computacional), sendo fácil modificar um Autômato com Pilha para satisfazer a outra definição. Nesta publicação, é adotada a com estados finais.

Um Autômato com Pilha ou Autômato com Pilha Não-Determinístico é composto, basicamente, por quatro partes, como segue:

- a) *Fita*. Análoga à do Autômato Finito;
- b) *Pilha*. Memória auxiliar que pode ser usada livremente para leitura e gravação;
- c) *Unidade de Controle*. Reflete o estado corrente da máquina. Possui uma cabeça de fita e uma cabeça de pilha;
- d) *Programa ou Função de Transição*. Comanda a leitura da fita, leitura e gravação da pilha e define o estado da máquina.

A pilha é dividida em células, armazenando, cada uma, um símbolo do alfabeto auxiliar (pode ser igual ao alfabeto de entrada). Em uma estrutura do tipo pilha, a leitura ou gravação é sempre na mesma extremidade, denominada topo. Não possui tamanho fixo e nem máximo, sendo seu tamanho corrente igual ao tamanho da palavra armazenada. Seu valor inicial é vazio.

A unidade de controle possui um número finito e predefinido de estados. Possui uma cabeça de fita e uma cabeça de pilha, como segue:

- a) *Cabeça da Fita*. Unidade de leitura a qual acessa uma célula da fita de cada vez e movimenta-se exclusivamente para a direita. É possível testar se a entrada foi completamente lida;
- b) *Cabeça da Pilha*. Unidade de leitura e gravação a qual move para a esquerda (ou "para cima") ao gravar e para a direita (ou "para baixo") ao ler um símbolo. Acessa um símbolo de cada vez, estando sempre posicionada no topo. A leitura exclui o símbolo lido. É possível testar se a pilha está vazia. Em uma mesma operação de gravação, é possível armazenar uma palavra composta por mais de um símbolo. Neste caso, o símbolo do topo é o mais à esquerda da palavra gravada.

O programa é uma função parcial que, dependendo do estado corrente, símbolo lido da fita e símbolo lido da pilha, determina o novo estado e a palavra a ser gravada (na pilha). Possui a facilidade de movimento vazio (análoga à do Autômato Finito), permitindo mudar de estado sem ler da fita.

Definição:

Autômato com Pilha

Um *Autômato com Pilha Não-Determinístico (APN)* ou simplesmente *Autômato com Pilha (AP)* M é uma 6-upla:

$$M = (\Sigma, Q, \delta, q_0, F, V)$$

onde:

Σ *alfabeto de símbolos de entrada*;

Q *conjunto de estados* possíveis do autômato o qual é finito;

δ *função programa ou de função de transição*:

$$\delta: Q \times (\Sigma \cup \{\epsilon, ?\}) \times (V \cup \{\epsilon, ?\}) \rightarrow 2^{Q \times V^*}$$

a qual é uma função parcial;

q_0 *estado inicial* do autômato tal que q_0 é elemento de Q ;

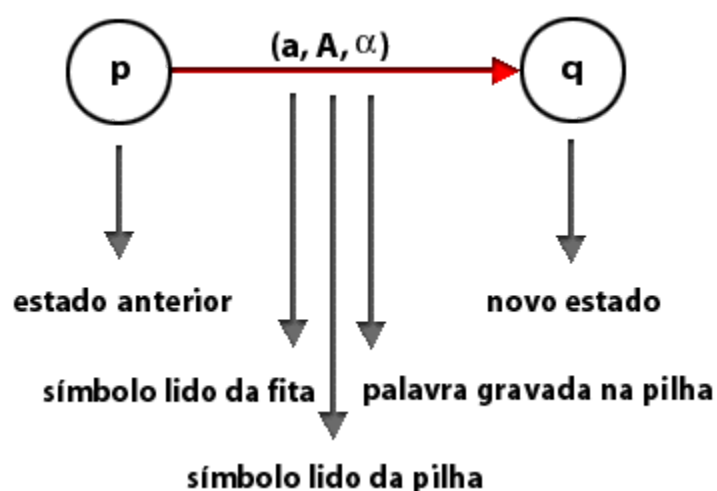
F *conjunto de estados finais* tal que F está contido em Q ;

V *alfabeto auxiliar* ou *alfabeto da pilha*.

As seguintes características da função programa devem ser consideradas:

- a função pode não ser total, ou seja, indefinida para alguns argumentos do conjunto de partida; a omissão do parâmetro de leitura, representada por "?", indica o teste de pilha vazia ou toda palavra de entrada lida;
- o símbolo ϵ na leitura indica a facilidade de movimento vazio da fita ou da pilha (o autômato não lê nem move a cabeça). Note-se que, para o movimento ser considerado não-determinístico, é suficiente que o movimento seja vazio na fita;
- o símbolo ϵ na gravação indica que nenhuma gravação é realizada na pilha (e não move a cabeça).

Por exemplo, $\delta(p, ?, \epsilon) = \{(q, \epsilon)\}$ indica que no estado p se a entrada foi completamente lida, não lê da pilha, assume o estado q e não grava na pilha. A função programa pode ser representada como um grafo direto, como ilustrado na Figura.



O processamento de um Autômato com Pilha, para uma palavra de entrada w , consiste na sucessiva aplicação da função programa para cada símbolo de w (da esquerda para a direita) até ocorrer uma condição de parada. Entretanto, é possível que um Autômato com Pilha nunca atinja uma condição de parada. Neste caso, fica processando indefinidamente (ciclo ou *loop* infinito). Um exemplo simples de ciclo infinito é um

programa que empilha e desempilha um mesmo símbolo indefinidamente, sem ler da fita.

Um Autômato com Pilha pode parar aceitando ou rejeitando a entrada ou ficar em *loop* infinito, como segue:

- Um dos caminhos alternativos assume um estado final: o autômato pára e a palavra é aceita;
- Todos os caminhos alternativos rejeitam a entrada: o autômato pára e a palavra é rejeitada;
- Pelo menos um caminho alternativo está em *loop* infinito e os demais rejeitam (ou também estão em *loop* infinito): o autômato está em *loop* infinito.

Para definir formalmente o comportamento de um Autômato com Pilha, é necessário estender a definição da função programa usando como argumento um estado e uma palavra. Esta extensão é sugerida como [exercício](#).

As seguintes notações são adotadas (suponha que M é um Autômato com Pilha):

- $ACEITA(M)$ ou $L(M)$: conjunto de todas as palavras de Σ^* aceitas por M ;
- $REJEITA(M)$: conjunto de todas as palavras de Σ^* rejeitadas por M ;
- $LOOP(M)$: conjunto de todas as palavras de Σ^* para as quais M fica processando indefinidamente.

As seguintes afirmações são verdadeiras:

- $ACEITA(M) \cap REJEITA(M) \cap LOOP(M) = \emptyset$
- $ACEITA(M) \cup REJEITA(M) \cup LOOP(M) = \Sigma^*$
- o complemento de:

$ACEITA(M)$ é $REJEITA(M) \cup LOOP(M)$

$REJEITA(M)$ é $ACEITA(M) \cup LOOP(M)$

$LOOP(M)$ é $ACEITA(M) \cup REJEITA(M)$

EXEMPLO:

Autômato com Pilha

Considere a seguinte linguagem:

$$L_1 = \{a^n b^n \mid n \geq 0\}$$

O Autômato com Pilha $M_1 = (\{a, b\}, \{q_0, q_1, q_f\}, \delta_1, q_0, \{q_f\}, \{B\})$, onde δ_1 é como abaixo, é tal que $ACEITA(M_1) = L_1$ (o conjunto $LOOP(M_1)$ é vazio?):

$$\delta_1(q_0, a, \varepsilon) = \{(q_0, B)\}$$

$$\delta_1(q_0, b, B) = \{(q_1, \varepsilon)\}$$

$$\delta_1(q_0, ?, ?) = \{(q_f, \varepsilon)\}$$

$$\delta_1(q_1, b, B) = \{(q_1, \varepsilon)\}$$

$$\delta_1(q_1, ?, ?) = \{(q_f, \varepsilon)\}$$

Note-se que o autômato é determinístico. No estado q_0 , para cada símbolo a lido da fita é armazenado um símbolo B na pilha. No estado q_1 , é realizado um batimento, verificando se para cada símbolo b da fita, existe um correspondente B na pilha. O algoritmo somente aceita se ao terminar de ler toda a palavra de entrada a pilha estiver vazia.

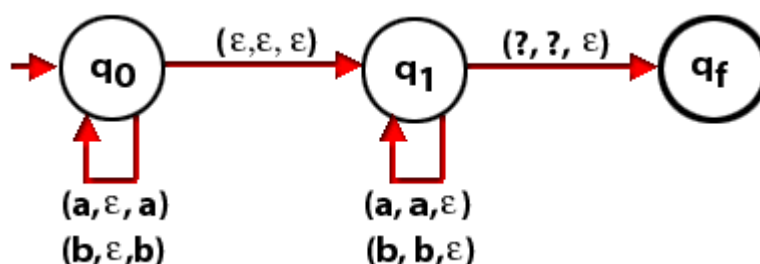
EXEMPLO:

Autômato com Pilha

Considere a seguinte linguagem:

$$L_3 = \{ww^r \mid w \text{ pertence a } \{a, b\}^*\}$$

O autômato M_3 ilustrado na Figura é tal que $ACEITA(M_3) = L_3$ (o conjunto $LOOP(M_3)$ é vazio?). Note-se que M_3 é não-determinístico devido ao movimento vazio de q_0 para q_1 . Adicionalmente, o alfabeto auxiliar é igual ao de entrada. Em q_0 , é empilhado o reverso do prefixo. A cada símbolo empilhado, ocorre um movimento não-determinista para q_1 o qual verifica se o sufixo da palavra é igual ao conteúdo da pilha.



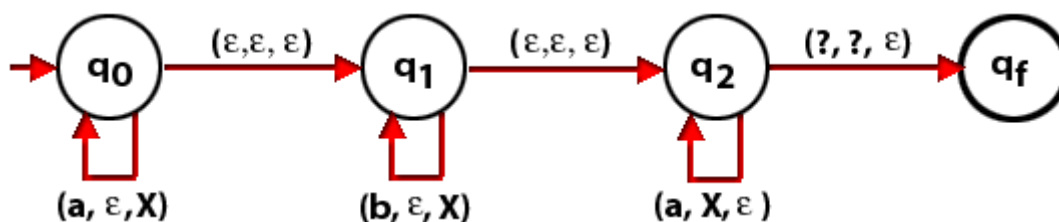
EXEMPLO:

Autômato com Pilha

Considere a seguinte linguagem:

$$L_4 = \{a^n b^m a^{n+m} \mid n \geq 0, m \geq 0\}$$

O autômato não-determinístico M_4 ilustrado na Figura é tal que $ACEITA(M_4) = L_4$ (o conjunto $LOOP(M_4)$ é vazio?). M_4 empilha um símbolo auxiliar X para cada a ou b em q_0 ou q_1 , respectivamente. Após, em q_2 , verifica se o número de a no sufixo é igual ao de X empilhado.



Autômato com Pilha e Linguagens Livres do Contexto

A classe das linguagens reconhecidas pelos Autômatos com Pilha é igual à Classe das Linguagens Livres do Contexto (ou seja, é igual à classe das linguagens geradas pelas Gramáticas Livres do Contexto). A demonstração apresentada a seguir é dividida em dois teoremas. O primeiro teorema apresenta a construção de um AP a partir de uma GLC qualquer, permitindo estabelecer as seguintes conclusões:

- a construção de um reconhecedor para uma Linguagem Livre do Contexto a partir de sua gramática é simples e imediata;
- qualquer Linguagem Livre do Contexto pode ser reconhecida por um Autômato com Pilha com somente um estado de controle lógico, o que significa que a facilidade de memorização de informações através de estados (como nos Autômatos Finitos) não aumenta o poder computacional dos AP.

Teorema:

Gramática Livre do Contexto → Autômato com pilha

Se L é uma Linguagem Livre do Contexto, então existe M , Autômato com pilha tal que $ACEITA(M) = L$.

Prova: Suponha que a palavra vazia não pertence à L . A demonstração consiste na construção de um Autômato com Pilha a partir da gramática transformada na Forma Normal de Greibach (produções da forma $A \rightarrow a\alpha$, α palavra de variáveis). O Autômato com Pilha gerado simula a derivação mais à esquerda, como segue (suponha a produção $A \rightarrow a\alpha$):

- lê o símbolo a da fita;
- lê o símbolo A da pilha;
- empilha a palavra α .

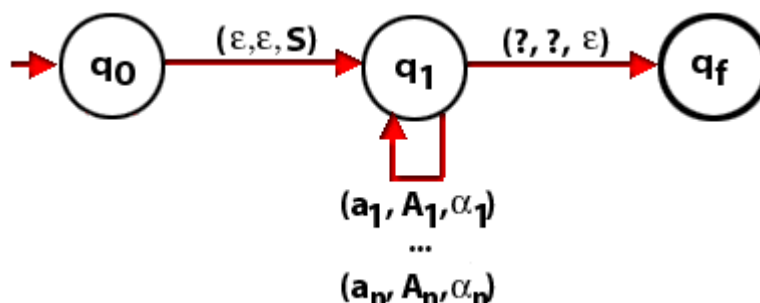
A simulação acima é realizada para cada produção, usando um único estado de controle. A construção do AP M a partir da gramática $G = (V, T, P, S)$ é como segue (veja a Figura):

- seja $G' = (V', T', P', S)$, a transformação de G na Forma Normal de Greibach;
- seja $M = (T', \{q_0, q_1, q_f\}, \delta, q_0, \{q_f\}, V')$, onde:

$$\delta(q_0, \epsilon, \epsilon) = \{(q_1, S)\}$$

$$\delta(q_1, a, A) = \{(q_1, \alpha) \mid A \rightarrow a\alpha \in P'\}$$

$$\delta(q_1, ?, ?) = \{(q_f, \varepsilon)\}$$



A demonstraco de que $ACEITA(M) = GERA(G')$  realizada por induco no nmero de movimentos de M (o derivao de G') e  sugerida como **exerccio**. Como o autmato pode ser modificado para tratar a palavra vazia?

EXEMPLO:

Construo de um AP a partir de uma Gramtica na FNG

A linguagem:

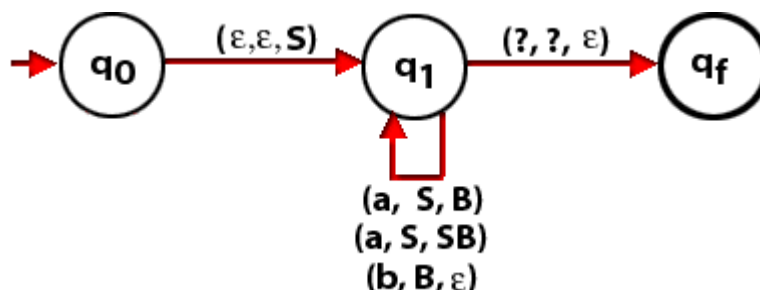
$$L_5 = \{a^n b^n \mid n \geq 1\}$$

representada pela seguinte gramtica na Forma Normal de Greibach:

$$G_5 = (\{S, B\}, \{a, b\}, P_5, S), \text{ onde:}$$

$$P_5 = \{S \rightarrow aB \mid aSB, B \rightarrow b\}$$

 reconhecida pelo Autmato com Pilha $M_5 = (\{a, b\}, \{q_0, q, q_f\}, \delta_5, q_0, \{q_f\}, \{S, B\})$ ilustrado na Figura, construdo a partir de G_5 .



Corolrio:

Autmato com Pilha × Nmero de Estados

Se L  uma Linguagem Livre do Contexto, ento:

- a) Existe M , Autômato com Pilha com controle de aceitação por estados finais, com somente três estados tal que $ACEITA(M) = L$;
- b) Existe M , Autômato com Pilha com controle de aceitação por pilha vazia, com somente um estado tal que $ACEITA(M) = L$.

O detalhamento da demonstração do item b) é simples e é sugerida como [exercício](#).

Corolário:

Corolário Existência de um Autômato com Pilha que Sempre Pára

Se L é uma Linguagem Livre do Contexto, então existe M , Autômato com Pilha tal que:

$$ACEITA(M) = L$$

$$REJEITA(M) = \Sigma^* - L$$

$$LOOP(M) = \emptyset$$

Ou seja, para qualquer Linguagem Livre do Contexto existe um Autômato com Pilha que sempre pára para qualquer entrada ([por quê?](#)).

Teorema:

Teorema Autômato com Pilha $\rightarrow$ Gramática Livre do Contexto

Se L é aceita por um Autômato com Pilha, então L é Linguagem Livre do Contexto.

Observação:

Estados Poder Computacional dos Autômatos com Pilha

A combinação dos resultados obtidos com os corolários e teoremas comprovam que o uso dos estados como "memória" não aumenta o poder de reconhecimento do Autômato com Pilha.

Número de Pilhas e o Poder Computacional

O Autômato com Pilha é um modelo freqüentemente referenciado em estudos aplicados e formais pois, além da estrutura de pilha ser adequada para implementação em computadores, poucas modificações sobre a definição básica determinam significativas alterações no poder computacional. Especificamente, os principais estudos de linguagens e computabilidade podem ser desenvolvidos usando exclusivamente o AP, variando o número de pilhas com ou sem a facilidade de não-determinismo. Os seguintes resultados podem ser enunciados:

- a) *Autômato com Pilha, sem usar a estrutura de pilha*. Se a estrutura de pilha não for usada, a única forma de memorizar informações passadas é usando os estados. Assim, claramente o AP sem usar a pilha é muito semelhante ao Autômato Finito. De fato, é fácil mostrar que a Classe das Linguagens aceitas

por AP sem pilha, com ou sem a facilidade de não-determinismo, é igual a Classe das Linguagens Regulares (o que se sugere como [exercício](#));

b) *Autômato com Pilha Determinístico*. O *Autômato com uma Pilha Determinístico (APD)* aceita um subconjunto próprio das Linguagens Livres do Contexto, denominadas *Linguagens Livres do Contexto Determinísticas (LLCD)*. A Classe das LLCD inclui muitas das linguagens aplicadas em informática, com destaque para as de programação. Uma das razões é que a implementação de um APD em um computador é simples e eficiente, facilitando, assim, o desenvolvimento de tradutores de linguagens. Algumas propriedades das LLCD são as seguintes:

b.1) É possível definir um tipo de gramática que gera exatamente a Classe das LLCD. Entretanto, não são restrições simples sobre a definição geral de gramática;

b.2) A Classe das LLCD é fechada para a operação de complemento;

b.3) A Classe das LLCD não é fechada para as operações de união, intersecção e concatenação;

c) *Autômato com Pilha Não-Determinístico*. Conforme já verificado, a classe das linguagens reconhecida pelo AP é exatamente a Livre do Contexto;

d) *Autômato com Duas Pilhas*. O *Autômato com Duas Pilhas (A2P)* é equivalente, em termos de poder computacional, à Máquina de Turing (a ser introduzida adiante), considerado o dispositivo mais geral de computação. Assim, se existe um algoritmo para resolver um problema (por exemplo, reconhecer uma determinada linguagem), então este algoritmo pode ser expresso como um A2P. A facilidade de não-determinismo não aumenta o poder computacional do Autômato com Duas Pilhas;

e) *Autômato com Mais de Duas Pilhas*. O poder computacional de um *Autômato com Múltiplas Pilhas (AnP, $n > 2$)* é equivalente ao do A2P. Ou seja, se um problema é solucionado por um AnP, então o mesmo problema pode ser solucionado por um A2P.

Propriedades das Linguagens Livres do Contexto

Embora as Linguagens Livres do Contexto sejam mais gerais que as Regulares, ainda são relativamente restritas. Ou seja, é fácil definir linguagens que não são Livres do Contexto, como, por exemplo:

$\{ww \mid w \text{ pertence a } \{a, b\}^*\}$

$\{a^n b^n c^n \mid n \geq 0\}$

Assim, algumas questões sobre as Linguagens Livres do Contexto necessitam ser analisadas:

- a) Como determinar se uma linguagem é Livre do Contexto?
- b) A Classe das Linguagens Livres do Contexto é fechada para operações como união, intersecção, concatenação e complemento?
- c) Como verificar se uma Linguagem Livre do Contexto é infinita ou finita (ou até mesmo vazia)?

Investigação se é Linguagem Livre do Contexto

Para mostrar que uma determinada linguagem é Livre do Contexto, é suficiente expressá-la usando os formalismos Gramática Livre do Contexto ou Autômato com Pilha. Entretanto, a demonstração que não é Livre do Contexto necessita ser realizada caso a caso.

Analogamente às Linguagens Regulares, as Linguagens Livres do Contexto possuem um Lema de Bombeamento o qual é útil no estudo das propriedades.

Lema:

Bombeamento para as Linguagens Livres do Contexto

Se L é uma Linguagem Livre do Contexto, então:

existe uma constante n tal que,

para qualquer palavra w de L onde $|w| \geq n$,

w pode ser definida como $w = uvxyz$ onde $|xvy| \leq n$, $|xy| \geq 1$ e,

para todo $i \geq 0$, $ux^i v y^i z$ é palavra de L .

Note-se que para $w = uvxyz$, tem-se que ou x ou y pode ser a palavra vazia (mas não ambas). Uma forma de demonstrar o lema é usando gramáticas na Forma Normal de Chomsky. Neste caso, se a gramática possui s variáveis, pode-se assumir que $n = 2^s$. O lema não será demonstrado.

EXEMPLO:

Linguagem Não-Livre do Contexto

A seguinte linguagem não é Livre do Contexto:

$$L = \{a^n b^n c^n \mid n \geq 0\}$$

A prova que segue usa o bombeamento e é por absurdo. Suponha que L é Livre do Contexto. Então:

existe uma gramática na FNC G com s variáveis que gera L ;

sejam $r = 2^s$ e $w = a^r b^r c^r$

Pelo bombeamento, w pode ser definida como $w = uvvyz$ onde:

$$|xvy| \leq r, |xy| \geq 1 \text{ e,}$$

para todo $i \geq 0$, $ux^i v y^i z$ é palavra de L ,

o que é um absurdo, pois, como $|xvy| \leq r$, não é possível supor que xy possui símbolos a e c , pois quaisquer ocorrências de a e c estão separadas por, pelo menos, r ocorrências de b . Consequentemente, xy jamais possuirá ocorrências de a , b e c simultaneamente. Assim, tem-se que:

- se xy possui somente símbolos a , é fácil verificar que a aplicação do bombeamento pode desbalancear as ocorrências de a , b e c ;
- analogamente para os seguintes casos: somente símbolos c , somente símbolos a e b e somente símbolos b e c .

Operações sobre Linguagens Livres do Contexto

O teorema a seguir mostra que a Classe das Linguagens Livres do Contexto é fechada para as operações de união e concatenação.

Relativamente à Classe das Linguagens Livres do Contexto, tem-se que:

- é fechada para as operações de união e concatenação;

não é fechada para as operações de intersecção e complemento.

Teorema:

Teorema Operações Fechadas sobre Linguagens Livres do Contexto

A Classe das Linguagens Livres do Contexto é fechada para as seguintes operações:

- União;
- Concatenação.

Prova:

União. A demonstração que segue é baseada no formalismo Autômato com Pilha e usa a facilidade de não-determinismo. Sugere-se como [exercício](#) a demonstração usando o formalismo Gramática Livre do Contexto.

Suponha L_1 e L_2 , LLC. Então, existem Autômatos com Pilha:

$$M_1 = (\Sigma_1, Q_1, \delta_1, q_{01}, F_1, V_1)$$

$$M_2 = (\Sigma_2, Q_2, \delta_2, q_{02}, F_2, V_2)$$

tais que $ACEITA(M_1) = L_1$ e $ACEITA(M_2) = L_2$. Seja M_3 construído como segue o qual é ilustrado na Figura abaixo (suponha que os conjuntos Q_1 e Q_2 , bem como V_1 e V_2 são disjuntos):

$$M_3 = (\Sigma_1 \cup \Sigma_2, Q_1 \cup Q_2 \cup \{q_0\}, \delta_3, q_0, F_1 \cup F_2, V_1 \cup V_2)$$

Claramente, M_3 reconhece $L_1 \cup L_2$.

Concatenação. A demonstração a seguir usa o formalismo Gramática Livre do Contexto. Sugere-se como **exercício** a demonstração usando o formalismo Autômato com Pilha.

Suponha L_1 e L_2 , LLC. Então, existem Gramáticas Livres do Contexto:

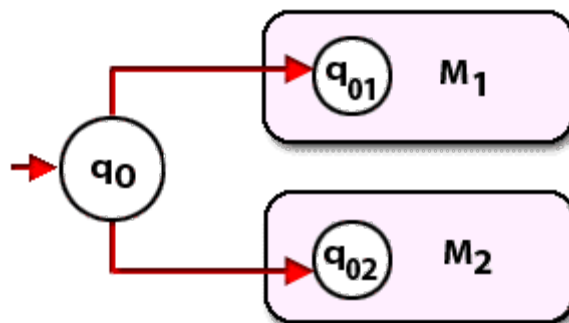
$$G_1 = (V_1, T_1, P_1, S_1)$$

$$G_2 = (V_2, T_2, P_2, S_2)$$

tais que $GERA(G_1) = L_1$ e $GERA(G_2) = L_2$. Seja G_3 construída como segue (suponha que V_1 e V_2 são conjuntos disjuntos):

$$G_3 = (V_1 \cup V_2 \cup \{S\}, T_1 \cup T_2, P_1 \cup P_2 \cup \{S \rightarrow S_1 S_2\}, S)$$

a qual é Livre do Contexto (por quê?). Como a única produção de S é $S \rightarrow S_1 S_2$, claramente qualquer palavra gerada por G_3 terá, como prefixo, uma palavra de L_1 e, como sufixo, uma palavra de L_2 . Logo, $L_1 L_2$ é LLC.



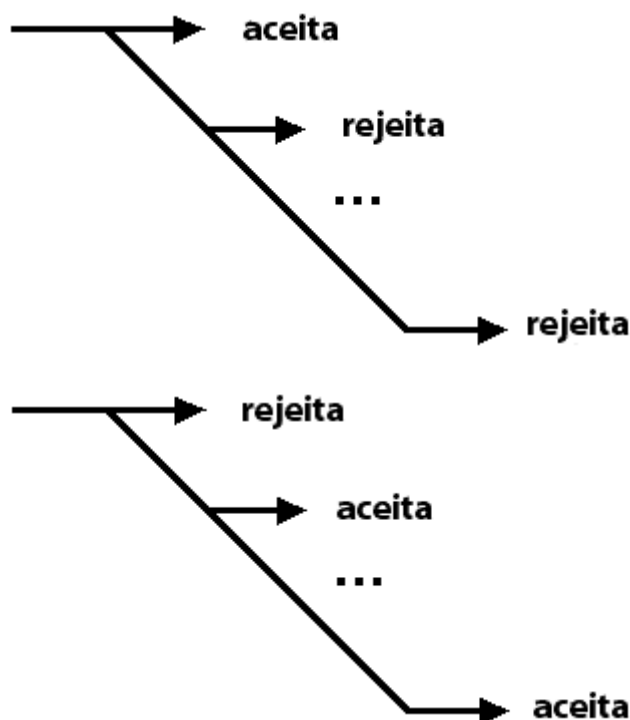
O teorema a seguir mostra que a Classe das Linguagens Livres do Contexto não é fechada para as operações de intersecção e complemento. Note-se que, o fato de não ser fechada para a operação de complemento é, aparentemente, uma contradição, pois:

- se L é LLC, então existe M , AP tal que $ACEITA(M) = L$ e $REJEITA(M) = L'$. Ou seja, M é capaz de rejeitar qualquer palavra que não pertença à L ;
- o teorema a seguir mostra que, se L é LLC, não se pode afirmar que L' também é LLC.

Assim, é perfeitamente possível **rejeitar** o complemento de uma LLC, embora nem sempre seja possível **aceitar** o complemento. Uma explicação intuitiva usando o formalismo AP é a seguinte onde, sem perda de generalidade, suponha que a função programa do AP em questão é total (por quê pode-se afirmar isto?):

- a condição para um AP aceitar uma entrada é que pelo menos um dos caminhos alternativos reconheça a palavra (mesmo que os demais rejeitem). Esta condição pode ser representada pelo diagrama ilustrado na Figura de cima;
- se os estados de aceita e rejeita forem invertidos, na tentativa de aceitar o complemento, a situação resultante continua sendo de aceitação (e não de rejeição), pois permanecerá pelo menos um caminho alternativo reconhecendo a entrada. O diagrama ilustrado na Figura de baixo representa a "negação" da condição acima.

Portanto, considerando a facilidade de não-determinismo, o fato de existir um AP capaz de rejeitar o complemento de uma linguagem não implica que existe um AP capaz de aceitar o mesmo complemento.



Teorema:

Teorema Operações Não-Fechadas sobre Linguagens Livres do Contexto

A Classe das Linguagens Livres do Contexto não é fechada para as seguintes operações:

- Intersecção;
- Complemento.

Prova:

Intersecção. Para mostrar que a Classe das LLC não é fechada para a operação de intersecção, é suficiente mostrar um contra-exemplo. Assim, sejam:

$$L_1 = \{a^n b^n c^m \mid n \geq 0, m \geq 0\}$$

$$L_2 = \{a^m b^n c^n \mid n \geq 0 \text{ e } m \geq 0\}$$

É fácil mostrar que L_1 e L_2 são LLC. Entretanto, L_3 como abaixo, resultante da intersecção de L_1 com L_2 , não é LLC:

$$L_3 = \{a^n b^n c^n \mid n \geq 0\}$$

Complemento. Como a operação de intersecção pode ser representada em termos da união e do complemento, e considerando que a Classe das LLC não é fechada para a operação de intersecção, então não pode-se afirmar que o complemento de uma LLC é LLC.

Investigação se uma Linguagem Livre do Contexto é

Vazia, Finita ou Infinita

O teorema a seguir mostra que é possível construir algoritmos para determinar se uma Linguagem Livre do Contexto é vazia, finita ou infinita.

Teorema:

Linguagem Livre do Contexto Vazia, Finita ou Infinita

Se L é LLC, então é possível determinar se L é:

- a) Vazia;
- b) Finita;
- c) Infinita.

Prova: Suponha L uma LLC.

Vazia. Seja $G = (V, T, P, S)$, GLC tal que $GERA(G) = L$. Seja $G' = (V', T', P', S)$ equivalente a G , eliminando os símbolos inúteis. Se P' for vazio, então L é vazia.

Finita e Infinita. Seja $G = (V, T, P, S)$ uma GLC tal que $GERA(G) = L$. Seja $G' = (V', T', P', S)$ equivalente a G na Forma Normal de Chomsky. Se existe A , variável de V' tal que:

- $A \rightarrow BC$, ou seja, A é referenciada no lado esquerdo de uma produção que não gera diretamente terminais;
- $X \rightarrow YA$ ou $X \rightarrow AY$, ou seja, se A é referenciada no lado direito de alguma produção;
- existe um ciclo em A do tipo $A \Rightarrow^+ \alpha A \beta$;

então A é capaz de gerar palavras de qualquer tamanho e, conseqüentemente, a linguagem é infinita. Caso não exista tal A , então a linguagem é finita.

Algoritmos de Reconhecimento

Verificar se uma determinada palavra pertence ou não a uma linguagem é uma das principais questões relacionadas com o estudo de Linguagens Formais. Um "dispositivo de reconhecimento" de uma classe de linguagens pode ser especificado como um modelo de autômato ou como um algoritmo implementável em um computador. Em qualquer caso, é importante determinar a "quantidade de recursos" (por exemplo: tempo e espaço) que o dispositivo necessita para realizar o reconhecimento. Deve-se destacar que o objetivo em questão é gerar dispositivos de reconhecimento válidos para qualquer linguagem dentro de uma classe. Os algoritmos apresentados a seguir são específicos para as Linguagens Livres do Contexto.

Os algoritmos de reconhecimento que seguem são construídos a partir de uma gramática que define a linguagem. Os reconhecedores gerados usando Autômato com Pilha são muito simples, mas, em geral, ineficientes. Para uma entrada w , seu tempo de processamento é proporcional a $k^{|w|}$ (o valor de k depende do autômato), não sendo recomendáveis para entradas de tamanhos consideráveis. Existe uma série de algoritmos bem mais eficientes, com tempo de processamento proporcional a $|w|^3$ ou até um pouco menos. Não é provado se o tempo proporcional a $|w|^3$ é efetivamente necessário para que um algoritmo genérico reconheça Linguagens Livres do Contexto.

Os reconhecedores podem ser, basicamente, de dois tipos, como segue:

- a) *Top-Down* ou *Preditivo*. Constrói uma árvore de derivação para a palavra de entrada (a ser reconhecida) a partir da raiz (símbolo inicial da gramática), gerando os ramos em direção às folhas (símbolos terminais que compõem a palavra);
- b) *Bottom-Up*. É, basicamente, o oposto do *top-down*, partindo das folhas e construindo a árvore de derivação em direção à raiz.

Autômato com Pilha como Reconhecedor

A construção de reconhecedores usando Autômato com Pilha é relativamente simples e imediata, havendo uma relação quase direta entre as produções da gramática e as transições do autômato. Os algoritmos apresentados são do tipo *top-down* e simulam a derivação mais à esquerda da palavra a ser reconhecida. A facilidade de não-determinismo é usada para testar as diversas produções alternativas da gramática para gerar os símbolos terminais.

Autômato com Pilha Gerado a Partir de uma Gramática na Forma Normal de Greibach

Já foi mostrado que qualquer Linguagem Livre do Contexto pode ser especificada por um Autômato com Pilha. O algoritmo em questão define um AP a partir de uma Gramática na Forma Normal de Greibach. Como cada produção (na FNG) gera exatamente um terminal, uma palavra w é gerada em $|w|$ etapas de derivação. Entretanto, como cada variável pode ter mais de uma produção associada, é necessário testar, sistematicamente, as diversas alternativas. Assim, o número de passos para reconhecer w é proporcional a $k^{|w|}$, onde k depende do AP (uma aproximação para k é a

metade do número médio de produções associadas às diversas variáveis). Portanto, o Autômato construído possui um tempo de reconhecimento proporcional ao expoente em $|w|$, o que pode ser muito ineficiente para entradas mais longas.

Autômato com Pilha Descendente

O Autômato com Pilha Descendente é uma forma alternativa de construir um AP a partir de uma Gramática Livre do Contexto. Trata-se de um algoritmo igualmente simples e com o mesmo nível de eficiência. A construção é gerada a partir de uma gramática sem recursão à esquerda e simula a derivação mais à esquerda, como segue:

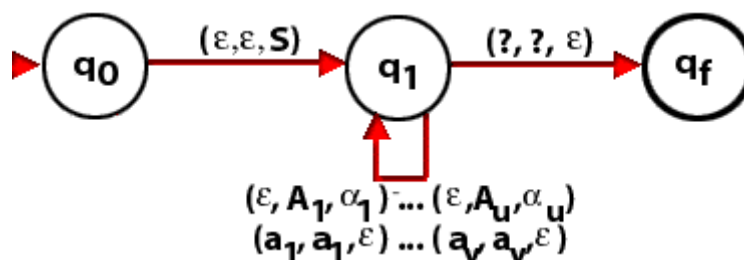
- inicialmente, empilha o símbolo inicial;
- sempre que existir uma variável no topo da pilha, substitui (de forma não-determinística) por todas as produções da variável;

se o topo da pilha for um terminal, verifica se é igual ao próximo símbolo da entrada.

Definição:

Definição Algoritmo para Construção de um AP Descendente

A *Algoritmo para Construção de um Autômato com Pilha Descendente* M a partir de uma gramática $G = (V, T, P, S)$, sem recursão à esquerda, é como segue e é ilustrada na Figura:



$M = (T, \{q_0, q_1, q_f\}, \delta, q_0, \{q_f\}, V \cup T)$, onde:

$$\delta(q_0, \varepsilon, \varepsilon) = \{(q_1, S)\}$$

$$\delta(q_1, \varepsilon, A) = \{(q_1, \alpha) \mid A \rightarrow \alpha \in P\}, \text{ para toda a variável } A \text{ de } V$$

$$\delta(q_1, a, a) = \{(q_1, \varepsilon)\}, \text{ para todo o terminal } a \text{ de } T$$

$$\delta(q_1, ?, ?) = \{(q_f, \varepsilon)\}$$

EXEMPLO:

Autômato com Pilha Descendente

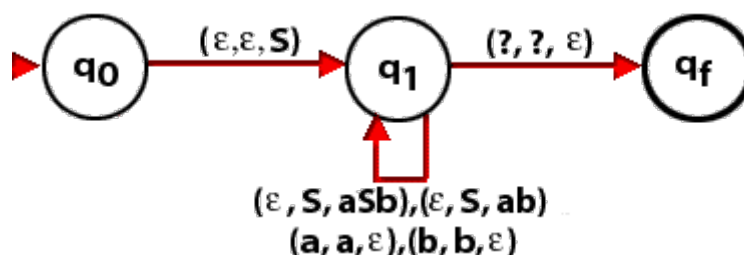
A linguagem $L_5 = \{a^n b^n \mid n \geq 1\}$, representada pela gramática (sem recursão à esquerda:

$G5' = (\{S\}, \{a, b\}, P5', S)$, onde:

$P5' = \{S \rightarrow aSb \mid ab\}$

é reconhecida pelo seguinte Autômato com Pilha Descendente o qual é ilustrado na Figura:

$M5' = (\{a, b\}, \{q_0, q_1, q_f\}, \delta5', q_0, \{q_f\}, \{S, a, b\})$



Algoritmo de Cocke-Younger-Kasami

O Algoritmo de Cocke-Younger-Kasami foi desenvolvido independentemente por Cocke, Younger e Kasami, em 1965. É construído sobre uma gramática na Forma Normal de Chomsky. Gera *bottom-up* todas as árvores de derivação da entrada em um tempo de processamento proporcional a $|w|^3$.

A idéia básica do algoritmo é a construção de uma tabela triangular de derivação, sendo que cada célula representa o conjunto de raízes que pode gerar a correspondente sub-árvore.

Definição:

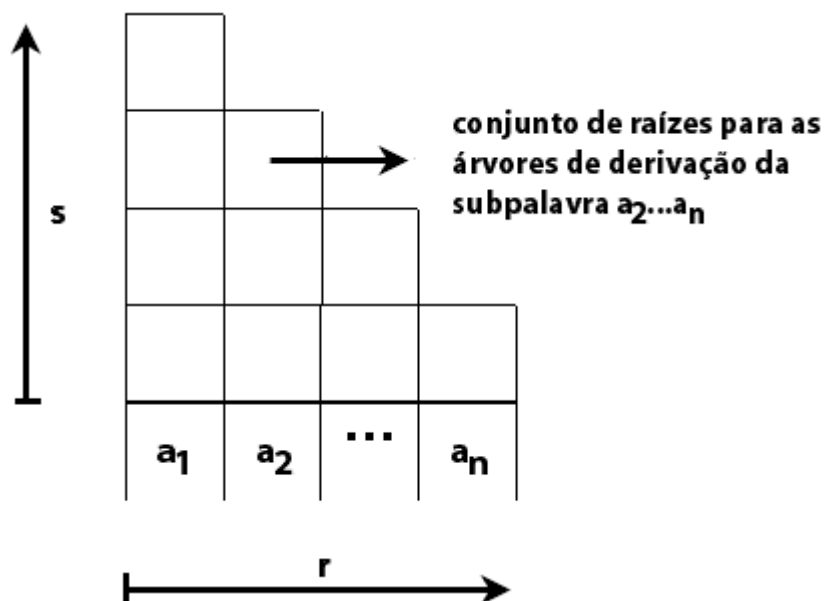
Algoritmo de Cocke-Younger-Kasami

Suponha $G = (V, T, P, S)$ uma gramática na Forma Normal de Chomsky onde $T = \{a_1, a_2, \dots, a_t\}$ e suponha $w = a_1a_2\dots a_n$ uma entrada a ser verificada. O *Algoritmo de Cocke-Younger-Kasami (CYK)* é composto pelas seguintes etapas onde $V_{r,s}$ representa as células de uma tabela triangular de derivação a qual é ilustrada na Figura:

a) *Variáveis que geram diretamente terminais* ($A \rightarrow a$).

para r variando de 1 até n

faça $V_{r,1} = \{A \mid A \rightarrow ar \in P\}$



b) *Produção que gera duas variáveis* ($A \rightarrow BC$).

para s variando de 2 até n

faça para r variando de 1 até $n - s + 1$

faça $V_{rs} = \emptyset$

para k variando de 1 até $s - 1$

faça $V_{rs} = V_{rs} \cup \{ A \mid A \rightarrow BC \in P, B \in V_{rk}, C \in V_{(r+k)(s-k)} \}$

Note-se que:

- limite de iteração para r é $(n - s + 1)$, pois a tabela é triangular;

EXEMPLO:

Algoritmo de Cocke-Younger-Kasami

Considere a seguinte gramática:

$G = (\{S, A\}, \{a, b\}, P, S)$, onde:

$P = \{S \rightarrow A A \mid A S \mid b, A \rightarrow S A \mid A S \mid a\}$

A tabela triangular de derivação para a palavra de entrada **abaab** é ilustrada na Figura.

Algoritmo de Early

O Algoritmo EARLY foi desenvolvido em 1968. É considerado o mais rápido algoritmo de reconhecimento conhecido para Gramáticas Livres do Contexto. É construído a partir de uma GLC qualquer e possui tempo de processamento proporcional a $|w|^3$.

Entretanto, para gramáticas não-ambíguas, pode ser implementado em um tempo proporcional a $|w|^2$. Adicionalmente, para muitas gramáticas de interesse prático, o tempo é proporcional ao tamanho da entrada, ou seja, $|w|$.

Trata-se de um algoritmo do tipo *top-down*, que parte do símbolo inicial e executa sempre a derivação mais à esquerda. Cada ciclo gera um terminal, o qual é comparado com o símbolo da entrada. A comparação com sucesso determina a construção de um conjunto de produções que, potencialmente, pode gerar o próximo símbolo.

Definição:

Algoritmo de Early

Suponha $G = (V, T, P, S)$ uma Gramática Livre do Contexto qualquer e $w = a_1a_2...a_n$ uma palavra a ser verificada. No que segue, o símbolo "." é usado como um marcador, antecedendo a posição, em cada produção, que será analisada na tentativa de gerar o próximo símbolo terminal. Adicionalmente, o sufixo "/u" adicionado a cada produção indica o u -ésimo ciclo em que esta produção passou a ser considerada. As etapas do *Algoritmo de Early* são as seguintes:

- a) *Construção de D_0 , o primeiro conjunto de produções*. São incluídas em D_0 todas as produções que partem do símbolo inicial S , bem como todas as produções que podem ser aplicadas em sucessivas derivações mais à esquerda (a partir de S). O algoritmo desta etapa é o seguinte:

$$D_0 = \emptyset$$

para toda $S \rightarrow \alpha \in P$ (1)

faça $D_0 = D_0 \cup \{ S \rightarrow \cdot \alpha / 0 \}$

repita (2)

para toda $A \rightarrow \cdot B \beta / 0 \in D_0$

faça para toda $B \rightarrow \phi \in P$

faça $D_0 = D_0 \cup \{ B \rightarrow \cdot \phi / 0 \}$

até que o cardinal de D_0 não aumente

No algoritmo acima, tem-se que:

(1) representa as produções que partem de S ;

(2) inclui todas as produções que podem derivar (mais à esquerda) o próximo símbolo;

- b) *Construção dos demais conjuntos de produção*. São construídos n conjuntos de produção a partir de D_0 , onde $n = |w|$. Ao gerar o símbolo a_r de w , o algoritmo

constrói o conjunto D_r , contendo as produções que potencialmente podem gerar o símbolo a_{r+1} . O algoritmo desta etapa é o seguinte:

para r variando de 1 até n (1)

faça $D_r = \emptyset$;

para toda $A \rightarrow \alpha. a_r \beta / s \in D_{r-1}$ (2)

faça $D_r = D_r \cup \{ A \rightarrow \alpha a_r . \beta / s \}$;

repita

para toda $A \rightarrow \alpha. B \beta / s \in D_r$ (3)

faça para toda $B \rightarrow \phi \in P$

faça $D_r = D_r \cup \{ B \rightarrow . \phi / r \}$

para toda $A \rightarrow \alpha. / s$ de D_r (4)

faça para toda $B \rightarrow \beta. A \phi / k \in D_s$

faça $D_r = D_r \cup \{ B \rightarrow \beta A . \phi / k \}$

até que o cardinal de D_r não aumente

No algoritmo acima, tem-se que:

(1) cada ciclo gera um conjunto de produções D_r ;

(2) gera o símbolo a_r ;

(3) inclui todas as produções que podem derivar (mais à esquerda) o próximo símbolo (análogo ao realizado na etapa a) acima);

(4) uma subpalavra de w foi reduzida à variável A : inclui em D_r todas as produções de D_s que referenciam $.A$;

c) **Condição de aceitação da entrada.** Se uma produção da forma $S \rightarrow \alpha. / 0$ pertence a D_n , então a palavra w de entrada foi aceita. Deve-se reparar que $S \rightarrow \alpha. / 0$ é uma produção que:

- parte do símbolo inicial S ;
- foi incluída em D_0 (" $/0$ ");
- todo o lado direito da produção foi analisado com sucesso (o marcador "." está no final de α).

Note-se que, para otimizar as etapas a) e b) do algoritmo acima, os ciclos **repita-até** podem ser restritos exclusivamente às produções recentemente incluídas em D_r ou em D_0 ainda não-analisadas.

EXEMPLO:*Algoritmo de Early*

Considere a seguinte gramática que gera expressão com colchetes balanceados e operação de adição e multiplicação (é análoga à definição de "expressão simples" da linguagem de programação Pascal):

$G = (\{E, T, F\}, \{+, *, [,], x\}, P, E)$, onde:

$P = \{E \rightarrow E \mid E+T, T \rightarrow F \mid T*F, F \rightarrow [E] \mid x\}$

O reconhecimento da palavra $x*x$ é como segue:

D0:

$E \rightarrow .T/0$ produções que partem

$E \rightarrow .E+T/0$ do símbolo inicial;

$T \rightarrow .F/0$ produções que podem ser aplicadas

$T \rightarrow .T*F/0$ em derivação mais à esquerda

$F \rightarrow .(E)/0$ a partir do símbolo inicial.

$F \rightarrow .x/0$

D1: reconhecimento de x em $x*x$

$F \rightarrow x./0$ x foi reduzido à F ;

$T \rightarrow F./0$ inclui todas as produções de D0 que

$T \rightarrow T.*F/0$ referenciam $.F$ direta ou indiretamente, (pois $F \rightarrow x./0$)

$E \rightarrow T./0$ movendo o marcador "."

$E \rightarrow E.+T/0$ um símbolo para a direita.

D2: reconhecimento de $*$ em $x*x$

$T \rightarrow T*.F/0$ gerou $*$; o próximo será gerado por F ;

$F \rightarrow .(E)/2$ inclui todas as produções de P que

$F \rightarrow .x/2$ podem gerar o próximo terminal a partir de F .

D3: reconhecimento de x em $x*x$

$F \rightarrow x./2$ x foi reduzido à F ;

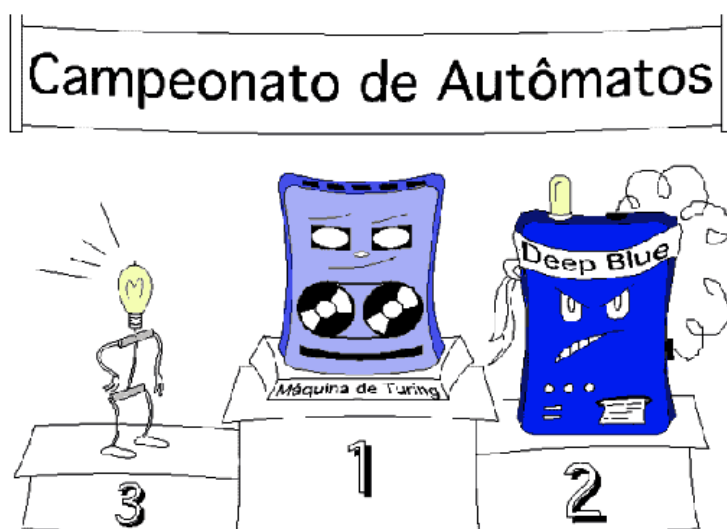
$T \rightarrow T^*F./0$ incluído de D_2 (pois $F \rightarrow x./2$); a entrada foi reduzida à T ;

$E \rightarrow T./0$ incluído de D_0 (pois $T \rightarrow T^*F./0$); a entrada foi reduzida à E ;

$T \rightarrow T.*F/0$ incluído de D_0 (pois $T \rightarrow T^*F./0$);

$E \rightarrow E.+T/0$ incluído de D_0 (pois $E \rightarrow T./0$).

Como $w = x^*x$ foi reduzida ao símbolo inicial E , ou seja, $E \rightarrow T./0$ pertence a D_3 , a entrada foi aceita.



Linguagens Enumeráveis Recursivamente e Sensíveis ao Contexto

Ciência da Computação é o conhecimento sistematizado relativo à computação. Sua origem é remota, tendo exemplos na antiga Grécia (século III a.C., no desenho de algoritmos por Euclides) e Babilônia (com estudos sobre complexidade e reducibilidade de problemas). No início do século XX, diversas pesquisas foram desenvolvidas com o objetivo de definir um modelo computacional suficientemente genérico, capaz de implementar qualquer "função computável".

Em 1936, Alan Turing propôs um modelo conhecido como *Máquina de Turing*. Atualmente, a Máquina de Turing é aceita como uma formalização de um *procedimento efetivo* (*algoritmo* ou *função computável*), ou seja, uma sequência finita de instruções, as quais podem ser realizadas mecanicamente, em um tempo finito.

Ainda em 1936, Alonzo Church apresentou a *Hipótese de Church*, a qual afirma que qualquer função computável pode ser processada por uma Máquina de Turing, ou seja, existe um procedimento expresso na forma de uma Máquina de Turing capaz de processar a função. Contudo, como a noção intuitiva de procedimentos não é matematicamente precisa, é impossível demonstrar formalmente se a Máquina de Turing é, efetivamente, o mais genérico dispositivo de computação. Entretanto, foi mostrado que todos os demais modelos propostos possuem, no máximo, a mesma capacidade computacional de Turing, o que reforça a Hipótese de Church.

As *Linguagens Enumeráveis Recursivamente* ou *Tipo 0* são aquelas que podem ser reconhecidas por uma Máquina de Turing. Considerando que, segundo a Hipótese de Church, a Máquina de Turing é o mais geral dispositivo de computação, então a Classe das Linguagens Enumeráveis Recursivamente representa o conjunto de todas as linguagens que podem ser reconhecidas mecanicamente e em um tempo finito.

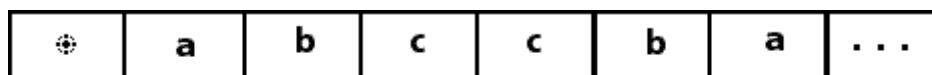
Analogamente às demais classes de linguagens, é possível representar as Linguagens Enumeráveis Recursivamente usando um formalismo axiomático ou gerador, na forma de gramática, denominado *Gramática Irrestrita*. Como o próprio nome indica, uma Gramática Irrestrita não possui qualquer restrição sobre a forma das produções.

As *Linguagens Sensíveis ao Contexto* ou *Tipo 1* estão contidas propriamente nas Enumeráveis Recursivamente (bem como nas Recursivas, introduzidas adiante). Entretanto, incluem praticamente todas as linguagens aplicadas.

Como nas demais classes de linguagens, o estudo da Classe das Linguagens Sensíveis ao Contexto pode ser desenvolvido a partir de formalismos gerador e reconhecedor, como segue:

a) *Gramática Sensível ao Contexto*. O termo "sensível ao contexto" deriva do fato de que o lado esquerdo das produções da gramática pode ser uma palavra de variáveis ou terminais, definindo um "contexto" de derivação;

b) *Máquina de Turing com Fita Limitada*. Trata-se de uma Máquina de Turing com limitação no tamanho da fita a qual, portanto, é finita. Deve-se destacar o fato de que não é conhecido se a facilidade de não-determinismo aumenta ou não o poder computacional das Máquinas de Turing com Fita Limitada.



Máquina de Turing

A Máquina de Turing para ser considerada como um modelo formal de *procedimento efetivo*, *algoritmo* ou *função computável* deve satisfazer às seguintes propriedades, entre outras:

a) A descrição do algoritmo deve ser finita;

b) Deve consistir de passos:

- discretos;
- executáveis mecanicamente;
- em um tempo finito.

O modelo proposto por Alan Turing em 1936, conhecido como Máquina de Turing, consiste basicamente de 3 partes:

a) *Fita*. Usada simultaneamente como dispositivo de entrada, saída e memória de trabalho;

b) *Unidade de Controle*. Reflete o estado corrente da máquina. Possui uma unidade de leitura e gravação (cabeça da fita) a qual acessa uma célula da fita de cada vez e movimenta-se para a esquerda ou direita;

c) *Programa* ou *Função de Transição*. Função que comanda as leituras e gravações, o sentido de movimento da cabeça e define o estado da máquina.

A fita é finita à esquerda e infinita à direita, sendo dividida em células, onde cada uma armazena um símbolo. Os símbolos podem pertencer ao alfabeto de entrada, ao alfabeto auxiliar ou ainda, ser "branco" ou "marcador de início de fita".

Inicialmente, a palavra a ser processada (ou seja, a informação de entrada para a máquina) ocupa as células mais à esquerda, após o marcador de início de fita, ficando as demais com "branco", como ilustrado na [figura abaixo](#), onde β e $\clubsuit$ representam "branco" e "marcador de início de fita", respectivamente.

A unidade de controle possui um número finito e predefinido de estados. A *cabeça da fita* lê o símbolo de uma célula de cada vez e grava um novo símbolo. Após a leitura/gravação, a cabeça move uma célula para a direita ou esquerda. O símbolo gravado e o sentido do movimento são definidos pelo programa.

O programa é uma função que, dependendo do estado corrente da máquina e do símbolo lido, determina o símbolo a ser gravado, o sentido do movimento da cabeça e o novo estado.

Definição:

Máquina de Turing

Uma *Máquina de Turing* é uma 8-upla:

$$M = (\Sigma, Q, \delta, q_0, F, V, \beta, \clubsuit)$$

onde:

Σ *alfabeto de símbolos de entrada*;

Q *conjunto de estados* possíveis da máquina o qual é finito;

δ *programa* ou *função de transição*:

$$\delta: Q \times (\Sigma \cup V \cup \{\beta, \clubsuit\}) \rightarrow Q \times (\Sigma \cup V \cup \{\beta, \clubsuit\}) \times \{E, D\}$$

a qual é uma função parcial;

q_0 *estado inicial* da máquina tal que q_0 é elemento de Q ;

F *conjunto de estados finais* tal que F está contido em Q ;

V *alfabeto auxiliar* (pode ser vazio);

β símbolo especial *branco*;

$\clubsuit$ *símbolo* ou *marcador de início* da fita.

O símbolo de início de fita sempre ocorre exatamente uma vez e na célula mais à esquerda da fita, auxiliando na identificação de que a cabeça da fita se encontra na célula mais à esquerda da fita.

A função programa, considera o estado corrente e o símbolo lido da fita para determinar o novo estado, o símbolo a ser gravado e sentido de movimento da cabeça onde esquerda e direita são representados por **E** e **D**, respectivamente. A função programa pode ser interpretada como um grafo finito direto, como ilustrado na [figura abaixo](#). Neste caso, os estados inicial e finais são representados como nos Autômatos Finitos.

O processamento de uma Máquina de Turing $M = (\Sigma, Q, \delta, q_0, F, V, \beta, \odot)$, para uma palavra de entrada w , consiste na sucessiva aplicação da função programa a partir do estado inicial q_0 e da cabeça posicionada na célula mais à esquerda da fita até ocorrer uma condição de parada. O processamento de M para a entrada w , pode parar ou ficar em *loop* infinito. A parada pode ser de duas maneiras: aceitando ou rejeitando a entrada w . As condições de parada são as seguintes:

- a) A máquina assume um estado final: a máquina pára e a palavra de entrada é aceita;
- b) A função programa é indefinida para o argumento (símbolo lido e estado corrente): a máquina pára e a palavra de entrada é rejeitada;
- c) O argumento corrente da função programa define um movimento à esquerda e a cabeça da fita já se encontra na célula mais à esquerda: a máquina pára e a palavra de entrada é rejeitada.

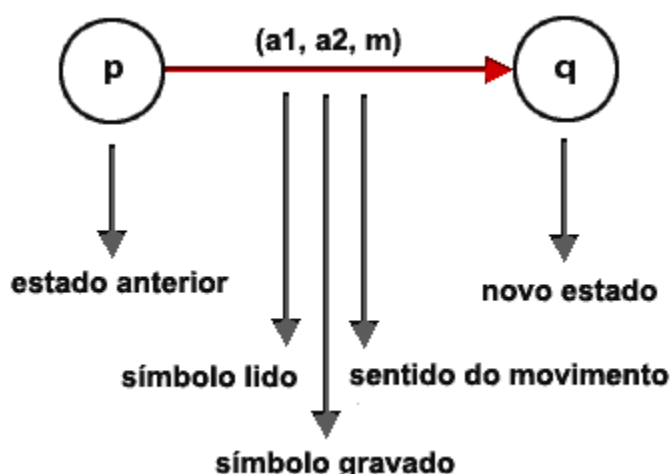


Figura Representação da função programa como um grafo

Para definir formalmente o comportamento de uma Máquina de Turing, é necessário estender a definição da função programa usando como argumento um estado e uma palavra. Esta extensão é sugerida como exercício.

As seguintes notações são adotadas, as quais são análogas às usadas para Autômatos com Pilha (suponha que M é uma Máquina de Turing):

- a) **ACEITA(M)** ou **L(M)**: conjunto de todas as palavras de Σ^* aceitas por M ;
- b) **REJEITA(M)**: conjunto de todas as palavras de Σ^* rejeitadas por M ;
- c) **LOOP(M)**: conjunto de todas as palavras de Σ^* para as quais M fica processando indefinidamente.

Da mesma forma, as seguintes afirmações são verdadeiras:

- $ACEITA(M) \cap REJEITA(M) \cap LOOP(M) = \emptyset$
- $ACEITA(M) \cup REJEITA(M) \cup LOOP(M) = \Sigma^*$
- o complemento de:

$ACEITA(M)$ é $REJEITA(M) \cup LOOP(M)$

$REJEITA(M)$ é $ACEITA(M) \cup LOOP(M)$

$LOOP(M)$ é $ACEITA(M) \cup REJEITA(M)$

EXEMPLO:

Máquina de Turing - Duplo Balanceamento

Considere a linguagem:

$$L_1 = \{a^n b^n \mid n \in \mathbb{N}\}$$

A Máquina de Turing:

$$M_1 = (\{a, b\}, \{q_0, q_1, q_2, q_3, q_4\}, \delta_1, q_0, \{q_4\}, \{A, B\}, \beta, \oplus)$$

ilustrada na [figura abaixo](#) e onde δ_1 é como na tabela abaixo, é tal que:

$$ACEITA(M_1) = L_1$$

$$REJEITA(M_1) = \Sigma^* - L_1$$

e, portanto, $LOOP(M_1) = \emptyset$.

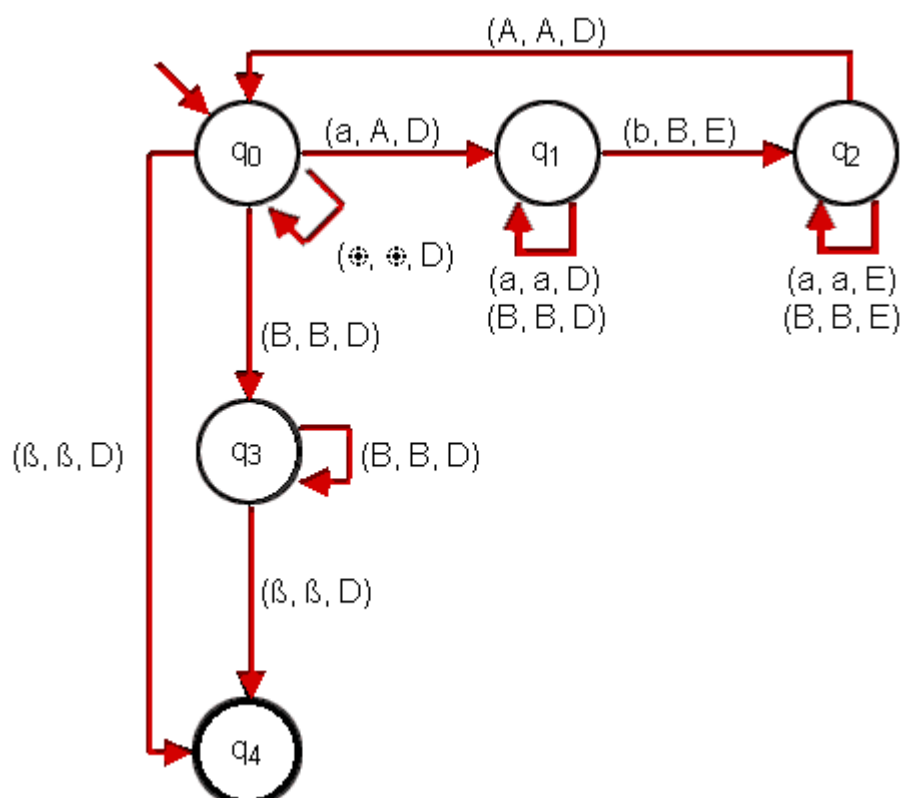


Figura Máquina de Turing

δ	$\odot$	a	b	A	B	β
q0	$(q_0, \odot, D)$	(q_1, A, D)			(q_3, B, D)	(q_4, β, D)
q1		(q_1, a, D)	(q_2, B, E)		(q_1, B, D)	
q2		(q_2, a, E)		(q_0, A, D)	(q_2, B, E)	
q3					(q_3, B, D)	(q_4, β, D)
q4						

O algoritmo apresentado reconhece o primeiro símbolo a , o qual é marcado como A , e movimenta a cabeça da fita à direita, procurando o b correspondente, o qual é marcado como B . Este ciclo é repetido sucessivamente até identificar para cada a o seu correspondente b . Adicionalmente, o algoritmo garante que qualquer outra palavra que não esteja na forma $a^n b^n$ é rejeitada. Note-se que o símbolo de início de fita não tem influência na solução proposta.

Modelos Equivalentes à Máquina de Turing

Uma das razões para considerar a Máquina de Turing como o mais geral dispositivo de computação é o fato de que todos os demais modelos e máquinas propostos, bem como diversas modificações da Máquina de Turing, possuem, no máximo, o mesmo poder computacional da Máquina de Turing. As modificações apresentadas a seguir são frequentemente usadas.

a) *Autômato com Múltiplas Pilhas*. Conforme citado no estudo das Linguagens Livres do Contexto, o poder computacional do Autômato com Duas Pilhas (A2P) é equivalente ao

da Máquina de Turing. Adicionalmente, um maior número de pilhas (AnP, $n > 2$) também não aumenta a capacidade computacional. A Definição: formal do A2P e AnP e a equivalência ao modelo da Máquina de Turing é sugerida como **exercício**;

b) *Máquina de Turing Não-Determinística*. A facilidade de não-determinismo não aumenta o poder computacional da Máquina de Turing;

c) *Máquina de Turing com Fita Infinita à Esquerda e à Direita*. A modificação da definição básica da Máquina de Turing permitindo que a fita seja infinita dos dois lados não aumenta o seu poder computacional. Na realidade, a fita infinita à esquerda e à direita pode ser facilmente simulável por uma fita tradicional, como ilustrado na [figura baixo](#) onde as células pares representam a parte direita da fita e as ímpares a parte esquerda;

d) *Máquina de Turing com Múltiplas Fitas*. A Máquina de Turing com múltiplas fitas possui k fitas infinitas à esquerda e à direita e k cabeças de fita. A função programa é como segue:

- dependendo do estado corrente da máquina e do símbolo lido em cada uma das fitas;
- grava um novo símbolo em cada uma das fitas;
- move cada uma das cabeças independentemente;
- a máquina assume um (único) novo estado.

Inicialmente, a palavra de entrada é armazenada na primeira fita, ficando as demais com valor branco;

e) *Máquina de Turing Multidimensional*. Neste modelo, a fita tradicional é substituída por uma estrutura do tipo arranjo k -dimensional, infinita em todas as $2k$ direções;

f) *Máquina de Turing com Múltiplas Cabeças*. A Máquina de Turing com esta modificação possui k cabeças de leitura e gravação sobre a mesma fita. Cada cabeça possui movimento independente. Assim, o processamento depende do estado corrente e do símbolo lido em cada uma das cabeças;

g) *Combinações de Modificações sobre a Máquina de Turing*. A combinação de algumas ou todas as modificações apresentadas não aumenta o poder computacional da Máquina de Turing. Por exemplo, uma Máquina de Turing não-determinística com múltiplas fitas e múltiplas cabeças pode ser simulada por uma Máquina de Turing tradicional.

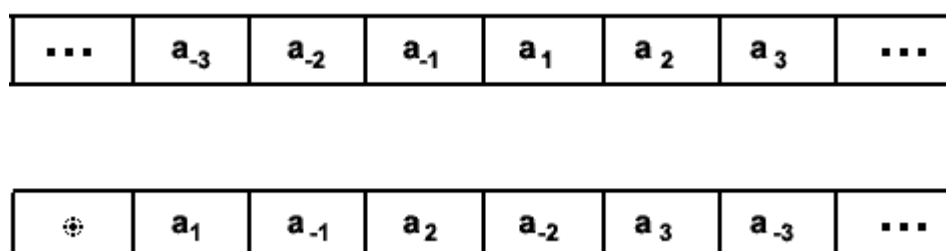


Figura 4.5 Simulação de uma fita infinita à esquerda e à direita

Hipótese de Church

A. M. Turing propôs, em 1936, um modelo abstrato de computação, conhecido como Máquina de Turing, com o objetivo de explorar os limites da capacidade de expressar soluções de problemas. Trata-se, portanto, de uma proposta de definição formal da noção intuitiva de algoritmo. Diversos outros trabalhos, como *Máquina de Post* (Post - 1936) e *Funções Recursivas* (Kleene - 1936), resultaram em conceitos equivalentes ao de Turing. O fato de todos estes trabalhos independentes gerarem o mesmo resultado em termos de capacidade de expressar computabilidade é um forte reforço no que é conhecido como *Hipótese de Church* ou *Hipótese de Turing-Church*:

"A capacidade de computação representada pela Máquina de Turing é o limite máximo que pode ser atingido por qualquer dispositivo de computação"

Em outras palavras, a Hipótese de Church afirma que qualquer outra forma de expressar algoritmos terá no máximo a mesma capacidade computacional da Máquina de Turing. Como a noção de algoritmo ou Função Computável é intuitiva, a Hipótese de Church não é demonstrável.

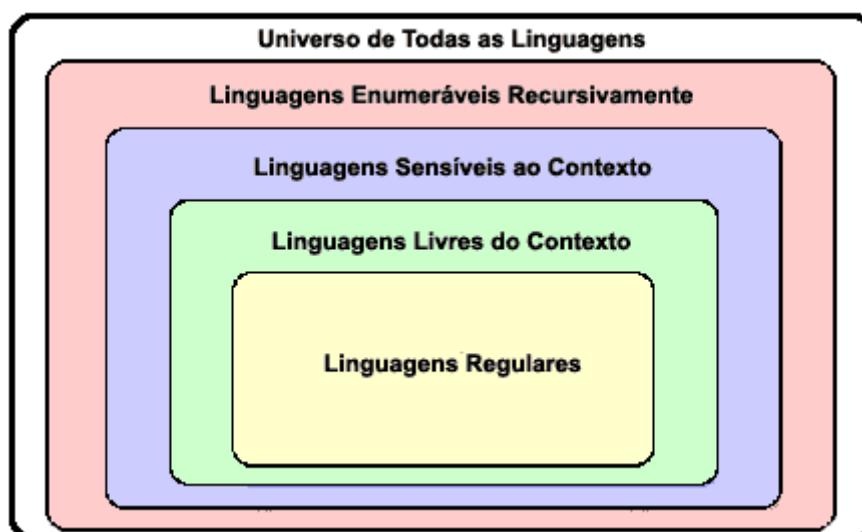
Máquinas de Turing como Reconhedores

Uma linguagem aceita por uma Máquina de Turing é dita Enumerável Recursivamente ou Tipo 0. Historicamente, o termo "enumerável" deriva do fato de que as palavras de qualquer Linguagem Enumerável Recursivamente podem ser enumeradas ("listadas") por uma Máquina de Turing; e "recursivamente" é um termo matemático anterior ao computador, com significado similar ao de "recursão", usado em informática.

A Classe das Linguagens Enumerável Recursivamente inclui algumas para as quais é impossível determinar mecanicamente se uma palavra não pertence à linguagem. Se L é uma destas linguagens, então para qualquer máquina M que aceita L , existe pelo menos uma palavra w não pertencente a L que, ao ser processada por M , a máquina entra em *loop* infinito. Assim, pode-se afirmar que:

- se w pertence a L , M pára e aceita a entrada;
- se w não pertence a L , M pode parar, rejeitando a palavra ou permanecer processando indefinidamente.

Portanto, é conveniente definir uma subclasse da Classe das Linguagens Enumerável Recursivamente, denominada Classe das Linguagens Recursivas, composta pelas linguagens para as quais existe pelo menos uma Máquina de Turing que pára para qualquer entrada, aceitando ou rejeitando.



Linguagens Enumeráveis Recursivamente

As Linguagens Enumeráveis Recursivamente são definidas a partir das Máquinas de Turing.

Definição:

Linguagem Enumerável Recursivamente ou Tipo 0

Uma linguagem aceita por uma Máquina de Turing é dita *Linguagem Enumerável Recursivamente* ou *Tipo 0*.

EXEMPLO:

Linguagem Enumerável Recursivamente

As seguintes linguagens são exemplos de Linguagens Enumeráveis Recursivamente sendo que, para a primeira, a correspondente Máquina de Turing foi construída em outro exemplo ([Exemplo Máquina de Turing - Duplo Balanceamento](#)) as demais são sugeridas como exercício:

- a) $\{a^n b^n \mid n \geq 0\}$
- b) $\{w \mid w \text{ tem o mesmo número de símbolos } a \text{ e } b\}$
- c) $\{a^i b^j c^k \mid i = j \text{ ou } j = k\}$

No teorema a seguir, mostra-se que existe pelo menos uma linguagem que não é Enumeráveis Recursivamente.

Teorema:

Linguagem Não-Enumerável Recursivamente

Suponha $\Sigma = \{a, b\}$.

a) Suponha que X_i representa o i -ésimo elemento na ordenação lexicográfica de Σ^* , ou seja, $\{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$;

b) Conforme é proposto nos exercícios, é possível codificar todas as Máquinas de Turing como uma palavra sobre Σ de tal forma que cada código represente uma única Máquinas de Turing. Suponha o conjunto dos códigos ordenados lexicograficamente e suponha que T_i representa o i -ésimo código nesta ordenação.

A linguagem que segue não é Enumerável Recursivamente:

$$L = \{X_i \mid X_i \text{ não é aceita por } T_i\}$$

Prova: A prova que segue é por redução ao absurdo. Suponha que L é Enumerável Recursivamente. Então, por definição, existe uma Máquina de Turing que aceita L . Seja T_k a codificação desta Máquina de Turing, ou seja, $ACEITA(T_k) = L$. Assim, as seguintes afirmações são válidas:

- a) Por definição de L , X_k pertence a L se, e somente se, X_k não é aceita por T_k ;
- b) Entretanto, como T_k aceita a linguagem L , X_k pertence a L se, e somente se, X_k é aceita por T_k .

Claramente b) contradiz a). Portanto, é absurdo supor que L é Enumerável Recursivamente. Logo, L não é Enumerável Recursivamente.

Observação:

Note-se que, na prova do **Teorema Linguagem Não-Enumerável Recursivamente**, afirma-se que existe uma codificação de todas as Máquinas de Turing. Portanto, o conjunto destas codificações é isomorfo a um subconjunto infinito dos números naturais. Logo, o conjunto de todas as Máquinas de Turing (ou de todos os problemas solucionáveis) é contável. Em contra-partida, é interessante destacar que o cardinal do conjunto das linguagens que não são Enumeráveis Recursivamente (ou dos problemas não-solucionáveis) é infinito e não-contável.

Linguagens Recursivas

As Linguagens Recursivas também são definidas a partir das Máquinas de Turing.

Definição:

Linguagem Recursiva

Uma linguagem L é dita **Linguagem Recursiva** se existe uma Máquina de Turing M tal que:

- a) $ACEITA(M) = L$
- b) $REJEITA(M) = \Sigma^* - L$

Ou seja, uma linguagem é Recursiva se existe pelo menos uma Máquina de Turing que aceita a linguagem e sempre pára para qualquer entrada.

EXEMPLO:

Linguagem Recursiva

É fácil verificar que as seguintes linguagens são Recursivas:

- a) $\{a^n b^n \mid n \in \mathbb{N}\}$
- b) $\{a^n b^n c^n \mid n \in \mathbb{N}\}$
- c) $\{w \mid w \in \{a, b\}^* \text{ e tem o dobro de símbolos } a \text{ que } b\}$

Propriedades das Linguagem Enumeráveis Recursivamente e Recursivas

A seguir são apresentadas algumas das principais propriedades das Linguagens Enumeráveis Recursivamente e das Recursivas.

Teorema:

Complemento de uma Linguagem Recursiva é Recursiva

Se uma linguagem L sobre um alfabeto Σ qualquer é Recursiva, então o seu complemento $\Sigma^* - L$ também é uma Linguagem Recursiva.

Prova: Suponha L uma Linguagem Recursiva sobre Σ . Então existe M , Máquina de Turing, que aceita a linguagem e sempre pára para qualquer entrada. Ou seja:

$$ACEITA(M) = L$$

$$REJEITA(M) = \Sigma^* - L$$

$$LOOP(M) = \emptyset$$

Seja M' uma Máquina de Turing construída a partir de M , mas invertendo-se as condições de $ACEITA$ por $REJEITA$ e vice-versa (como a inversão pode ser implementada?). Portanto, M' aceita $\Sigma^* - L$ e sempre pára para qualquer entrada. Ou seja:

$$ACEITA(M') = \Sigma^* - L$$

$$REJEITA(M') = L$$

$$LOOP(M') = \emptyset$$

Logo, $\Sigma^* - L$ é uma Linguagem Recursiva.

Teorema:

Linguagem Recursiva $\leftrightarrow$ Enumerável Recursivamente

Uma linguagem L sobre um alfabeto Σ qualquer é Recursiva se, e somente se, L e $\Sigma^* - L$ são Enumeráveis Recursivamente.

Prova:

a) Suponha L uma Linguagem Recursiva sobre Σ . Então, como foi mostrado no Teorema 4.5, $\Sigma^* - L$ é Recursiva. Como toda Linguagem Recursiva também é Enumerável Recursivamente, então L e $\Sigma^* - L$ são Enumeráveis Recursivamente;

b) Suponha L uma linguagem sobre Σ tal que L e $\Sigma^* - L$ são Enumeráveis Recursivamente. Então existem M_1 e M_2 , Máquinas de Turing tais que:

$$ACEITA(M_1) = L$$

$$ACEITA(M_2) = \Sigma^* - L$$

Seja M Máquina de Turing não-determinística definida conforme esquema ilustrado na figura abaixo (como seria, detalhadamente, a definição de M ?) Para qualquer palavra de entrada, M aceita se M_1 aceita e M rejeita se M_2 aceita. Portanto, claramente M sempre pára. Logo, L é Recursiva.

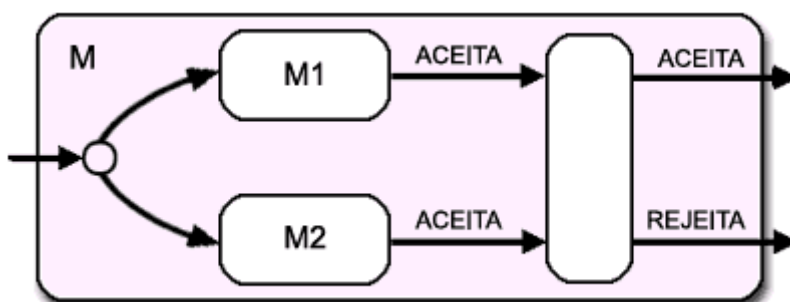
Teorema:

Linguagem Recursiva $\subseteq$ Enumerável Recursivamente

A Classe das Linguagens Recursivas está contida propriamente na Classe das Linguagens Enumeráveis Recursivamente.

Prova: Para mostrar que a inclusão é própria, é suficiente mostrar que existe pelos menos uma Linguagem Enumerável Recursivamente que não é Recursiva. A Linguagem Enumerável Recursivamente que segue não é Recursiva, onde X_i e T_i são como definidas em Teorema 4.3 (cuidado para não confundir com a linguagem $\{X_i \mid X_{i \text{ não é aceita por } T_i}\}$):

$$L = \{X_i \mid X_i \text{ é aceita por } T_i\}$$



a) L é *Enumerável Recursivamente*. Segue um esboço da construção de uma Máquina de Turing M que aceita uma palavra w qualquer pertencente a L :

a.1) M gera as palavras $X_1, X_2, \dots$ em ordem lexicográfica, comparando com w . Quando $X_i = w$, M sabe que w é a i -ésima palavra na enumeração;

a.2) M gera T_i , a i -ésima Máquina de Turing (conforme algoritmo proposto nos exercícios);

a.3) M simula T_i para a entrada $w = X_i$ e, se w pertence a $ACEITA(T_i)$, então w pertence a $ACEITA(M)$ (o algoritmo do simulador também é proposto como exercício);

Portanto, M aceita w se, e somente se, $X_i = w$ é aceita por T_i . Logo, L é Enumerável Recursivamente;

b) L não é *Recursiva*. Conforme o Teorema 4.6, L é Recursiva se, e somente se, L e seu complemento são Enumeráveis Recursivamente. Como o complemento de L , conforme o Teorema 4.3, não é Enumerável Recursivamente, então L não é Recursiva.

Gramática Irrestrita

Contrariamente às demais gramáticas introduzidas anteriormente, quais sejam, Regulares e Livres do Contexto, uma Gramática Irrestrita é simplesmente uma gramática, sem qualquer restrição nas produções.

Definição:

Gramática Irrestrita

Qualquer gramática $G = (V, T, P, S)$ é uma *Gramática Irrestrita* ou *Tipo 0*.

EXEMPLO:

Gramática Irrestrita

A linguagem $L = \{a^n b^n c^n \mid n \geq 0\}$ é gerada pela seguinte Gramática Irrestrita:

$G = (\{S, C\}, \{a, b, c\}, P, S)$, onde:

$P = \{ S \rightarrow abc \mid \epsilon,$

$ab \rightarrow aabbC,$

$Cb \rightarrow bC,$

$Cc \rightarrow cc\}$

A palavra **aaabbbccc** pode ser derivada como segue (existe alguma outra sequência de derivação que gera a mesma palavra?):

$S \Rightarrow abc \Rightarrow aabbCc \Rightarrow aaabbCbCc \Rightarrow aaabbCbcc \Rightarrow aaabbbCcc \Rightarrow aaabbbccc$

Deve-se reparar que a variável **C** "caminha" na palavra até a posição correta para gerar um terminal **c**.

Teorema:

Ling. Enumerável Recursivamente $\times$ **Gramática Irrestrita.**

L é uma Linguagem Enumerável Recursivamente se, e somente se, **L** é gerada por uma Gramática Irrestrita.

Linguagem Sensível ao Contexto

As Linguagens Sensíveis ao Contexto são definidas a partir das Gramáticas Sensíveis ao Contexto.

Definição:

Gramática Sensível ao Contexto

Uma **Gramática Sensível ao Contexto** **G** é uma gramática $G = (V, T, P, S)$ com a restrição de que qualquer regra de produção de **P** é da forma $\alpha \rightarrow \beta$, onde:

a) α é uma palavra de $(V \cup T)^+$

b) β uma palavra de $(V \cup T)^*$

c) $|\alpha| \leq |\beta|$, excetuando-se, eventualmente, para $S \rightarrow \epsilon$. Neste caso, **S** não pode estar presente no lado direito de qualquer produção.

Ou seja, a cada etapa de derivação, o tamanho da palavra derivada não pode diminuir, excetuando-se para gerar a palavra vazia, se esta pertencer à linguagem.

Definição:

Linguagem Sensível ao Contexto ou Tipo 1

Uma linguagem gerada por uma Gramática Sensível ao Contexto é dita **Linguagem Sensível ao Contexto** ou **Tipo 1**.

EXEMPLO:

Linguagem Sensível ao Contexto

A linguagem (introduzida no **Capítulo 3 - Linguagens Livres do Contexto**, como exemplo de linguagem que não é Livre do Contexto):

$L = \{ww \mid w \text{ é palavra de } \{a, b\}^*\}$

pode ser gerada por um Gramática Sensível ao Contexto como segue:

$G = (\{S, X, Y, A, B, \langle aa \rangle, \langle ab \rangle, \langle ba \rangle, \langle bb \rangle\}, \{a, b\}, P, S)$, onde:

$P = \{ S \rightarrow XY \mid aa \mid bb \mid \varepsilon,$

$X \rightarrow XaA \mid XbB \mid aa\langle aa \rangle \mid ab\langle ab \rangle \mid ba\langle ba \rangle \mid bb\langle bb \rangle,$

$Aa \rightarrow aA, Ab \rightarrow bA, AY \rightarrow Ya,$

$Ba \rightarrow aB, Bb \rightarrow bB, BY \rightarrow Yb,$

$\langle aa \rangle a \rightarrow a\langle aa \rangle, \langle aa \rangle b \rightarrow b\langle aa \rangle, \langle aa \rangle Y \rightarrow aa,$

$\langle ab \rangle a \rightarrow a\langle ab \rangle, \langle ab \rangle b \rightarrow b\langle ab \rangle, \langle ab \rangle Y \rightarrow ab,$

$\langle ba \rangle a \rightarrow a\langle ba \rangle, \langle ba \rangle b \rightarrow b\langle ba \rangle, \langle ba \rangle Y \rightarrow ba,$

$\langle bb \rangle a \rightarrow a\langle bb \rangle, \langle bb \rangle b \rightarrow b\langle bb \rangle, \langle bb \rangle Y \rightarrow bb \}$

Excetuando-se para $|w| \leq 1$, a gramática apresentada gera o primeiro w após X e o segundo w após Y , como segue:

- a cada símbolo terminal gerado após X , é gerada uma variável correspondente;
- esta variável "caminha" na palavra até passar por Y , quando deriva o correspondente terminal;
- para encerrar, X deriva uma subpalavra de dois terminais e uma correspondente variável a qual "caminha" até encontrar Y quando é derivada a mesma subpalavra de dois terminais. Note-se que, se a gramática fosse modificada para X derivar uma subpalavra de somente um terminal (e a correspondente variável) como por exemplo $X \rightarrow u\langle u \rangle$, ao encontrar Y , seria necessário derivar usando uma produção do tipo $\langle u \rangle Y \rightarrow u$ onde o lado direito possui comprimento menor que o lado esquerdo e, conseqüentemente, a gramática não seria sensível ao contexto.



Máquina de Turing com Fita Limitada

Uma *Máquina de Turing com Fita Limitada*, também conhecida como *Autômato Limitado Linearmente* ou *Autômato de Fita Limitada*, é, basicamente, uma Máquina de Turing com a fita limitada ao tamanho da entrada mais duas células contendo marcadores de início e de fim de fita. Como não é conhecido se a facilidade de não-determinismo aumenta o poder computacional das Máquinas de Turing com Fita Limitada, a sua definição inclui a facilidade de não-determinismo.

Definição:

Máquina de Turing com Fita Limitada

Uma *Máquina de Turing com Fita Limitada* é uma 8-upla:

$$M = (\Sigma, Q, \delta, q_0, F, V, \clubsuit, \dagger)$$

onde:

Σ *alfabeto de símbolos de entrada*;

Q *conjunto de estados* possíveis da máquina o qual é finito;

δ *programa* ou *função de transição*:

$$\delta: Q \times (\Sigma \cup V \cup \{\clubsuit, \dagger\}) \rightarrow 2^Q \times (\Sigma \cup V \cup \{\clubsuit, \dagger\}) \times \{E, D\}$$

a qual é uma função parcial;

q_0 *estado inicial* da máquina tal que q_0 é elemento de Q ;

F *conjunto de estados finais* tal que F está contido em Q ;

V *alfabeto auxiliar* (pode ser vazio);

$\clubsuit$ *símbolo* ou *marcador de início* o qual marca o início da fita;

$\dagger$ *símbolo* ou *marcador de fim* o qual marca o fim da fita.

EXEMPLO:**Máquina de Turing com Fita Limitada**

Considere a linguagem:

$$L = \{ww \mid w \text{ é palavra de } \{a, b\}^*\}$$

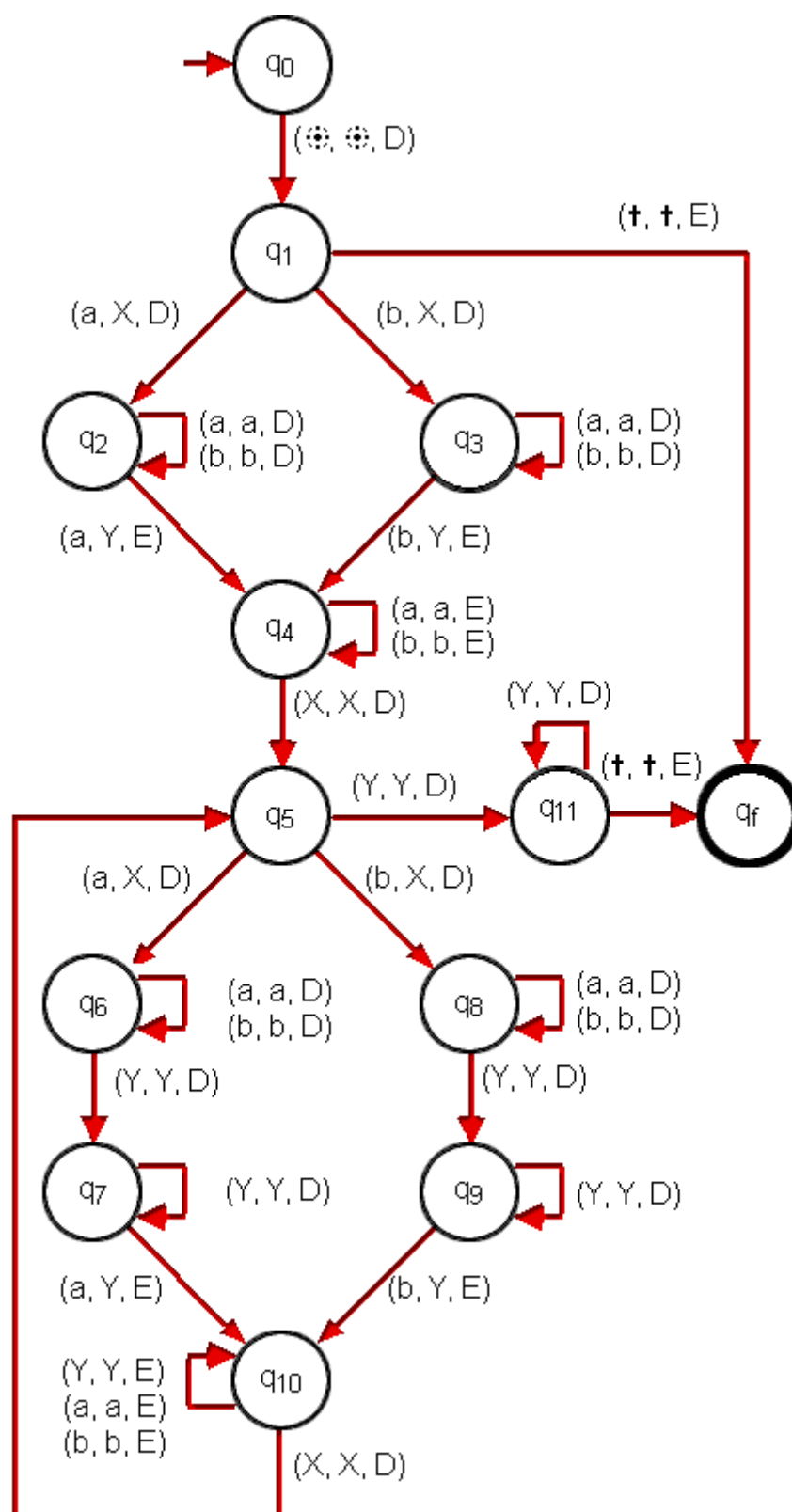


Figura Máquina de Turing com Fita Limitada

A Máquina de Turing com Fita Limitada:

$$M = (\{a, b\}, \{q_0, q_1, \dots, q_9, q_f\}, \delta, q_0, \{q_f\}, \{X, Y\}, \{\epsilon, \uparrow\})$$

ilustrada na figura acima é tal que:

$$\text{ACEITA}(M) = L$$

$$\text{REJEITA}(M) = \Sigma^* - L$$

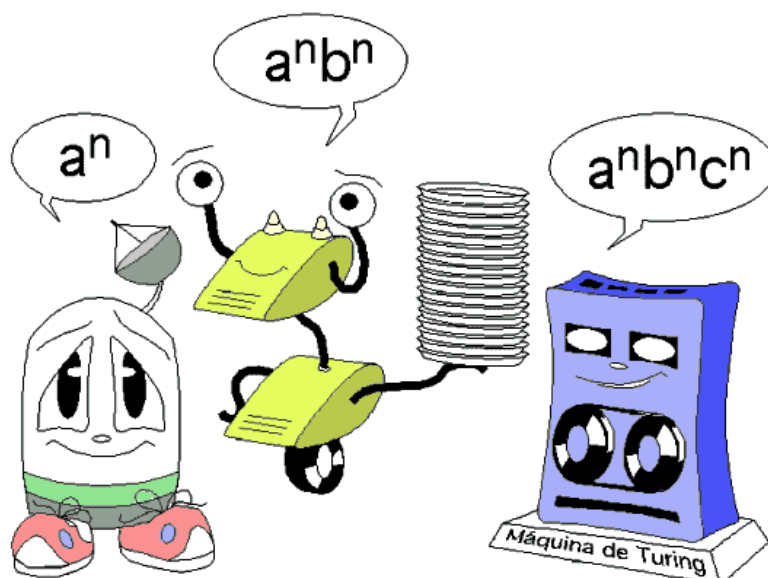
e, portanto, $\text{LOOP}(M) = \emptyset$. A partir de q_1 , o início do primeiro w é marcado com um X . Os estados q_2 e q_3 definem não-determinismos, com o objetivo de marcar com um Y o início do segundo w . De q_5 a q_{11} é verificada a igualdade da primeira com a segunda metade da palavra.

Teorema:

Linguagem Sensível ao Contexto $\times$ Máquina de Turing com Fita Limitada.

L é uma Linguagem Sensível ao Contexto se, e somente se, L é reconhecida por uma Máquina de Turing com Fita Limitada.

O teorema não será demonstrado.

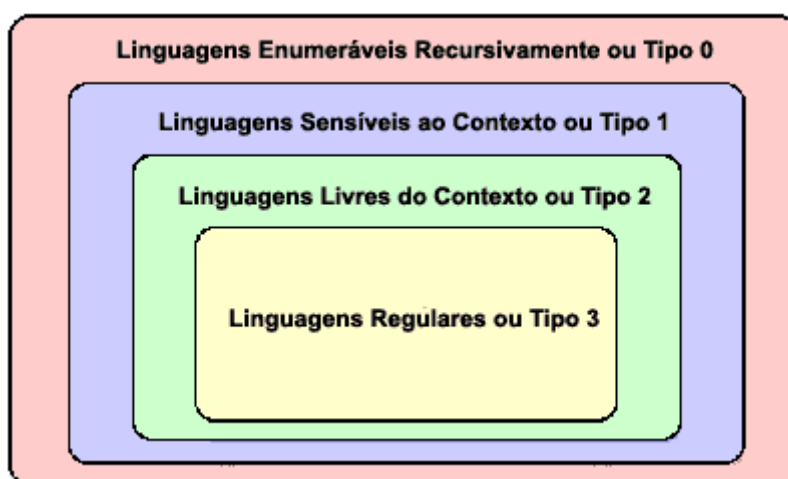


Hierarquia de Classes de Linguagens e Conclusões

As seguintes classes básicas de linguagens estudadas:

- Linguagens Regulares ou Tipos 3;
- Livres do Contexto ou Tipo 2;
- Sensíveis ao Contexto ou Tipo 1;
- Enumeráveis Recursivamente ou Tipo 0;

e as suas inclusões próprias como ilustrado na [figura abaixo](#), constituem o que normalmente é conhecido como a *Hierarquia de Chomsky*. Noam Chomsky definiu estas classes como (potenciais) modelos para linguagens naturais.



[Figura](#) Hierarquia de Chomsky

No caso específico das linguagens naturais, o estudo das Linguagens Livres do Contexto tem sido de especial interesse, pois elas permitem uma representação simples da sintaxe, adequada tanto para a estruturação formal, como para a análise computacional. Entretanto, esse estudo tem mostrado problemas não-solucionáveis,

como determinar se uma gramática é ambígua (ou seja, se existem duas ou mais árvores de derivação distintas para uma mesma palavra).

No que se refere às linguagens de programação, a Teoria das Linguagens Formais oferece meios para modelar e desenvolver ferramentas que descrevam sintaticamente a linguagem e seus processos de análise, bem como suas propriedades e limitações algorítmicas.

Entretanto, nem sempre as linguagens de programação são tratadas adequadamente na Hierarquia de Chomsky. Existem linguagens que não são Livres do Contexto, para as quais o poder dos formalismos Sensíveis ao Contexto é excessivo, sendo inadequados principalmente no que se refere à complexidade computacional (eficiência). Adicionalmente o conhecimento das Linguagens Sensíveis ao Contexto é relativamente limitado, o que dificulta o seu tratamento. Alguns exemplos de problemas que não podem ser tratados como Livres do Contexto são os seguintes:

- múltiplas ocorrências de um mesmo trecho de programa, como a declaração de um identificador e suas referências de uso (problema análogo à linguagem $\{wcw \mid w \text{ é palavra de } \{a, b\}^*\}$, a qual não é linguagem Livre do Contexto);
- alguns casos de validação de expressões com variáveis de tipos diferentes;
- a associação de um significado (semântica) de um trecho de programa, que dependeria da análise de um conjunto de informações como identificadores, ambientes, tipos de dados, localização, seqüências de operações, etc.

Nestes e em outros casos, a quantidade e o tipo de "memória" dos formalismos Livres do Contexto não são suficientes para manipular as informações necessárias para a adequada validação.

Por outro lado, para algumas linguagens de programação, a Classe das Linguagens Livres do Contexto é excessiva, e a das Regulares, insuficiente. Alguns exemplos de problemas para este tipo de linguagem são os seguintes:

- não existe um algoritmo que verifique a igualdade de duas linguagens Livres do Contexto, o que dificulta a otimização e o teste de processadores de linguagens;
- o Autômato com Pilha, em sua definição básica, possui a facilidade de não-determinismo. Entretanto, se a linguagem pode ser representada por um Autômato com Pilha Determinístico (ou seja, se é uma Linguagem Livre do Contexto Determinística), então é possível implementar (facilmente) um reconhecedor com tempo de processamento proporcional a $2n$ (onde n é o tamanho da entrada), o que é muito mais eficiente que o melhor algoritmo conhecido para as Linguagens Livres do Contexto.

Alguns problemas referentes à Teoria das Linguagens Formais, embora atuais, possuem muitas questões em aberto, como:

- o tratamento de linguagens n -dimensionais, com destaque para as bi e as tri-dimensionais, com aplicações no processamento de imagens no plano e no espaço;
- a tradução de linguagens, tanto naturais como de programação.

Como ilustração de uma abordagem das Linguagens Formais às linguagens n-dimensionais, no que segue é introduzida a noção de *Gramática de Grafos*.

Gramáticas de Grafos podem ser usadas para definir linguagens baseadas em grafos, bem como para descrever e analisar sistemas seqüenciais, concorrentes e distribuídos, avaliar expressões funcionais, definir mecanismos de sincronização, modelar sistemas biológicos, gerar imagens (como, por exemplo, fractais), etc.

A idéia básica das Gramáticas de Grafos é análoga à das de Gramáticas de Chomsky, ou seja:

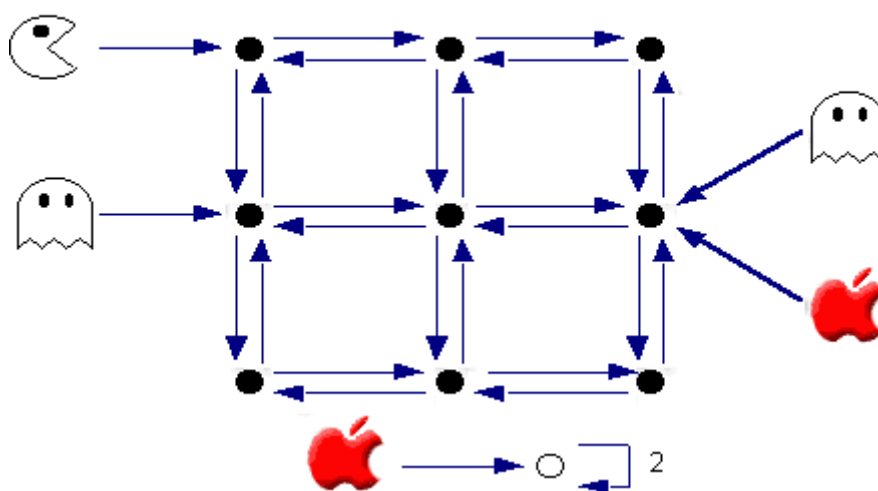


Figura Grafo do jogo PacMan

- regras de produção são pares (mas de grafos);
- uma derivação é a substituição de um subgrafo de acordo com uma regra de produção.

As Gramáticas de Grafos são normalmente tratadas em um contexto categorial (nenhum conceito de Teoria das Categorias é formalmente introduzido nesta publicação). Informalmente, alguns conceitos podem ser visualizados no seguinte exemplo o qual ilustra o jogo de videogame PacMan.

O jogo PacMan (simplificado) possui um tabuleiro juntamente com algumas entidades especiais como um PacMan e dois conjuntos, um de fantasmas e outro de maçãs. A [Figura acima](#) ilustra um grafo ("palavra") da linguagem onde:

- os nodos pretos, representam os lugares do tabuleiro e as correspondentes arestas, os caminhos possíveis entre dois lugares;
- as entidades PacMan, fantasmas e maçãs são representadas por nodos com simbologia própria, onde os correspondentes arcos denotam o seu posicionamento no tabuleiro;
- o nodo branco descreve a fase em que se encontra o jogo (na [figura acima](#), a fase é 2). Adicionalmente, as maçãs cujos arcos são associados ao nodo branco, representam as maçãs já comidas pelo PacMan (na [figura acima](#), uma maçã foi comida).

<>Figura 5.3 ilustra as regras de produção **move**, **come** e **mata**. Por fim, a **figura abaixo** ilustra o grafo resultante da aplicação da regra **move** ao grafo ilustrado na **figura acima**. Note-se que, no grafo resultante, o PacMan encontra-se em um lugar onde existe um fantasma. Assim, a próxima produção pode ser **mata** ou **move**.

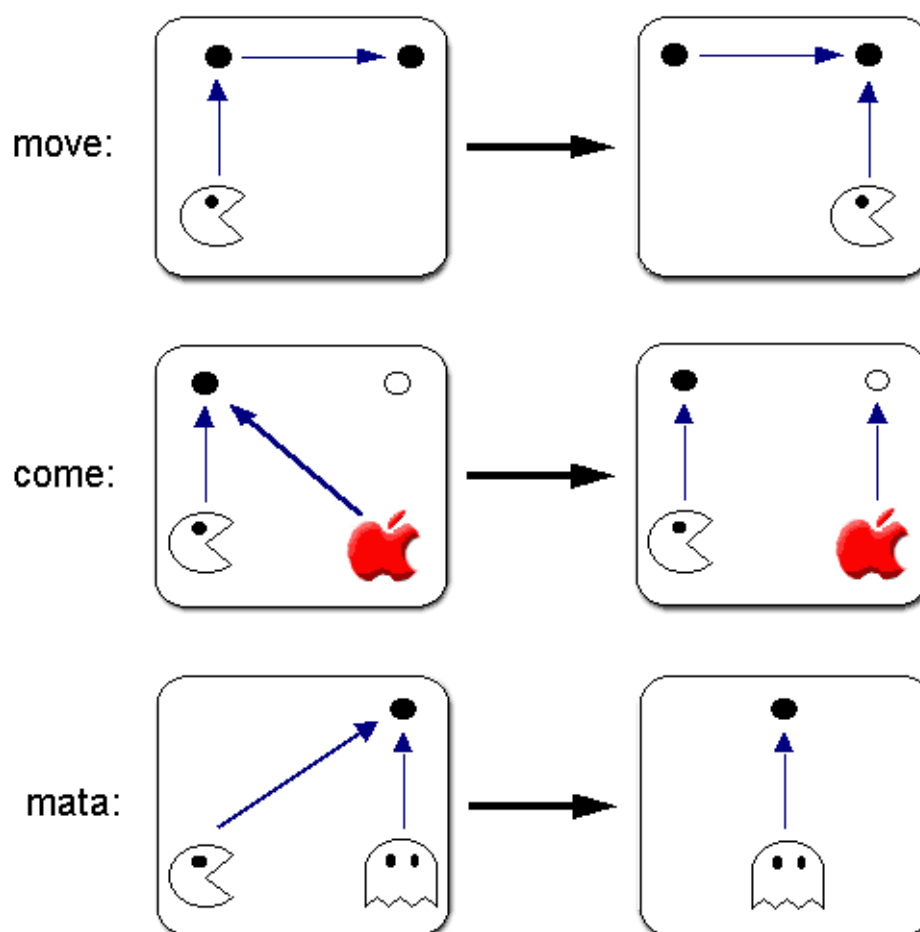


Figura Regras de produção do PacMan

Assim, dependendo da linguagem e dos objetivos do trabalho, alguns estudos específicos, eventualmente fora da Hierarquia de Chomsky, são recomendados ou necessários. Estes transcendem o objetivo desta publicação e algumas referências sobre o assunto podem ser encontradas na bibliografia recomendada.

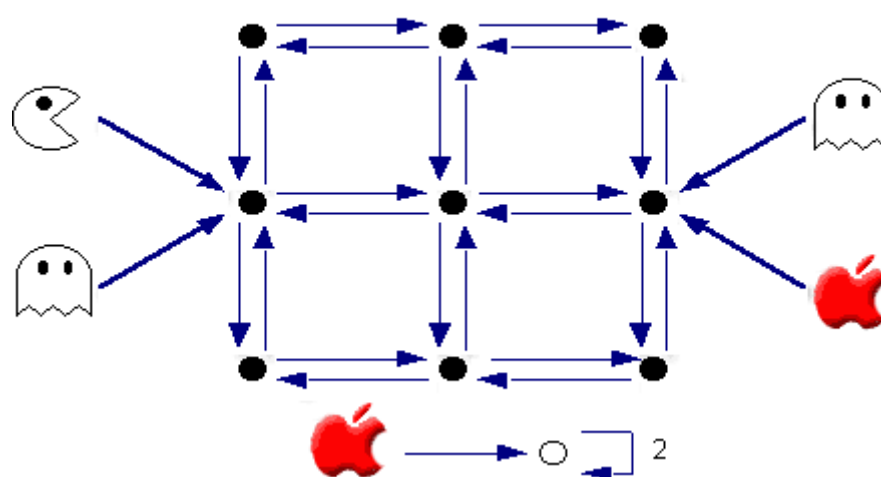


Figura Grafo resultante da aplicação da regra *move*